# Chapter 2

# Programming in Python

Python is a simple, high level language with a clean syntax. It offers strong support for integration with other languages and tools, comes with extensive standard libraries, and can be learned within few days. Many Python programmers report substantial productivity gains and feel the language encourages the development of higher quality, more maintainable code. To know more visit the Python website.[1]

## 2.1 Getting started with Python

To start programming in Python, we have to learn how to type the source code and save it to a file, using a text editor program. We also need to know how to open a Command Terminal and start the Python Interpreter. The details of this process may vary from one system to another. On an Ubuntu GNU/Linux system, you can open the *Text Editor* and the *Terminal* from the Applications->Accessories menu. You may also choose an Integrated Development Environment. IDEs allows you to edit and run the program, and also has features for managing projects with multiple files. 'Geany'[2] is a simple and lightweight IDE.

The current version of Python differs from the earlier version in some ways. All the examples given in this book are tested on python3. In case, if you do not have any computer with Python installed try online[3]. Another option is to use Android apps like Pydroid 3 ( for Pydroid 3, you also need to install the 'pydroid repository plugin' inorder to enable the installation of Python modules like Numpy, matplotlib etc.)

### 2.1.1 Two modes of using Python Interpreter

If you issue the command 'python3', without any argument, from the command terminal, the Python interpreter will start and display a '>>>' prompt where you can type Python statements. Use this method only for viewing the results of single Python statements, for example to use Python as a calculator. It could be confusing when you start writing larger programs, having looping and conditional statements. The preferred way is to enter your source code in a text editor, save it to a file (with .py extension) and execute it from the command terminal using Python. A screen-shot of the Desktop with Text Editor and Terminal is shown in figure 2.2.

[1] http://docs.python.org/tutorial/
[2] sudo apt install geany
[3] https://trinket.io/python3

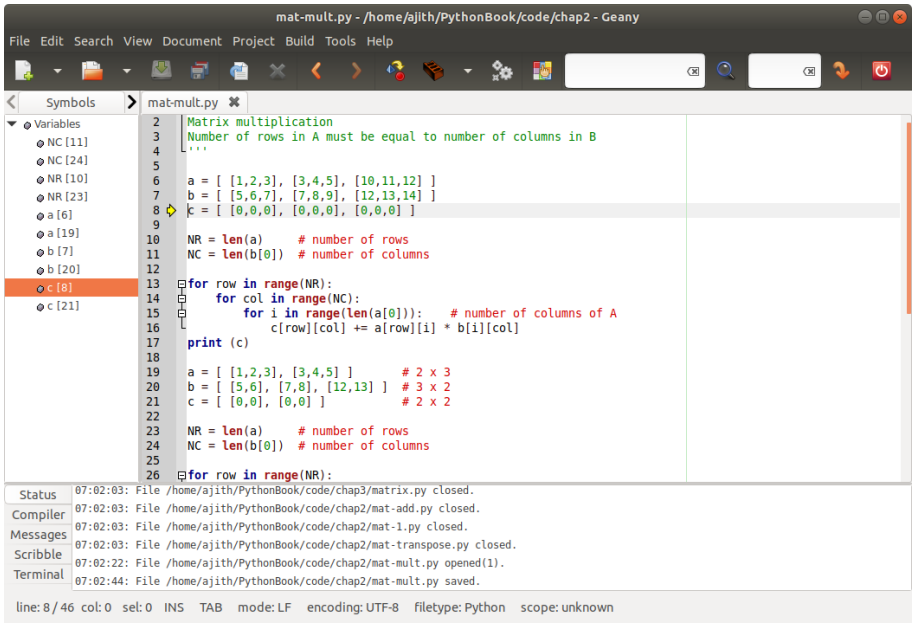



Figure 2.1: Geany IDE

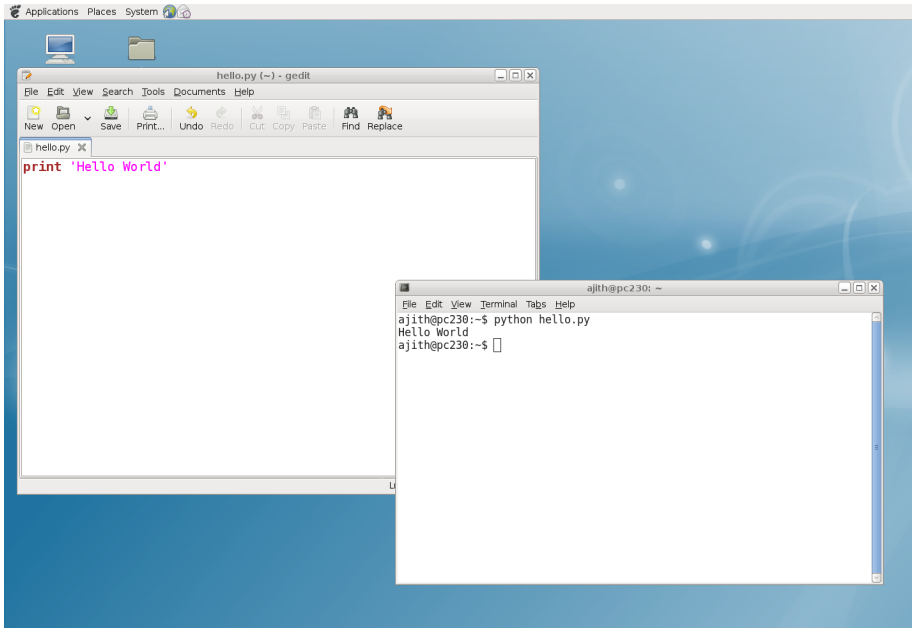



Figure 2.2: Text Editor and Terminal Windows.

In this document, we will start writing small programs showing the essential elements of the language without going into the details. The reader is expected to run these example programs and also to modify them in different ways.It is like learning to drive a car, you master it by practicing.

Let us start with a program to display the words *Hello World* on the computer screen. This is the customary 'hello world' program. There is another version that prints 'Goodbye cruel world', probably invented by those who give up at this point! The Python 'hello world' program is shown below.

*Example. hello.py*

```
print ('Hello World')
```

This should be entered into a text file using any text editor. On a GNU/Linux system you may use the text editor like 'gedit' to create the source file, save it as **hello.py** . The next step is to call the Python Interpreter to execute the new program. For that, open a command terminal and (at the $ prompt) type:

```
$ python3 hello.py
```

The filenames of python programs should not be any of the the keywords of python. For example, naming a file as 'string.py' or 'int.py' may result in hard to track error messages.

## 2.2 Variables and Data Types

As mentioned earlier, any high level programming language should support several data types. The problem to be solved is represented using variables belonging to the supported data types. Python supports numeric data types like integers, floating point numbers and complex numbers. To handle character strings, it uses the String data type. Python also supports other data types like lists, tuples, dictionaries etc.

In languages like C, C++ and Java, we need to explicitly declare the type of a variable. This is not required in Python. The data type of a variable is decided by the value assigned to it. This is called dynamic data typing. The type of a particular variable can change during the execution of the program. If required, one type of variable can be converted in to another type by explicit type casting, like $y = float(3)$. Strings are enclosed within single quotes or double quotes.

The program **first.py** shows how to define variables of different data types. It also shows how to embed comments inside a program.

*Example: first.py*

```
'''
A multi-line comment, within a pair of three single quotes.
In a line, anything after a # sign is also a comment
'''
x = 10
print (x, type(x))        # print x and its type
x = 10.4
print (x, type(x))
x = 3 + 4j
print (x, type(x))
x = 'I am a String '
print (x,  type(x))
```

The output of the program is shown below. Note that the type of the variable $x$ changes during the execution of the program, depending on the value assigned to it.

```
10 <type 'int'>
10.4 <type 'float'>
(3+4j) <type 'complex'>
I am a String  <type 'str'>
```

The program treats the variables like humans treat labelled envelopes. We can pick an envelope, write some name on it and keep something inside it for future use. In a similar manner the program creates a variable, gives it a name and keeps some value inside it, to be used in subsequent steps. So far we have used four data types of Python: int, float, complex and str. To become familiar with them, you may write simple programs performing arithmetic and logical operations using them.

*Example: oper.py*

```
x = 2
y = 4
print(x + y * 2)
s = 'Hello '
print(s + s)
print(3 * s)
print(x == y)
print(y == 2 * x)
print(5/2 , 5//2)
```

Running the program **oper.py** will generate the following output.

```
10
Hello Hello
Hello Hello Hello
False
True
2.5  2
```

Note that a String can be added to another string and it can be multiplied by an integer. Try to understand the logic behind that and also try adding a String to an Integer to see what is the error message you will get. We have used the logical operator == for comparing two variables. The last line demonstrates the nature of division operator on python3. By default it does floating point division. Integer division can be forced by the operator //.

## 2.3 Operators and their Precedence

Python supports a large number of arithmetic and logical operators. They are summarized in the table 2.1. An important thing to remember is their precedence. In the expression *2+3*4*, is the addition done first or the multiplication? According to elementary arithmetics, the multiplication should be done first. It means that the multiplication operator has higher precedence than the addition operator. If you want the addition to be done first, enforce it by using parenthesis like $(2 + 3) * 4$. Whenever there is ambiguity in evaluation, use parenthesis to clarify the order of evaluation.

| Operator | Description | Expression | Result |
|---|---|---|---|
| **or** | Boolean OR | 0 or 4 | 4 |
| **and** | Boolean AND | 3 and 0 | 0 |
| **not x** | Boolean NOT | not 0 | True |
| **in, not in** | Membership tests | 3 in [2.2,3,12] | True |
| <, <=, >, >=, !=, == | Comparisons | 2 > 3 | False |
| \| | Bitwise OR | 1 \| 2 | 3 |
| ^ | Bitwise XOR | 1 ^ 5 | 4 |
| & | Bitwise AND | 1 & 3 | 1 |
| <<, >> | Bitwise Shifting | 1 << 3 | 8 |
| + , - | Add, Subtract | 6 - 4 | 2 |
| *, / | Multiply, divide | 5 / 2 | 2.5 |
| % | reminder | 5%2 | 1 |
| // | integer division | 3/2 | 1 |
| +x , -x | Positive, Negative | -5*2 | -10 |
| ~ | Bitwise NOT | ~1 | -2 |
| ** | Exponentiation | 2 ** 3 | 8 |
| **x[index]** | Subscription | a='abcd' ; a[1] | 'b' |

Table 2.1: Operators in Python listed according to their precedence.

## 2.4 Python Strings

So far we have come across four data types: Integer, Float, Complex and String. Out of which, String is somewhat different from the other three. It is a sequence of same kind of elements, characters. The individual elements of a String can be accessed by indexing as shown in *string1.py*. String is a compound, or collection, data type.

*Example: string1.py*

```
s = 'hello world'
print(s[0])           # print first element, h
print(s[1])           # print e
print(s[-1])          # will print the last character
```

Addition and multiplication is defined for Strings, as demonstrated by string2.py.

*Example: string2.py*

```
a = 'hello'+'world'
print a
b = 'ha' * 3
print b
print a[-1] + b[0]
```

will give the output

```
helloworld
hahaha
dh
```

The last element of $a$ and first element of $b$ are added, resulting in the string 'dh'

### 2.4.1 Slicing

Part of a String can be extracted using the slicing operation. It can be considered as a modified form of indexing a single character. Indexing using $s[a:b]$ extracts elements $s[a]$ to $s[b-1]$. We can skip one of the indices. If the index on the left side of the colon is skipped, slicing starts from the first element and if the index on right side is skipped, slicing ends with the last element.

*Example: slice.py*

```
a = 'hello world'
print(a[3:5])
print(a[6:])
print(a[:5])
```

The reader can guess the nature of slicing operation from the output of this code, shown below.

```
'lo'
'world'
'hello'
```

Please note that specifying a right side index more than the length of the string is equivalent to skipping it. Modify **slice.py** to print the result of $a[6:20]$ to demonstrate it.

## 2.5 Python Lists

List is an important data type of Python. It is much more flexible than String. The individual elements can be of any type, even another list. Lists are defined by enclosing the elements inside a pair of square brackets, separated by commas. The program **list1.py** defines a list and print its elements.

*Example: list1.py*

```
a = [2.3, 3.5, 234]   # make a list
print (a[0])
a[1] = 'haha'       # Change an element
print (a)
```

The output is shown below.

```
2.3
[2.3, 'haha', 234]
```

Lists can be sliced in a manner similar to that if Strings. List addition and multiplication are demonstrated by the following example. We can also have another list as an element of a list.

*Example: list2.py*

```
a = [1,2]
print (a * 2)
print (a + [3,4])
b = [10, 20, a]
print (b)
```

The output of this program is shown below.

```
[1, 2, 1, 2]
[1, 2, 3, 4]
[10, 20, [1, 2] ]
```

## 2.6 Mutable and Immutable Types

There is one major difference between String and List types, List is mutable but String is not. *We can change the value of an element in a list, add new elements to it and remove any existing element. This is not possible with String type.* Uncomment the last line of **third.py** and run it to clarify this point.

*Example: third.py*

```
s = [3, 3.5, 234]   # make a list
s[2] = 'haha'       # Change an element
print s
x = 'myname'        # String type
#x[1] = 2           # uncomment to get ERROR
```

The List data type is very flexible, an element of a list can be another list. We will be using lists extensively in the coming chapters. Tuple is another data type similar to List, except that it is immutable. List is defined inside square brackets, tuple is defined in a similar manner but inside parenthesis, like (3, 3.5, 24).

## 2.7 Input from the Keyboard

Since most of the programs require some input from the user, let us introduce this feature before proceeding further. In python3, the *input()* function gets String type data from the keyboard, that can be converted to integer, float or complex by using eval() function. A message to be displayed can be given as an argument while calling these functions.[4]

*Example: kin1.py*

```
x = eval(input('Enter an integer '))
y = complex(input('Enter a complex number, (like 2+5j)  '))
print ('The sum is ', x + y)
s = input('Enter a String ')
print ('You entered ', s)
```

It is also possible to read more than one variable using a single input() statement. *String* type data read using input() may be converted into *integer* or *float* type if they contain only the valid characters. In order to show the effect of conversion explicitly, we multiply the variables by 2 before printing. Multiplying a String by 2 prints it twice. If the String contains invalid characters then eval() will give error.

*Example: kin2.py*

```
x,y = eval(input('Enter x and y separated by comma '))
print 'The sum is ', x + y
s = input('Enter a decimal number ')
a = float(s)
print s * 2    # prints string twice
print a * 2    # converted value times 2
```

[4] Functions will be introduced later. For the time being, understand that it is an isolated piece of code, called from the main program with some input arguments and returns some output.

We have learned about the basic data types of Python and how to get input data from the keyboard. This is enough to try some simple problems and algorithms to solve them.

*Example:* area.py

```
pi = 3.1416
r = input('Enter Radius ')
a = pi * r ** 2       # A = πr²
print ('Area = ', a)
```

The above example calculates the area of a circle. Line three calculates $r^2$ using the exponentiation operator $**$, and multiply it with $\pi$ using the multiplication operator $*$. $r^2$ is evaluated first because ** has higher precedence than *, otherwise the result would be $(\pi r)^2$.

## 2.8 Python Syntax, Colon & Indentation

Python was designed to be a highly readable language. It has a relatively uncluttered visual layout, uses English keywords frequently where other languages use punctuation, and has notably fewer syntactic constructions than other popular structured languages.

There are mainly two things to remember about Python syntax: indentation and colon. **Python uses indentation to delimit blocks of code.** Both space characters and tab characters are currently accepted as forms of indentation in Python. **Mixing spaces and tabs can create bugs that are hard to track**, since the text editor does not show the difference. There should not be any extra white spaces in the beginning of any line.

**The line before any indented block must end with a colon** (:) **character.**

## 2.9 Controlling the Program Flow

If programs can only execute from the first line to the last in that order, as shown in the previous examples, it would be impossible to write any useful program. There are two types of control flow statements; Iteration (or loops) and Conditional execution. Python has two keywords for implementing loops, **while** and **for**. Conditional execution is achieved using **if, elif** and **else** keywords.

### 2.9.1 Iteration: while loops

For example, we need to print the multiplication table of eight. Using our present knowledge, it would look like the following

*Example:* badtable.py

```
print (1 * 8)
print (2 * 8)
print (3 * 8)
print (4 * 8)
print (5 * 8)
```

Well, we are stopping here and looking for a better way to do this job.

The solution is to use the *while* loop of Python. The logical expression in front of *while* is evaluated, and if it is True, the body of the while loop (the indented lines below the while

statement) is executed. The process is repeated until the condition becomes false. We should have some statement inside the body of the loop that will make this condition false after few iterations. Otherwise the program will run in an infinite loop and you will have to press Control-C to terminate it.

The program *table.py*, defines a variable $x$ and assigns it an initial value of 1. Inside the while loop $x * 8$ is printed and the value of $x$ is incremented. This process will be repeated until the value of $x$ becomes greater than 10.

*Example:* table.py

```
x = 1
while x <= 10:
    print (x * 8)
    x = x + 1
```

As per the Python syntax, the while statement ends with a colon and the lines inside the *while* loop are indented. Indentation can be done using tab or few spaces. In this example, we have demonstrated a simple algorithm.

## 2.10 Iteration: for loops

Python *for* loops are slightly different from the for loops of other languages. Python *for* loop iterates over a sequence data type like a String, List or Tuple. During each iteration, one member of the data is assigned to the loop variable. The flexibility of this can be seen from the examples below.

*Example:* forloop.py

```
a = 'Hello'
for ch in a:  # ch is the loop variable
    print (ch)
b = ['haha', 3.4, 2345, 3+5j]
for item in b:
    print (item)
```

which gives the output :

H
e
l
l
o
haha
3.4
2345
(3+5j)

For constructing for loops that executes a fixed number of times, we can use the range() function and run the for loop over that.

*Example:* forloop2.py

```
for item in mylist:
    print item
```

The output will look like :

0
1
2
3
4

In python3 the output of range() function is an object belonging to a class called range. It is possible to specify the starting point and increment as arguments in the form range(start, end+1, step). The following example prints the table of 5 using this feature.

*Example:* forloop3.py

```
mylist = range(5,51,5)
for item in mylist:
    print (item , end = ' ')
```

The output is shown below.

5 10 15 20 25 30 35 40 45 50

The print statement inserts a newline at the end by default. The end = ' ' argument changes this to a space.

In some cases, we may need to traverse the list to modify some or all of the elements. This can be done by looping over a list of indices generated by the range() function.For example, the program **forloop4.py** zeros all the elements of the list.

*Example:* forloop4.py

```
a = [2, 5, 3, 4, 12]
size = len(a)
for k in range(size):
    a[k] = 0
print a
```

The output is

```
[0, 0, 0, 0, 0]
```

## 2.11 Conditional Execution: if, elif and else

In some cases, we may need to execute some section of the code only if certain conditions are true. Python implements this feature using the *if, elif* and *else* keywords, as shown in the next example. The indentation levels of *if* and the corresponding *elif* and *else* must be kept the same.

*Example:* compare.py

```
x = input('Enter a string ')
if x == 'hello':
    print ('You typed ', x)
```

*Example:* big.py

```
x = eval(input('Enter a number '))
if x > 10:
    print ('Bigger Number')
elif x < 10:
    print ('Smaller Number')
else:
    print ('Same Number')
```

The statement *x > 10 and x < 15* can be expressed in a short form, like *10 < x < 15*.

The next example uses *while* and *if* keywords in the same program. Note the level of indentation when the if statement comes inside the while loop. Remember that, the *if* statement must be aligned with the corresponding *elif and else*.

*Example:* big2.py

```
x = 1
while x < 11:
    if x < 5:
        print ('Small ', x)
    else:
        print ('Big ', x)
    x = x + 1
print ('Done')
```

## 2.12 Modify loops : break and continue

We can use the *break* statement to terminate a loop, if some condition is met. The *continue* statement is used to skip the rest of the block and go to the beginning again. Both are demonstrated in the program **big3.py** shown below.

*Example:* big3.py

```
x = 1
while x < 10:
   if x < 3:
       print ('skipping work', x)
       x = x + 1
       continue
   print (x)
   if x == 4:
      print ('Enough of work')
      break
   x = x  + 1
print ('Done')
```

The output of big3.py is listed below.

skipping work 1
skipping work 2
3
4

Enough of work
Done

Now let us write a program to find out the largest positive number entered by the user. The algorithm works in the following manner. To start with, we assume that the largest number is zero. After reading a number, the program checks whether it is bigger than the current value of the largest number. If so the value of the largest number is replaced with the current number. The program terminates when the user enters zero. Modify **max.py** to work with negative numbers also.

Example: max.py

```
max = 0
while True:     # Infinite loop
   x = eval(input('Enter a number '))
   if x > max:
      max = x
   if x == 0:
      print (max)
      break
```

## 2.13 Line joining

Python interpreter processes the code line by line. A program may have a long line of code that may not physically fit in the width of the text editor. In such cases, we can split a logical line of code into more than one physical lines, using backslash characters (\), in other words multiple physical lines are joined to form a logical line before interpreting it.

```
if 1900 < year < 2100 and 1 <= month <= 12 :
```

can be split like

```
if 1900 < year < 2100 \
    and 1 <= month <= 12 :
```

Do not split in the middle of words except for Strings. A long String can be split as shown below.

```
longname = 'I am so long and will \
not fit in a single line'
print (longname)
```

## 2.14 Exercises

We have now covered the minimum essentials of Python; defining variables, performing arithmetic and logical operations on them, user input from keyboard and the control flow statements. It would be better to get a grip of it before proceeding further, by writing some code. Writing a program to solve a problem involves two steps:

- Developing an algorithm to solve the problem
- implement it in some programming language

Some problems given below requires the development of suitable algorithms, ie. breaking down the problem in to sequence of simple steps.

1. Modify the expression *print 5+3*2* to get a result of 16
2. Print all even numbers upto 30 (hint: use the reminder ( % ) operator)
3. Write Python code to remove the last two characters of 'I am a long string' by slicing, without counting the characters. (hint: use negative indexing)
4. s = '012345' . (a) Slice it to remove last two elements (b) remove first two element.
5. a = [1,2,3,4,5]. Use Slicing and multiplication to generate [2,3,4,2,3,4] from it.
6. Compare the results of 5/2, 5//2 and 2.0/3.
7. Print the following pattern using a while loop
   +
   ++
   +++
   ++++
8. Print the following pattern using a for loop.
   +
   +++
   +++++
9. Write code to print a number in the binary format (for example 5 will be printed as 101)
10. Write code to print all perfect cubes upto 2000.
11. Write a Python program to print the multiplication table of 5.
12. Write a program to find the volume of a box with sides 3,4 and 5 inches in $cm^3$( 1 inch = 2.54 cm)
13. Write a program to find the percentage of volume occupied by a sphere of diameter $r$ fitted in a cube of side $r$. Read $r$ from the keyboard.
14. Write a Python program to calculate the area of a circle.
15. Write a program to divide an integer by another without using the / operator. (hint: use - operator)
16. Count the number of times the character 'a' appears in a String read from the keyboard. Keep on prompting for the string until there is no 'a' in the input.
17. Create an adding machine that will keep on asking the user for numbers, add them together and show the total after each step. Terminate when user enters a zero.
18. Modify the adding machine to check for errors like user entering invalid characters.
19. Create a script that will convert Celsius to Fahrenheit. The program should ask the users to enter the temperature in Celsius and should print out the temperature in Fahrenheit, using $f = \frac{9}{5}c + 32$.
20. Write a program to convert Fahrenheit to Celsius.

21. Define $2 + 5j$ and $2 - 5j$ as complex numbers , and find their product. Verify the result by defining the real and imaginary parts separately and using the multiplication formula.

22. Write the multiplication table of a number, from the user, using for loop.

23. Print the powers of 2 up to 1024 using a for loop. (only two lines of code)

24. Define the list a = [123, 12.4, 'haha', 3.4]
    a) print all members using a for loop
    b) print the float type members ( use type() function)
    c) insert a member after 12.4
    d) append more members

25. Make a list containing 10 members using a for loop.

26. Generate multiplication table of 5 with two lines of Python code. (hint: range function)

27. Write a program to find the sum of five numbers read from the keyboard.

28. Write a program to read numbers from the keyboard until their sum exceeds 200. Modify the program to ignore numbers greater than 99.

## 2.15 Functions

Large programs need to be divided into small logical units. A function is generally an isolated unit of code that has a name and performs a well defined job. A function groups several program statements into a unit and gives it a name. This unit can be invoked from other parts of a program. Python allows you to define functions using the *def* keyword. A function may have one or more variables as parameters, which receive their values from the calling program.

In the example shown below, function parameters (a and b) get the values 3 and 4 respectively from the caller. One can specify more than one variables in the return statement, separated by commas. The function will return a tuple containing those variables. Some functions may not have any arguments, but while calling them we need to use an empty parenthesis, otherwise the function will not be invoked. If there is no return statement, a None is returned to the caller.

*Example func.py*

```
def sum(a,b):    # a trivial function
    return a + b

print (sum(3, 4))
```

The function **factorial.py** calls itself recursively. The value of argument is decremented before each call. Try to understand the working of this by inserting print statements inside the function.

*Example factor.py*

```
def factorial(n): # a recursive function
    if n == 0:
         return 1
    else:
         return n * factorial(n-1)

print (factorial(10))
```

*Example* ***fibonacci.py***

```
def fib(n): # print Fibonacci series up to n
   a, b = 0, 1
   while b < n:
      print (b)
      a, b = b, a+b

print fib(30)
```

Runing *fibonacci.py* will print

1 1 2 3 5 8 13 21

Modify the code to replace $a, b = b, a + b$ by two separate assignment statements, if required introduce a third variable.

### 2.15.1 Scope of variables

The variables defined inside a function are not known outside the function. There could be two unrelated variables, one inside and one outside, having the same name. The program **scope.py** demonstrates this feature.

*Example scope.py*

```
def change(x):
   counter = x

counter = 10
change(5)
print (counter)
```

The program will print 10 and not 5. The two variables, both named counter, are not related to each other. In some cases, it may be desirable to allow the function to change some external variable. This can be achieved by using the *global* keyword, as shown in *global.py*.

*Example global.py*

```
def change(x):
   global counter  # use the global variable
   counter = x

counter = 10
change(5)
print (counter)
```

The program will now print 5. Functions with global variables should be used carefully, to avoid inadvertent side effects.

### 2.15.2 Optional and Named Arguments

Python allows function arguments to have default values; if the function is called without a particular argument, its default value will be taken. Due to this feature, the same function can be called with different number of arguments. The arguments without default values must appear first in the argument list and they cannot be omitted while invoking the function. The following example shows a function named power() that does exponentiation, but the default value of exponent is set to 2.

*Example power.py*

```
def power(mant, exp = 2.0):
    return mant ** exp

print (power(5., 3))
print (power(4.))      # prints 16
print (power())        # Gives Error
```

Arguments can be specified in any order by using named arguments.

*Example named.py*

```
def power(mant = 10.0, exp = 2.0):
    return mant ** exp

print (power(5., 3))
print (power(4.))      # prints 16
print (power(exp=3))   # mant gets 10.0, prints 1000
```

## 2.16 More on Strings and Lists

Before proceeding further, we will explore some of the functions provided for manipulating strings and lists. Python strings can be manipulated in many ways. The following program prints the length of a string, makes an upper case version for printing and prints a help message on the String class.

*Example: stringhelp.py*

```
s = 'hello world'
print (len(s))
print (s.upper())
help(str)          # press q to exit help
```

Python is an object oriented language and all variables are objects belonging to various classes. The method upper() (a function belonging to a class is called a method) is invoked using the dot operator. All we need to know at this stage is that there are several methods that can be used for manipulating objects and they can be invoked like: *variable_name.method_name()*.

### 2.16.1 split and join

Splitting a String will result in a list of smaller strings. If you do not specify the separator, the space character is assumed by default. To demonstrate the working of these functions, few lines of code and its output are listed below.

*Example: split.py*

```
s = 'I am a long string'
print (s.split())
a = 'abc.abc.abc'
aa = a.split('.')
print (aa)
mm = '+'.join(aa)
print (mm)
```

The result is shown below

['I', 'am', 'a', 'long', 'string']
['abc', 'abc', 'abc']
'abc+abc+abc'

The List of strings generated by split is joined using '+' character, resulting in the last line of the output.

### 2.16.2 Manipulating Lists

Python lists are very flexible, we can append, insert, delete and modify elements of a list. The program **list3.py** demonstrates some of them.

*Example: list3.py*

```
a = []          # make an empty list
a.append(3)     # Add an element
a.insert(0,2.5) # insert 2.5 as first element
print (a, a[0])
print (len(a))
```

The output is shown below.
[2.5, 3] 2.5
2

### 2.16.3 Copying Lists

Lists cannot be copied like numeric data types. The statement $b = a$ will not create a new list b from list a, it just make a reference to a. The following example will clarify this point. To make a duplicate copy of a list, we need to use the *copy* module.

*Example: list_ copy.py*

```
a = [1,2,3,4]
print (a)
b = a        # b refers to a
print (a == b) # True
b[0] = 5     # Modifies a[0]
print (a)
import copy
c = copy.copy(a)
c[1] = 100
print (a is c)  # is False
print (a, c)
```

The output is shown below.

[1, 2, 3, 4]
True
[5, 2, 3, 4]
False
[5, 2, 3, 4] [5, 100, 3, 4]

## 2.17 Python Modules and Packages

One of the major advantages of Python is the availability of libraries for various applications like graphics, networking and scientific computation. The standard library distributed with Python itself has a large number of modules: time, random, sys, etc. are some of them. The site http://docs.python.org/library/ has the complete reference.

Modules are loaded by using the *import* keyword. Several ways of using *import* is explained below, using the math (containing mathematical functions) module as an example.[5]

### 2.17.1 Different ways to import

simplest way to use import is shown in *mathsin.py*, where the function is invoked using the form *module_name.function_name()*. In the next example, we use an alias for the module name.

*Example mathsin.py*

```
import math
print (math.sin(0.5)) # module_name.method_name
```

*Example mathsin2.py*

```
import math as m   # Give another name for math
print (m.sin(0.5))   # Refer by the new name
```

We can also import the functions to behave like local (like the ones within our source file) function, as shown below. The character * is a wild card for importing all the functions.

[5] While giving names to your Python programs, make sure that you are not directly or indirectly importing any Python module having same name. For example, if you create a program named *math.py* and keep it in your working directory, the *import math* statement from any other program started from that directory will try to import your file named *math.py* and give error. If you ever do that by mistake, delete all the files with .pyc extension from your directory.

*Example mathlocal.py*

```
from math import sin  # sin is imported as local
print (sin(0.5))
```

*Example mathlocal2.py*

```
from math import *   # import everything from math
print (sin(0.5))
```

In the third and fourth cases, we need not type the module name every time. But there could be trouble if two modules imported contains a function with same name. In the program *conflict.py*, the sin() from *numpy* is capable of handling a list argument. After importing *math.py*, line 4, the sin function from *math* module replaces the one from *numpy*. The error occurs because the sin() from *math* can accept only a numeric type argument.

*Example* ***conflict.py***

```
from numpy import *
x = [0.1, 0.2, 0.3]
print (sin(x))         # numpy's sin can handle lists
from math import *     # sin of math becomes effective
print (sin(x))         # will give ERROR
```

### 2.17.2 Packages

Packages are used for organizing multiple modules. The module name A.B designates a module named B in a package named A. The concept is demonstrated using the packages Numpy[6] and Scipy.

*Example* ***submodule.py***

```
import numpy as np
print (np.random.normal())
import scipy.special
print (scipy.special.j0(.1))
```

In this example *random* is a module inside the package *NumPy*. Similarly *special* is a module inside the package *Scipy*. We use both of them in the package.module.function() format. But there is some difference. In the case of Numpy, the random module is loaded by default, importing scipy does not import the module special by default. This behavior can be defined while writing the Package and it is up to the author of the package.

## 2.18 File Input/Output

Disk files can be opened using the function named open() that returns a File object. Files can be opened for reading or writing. There are several methods belonging to the File class that can be used for reading and writing data.

*Example* ***wfile.py***

[6] NumPy will be discusssed later in chapter 3.

```
f = open('test.dat' , 'w')
f.write ('This is a test file')
f.close()
```

Above program creates a new file named 'test.dat' (any existing file with the same name will be deleted) and writes a String to it. The following program opens this file for reading the data.

*Example rfile.py*

```
f = open('test.dat' , 'r')
print (f.read())
f.close()
```

Note that the data written/read are character strings. read() function can also be used to read a fixed number of characters, as shown below.

*Example rfile2.py*

```
f = open('test.dat' , 'r')
print (f.read(7))      # get first seven characters
print (f.read())       # get the remaining ones
f.close()
```

Now we will examine how to read a text data from a file and convert it into numeric type. First we will create a file with a column of numbers.

*Example* ***wfile2.py***

```
f = open('data.dat' , 'w')
for k in range(1,4):
      s = '%3d\n' %(k)
      f.write(s)
f.close()
```

The contents of the file created will look like this.

```
1
2
3
```

Now we write a program to read this file, line by line, and convert the string type data to integer type, and print the numbers.[7]

*Example rfile3.py*

```
f = open('data.dat' , 'r')
while 1: # infinite loop
    s = f.readline()
    if s == '' :    # Empty string means end of file
         break      # terminate the loop
    m = eval(s)     # Convert to integer
    print (m * 5)
f.close()
```

[7] This will give error if there is a blank line in the data file. This can be corrected by changing the comparison statement to if *len(s) < 1:* , so that the processing stops at a blank line. Modify the code to skip a blank line instead of exiting (hint: use continue ).

| Conversion | Conversion | Example | Result |
|---|---|---|---|
| d , i | signed Integer | '%6d'%(12) | '  12' |
| f | floating point decimal | '%6.4f'%(2.0/3) | 0.667 |
| e | floating point exponential | '%6.2e'%(2.0/3) | 6.67e-01 |
| x | hexadecimal | '%x'%(16) | 10 |
| o | octal | '%o'%(8) | 10 |
| s | string | '%s'%('abcd') | abcd |
| 0d | modified 'd' | '%05d'%(12) | 00012 |

Table 2.2: Formatted Printing in Python

## 2.19 Formatted Printing

Formatted printing is done by using a format string followed by the % operator and the values to be printed. If format requires a single argument, values may be a single variable. Otherwise, values must be a tuple (just place them inside parenthesis, separated by commas) with exactly the number of items specified by the format string.

*Example:* ***format.py***

```
a = 2.0 /3
print (a)
print ('a = %5.3f' %(a)) # upto 3 decimal places
```

Data can be printed in various formats. The conversion types are summarized in the following table. There are several flags that can be used to modify the formatting, like justification, filling etc.

The following example shows some of the features available with formatted printing.

*Example: format2.py*

```
a = 'justify as you like'
print ('%30s'%a)         # right justified
print ('%-30s'%a)        # minus sign for left justification
for k in range(1,11):   # A good looking table
    print ('5 x %2d = %2d' %(k, k*5))
```

The output of ***format2.py*** is given below.

```
           justify as you like
justify as you like
5 x  1 =  5
5 x  2 = 10
5 x  3 = 15
5 x  4 = 20
5 x  5 = 25
5 x  6 = 30
5 x  7 = 35
5 x  8 = 40
5 x  9 = 45
5 x 10 = 50
```

## 2.20 Exception Handling

Errors detected during execution are called exceptions, like divide by zero. If the program does not handle exceptions, the Python Interpreter reports the exception and terminates the program. We will demonstrate handling exceptions using *try* and *except* keywords, in the example except.py.

*Example: except.py*

```
x = input('Enter a number ')
try:
    print (10.0/x)
except:
    print ('Division by zero not allowed')
```

If any exception occurs while running the code inside the try block, the code inside the except block is executed. The following program implements error checking on input using exceptions.

*Example:* ***except2.py***

```
def get_number():
   while 1:
      try:
         a = input('Enter a number ')
         x = atof(a)
         return x
      except:
         print ('Enter a valid number')

print get_number()
```

## 2.21 Matrices in pure Python

In this chapter we will explore some matrix operations, using the *list* datatype. Lists inside list will be used to represent 2D matrices. This will be done in pure python with the objective of getting familiar with Python coding. For doing serious scientific computation one should use the packages like Numpy and Scipy, providing special data types to implement matrices. They will be introduced in the next chapter. Examining the code fragments given below along with their outputs should give some idea about coding in Python. For example, the code

```
x = [1,2]              #1D matrix
a = [ [1,2], [3,4] ]   # 2x2 matrix
print (a[1][0])       # result is 3
print ('NR = ', len(a), 'NC = ', len(a[0]))
```

Prints

```
3
NR = 2 NC = 2
```

It indicates how the row and column are indexed, $y[1][0]$ points to element at second row and first column. The number of rows is the number of lists inside $a$ , and the number of columns is the size of each list.

According to matrix algebra $cA = c\sum_{ij} A_{ij}$ , means all elements should be multiplied by $c$. Code should have two nested loops to traverse the rows and columns, as shown below.

```
c = 5
for row in range(2):
    for col in range(2):
        a[row][col] *= c   # x *= k is same as x = x * k
print(a)
```

Prints

```
[[5, 10], [15, 20]]
```

How to add two matrices $y = [[1,2],[3,4]]$ and $z = [[5,6],[7,8]]$. The code required is,

```
a = [ [1,2], [3,4] ]
b = [ [5,6], [7,8] ]
c = [ [0,0], [0,0] ]
for row in range(2):
    for col in range(2):
        c[row][col] = a[row][col] + b[row][col]
print(a)
```

Prints

```
[[6, 8], [10, 12]]
```

### 2.21.1 Transpose of a matrix

Transpose is obtained by interchanging rows and columns of a matrix.

```
a = [[1,2], [3,4], [5,6] ]  # 3x2 matrix
res = [[0,0,0], [0,0,0]]    # 2x3 matrix for results

for i in range(len(a)):
    for j in range(len(a[0])):
        res[j][i] = a[i][j]
print (res)
```

### 2.21.2 Matrix multiplication

Matrix multiplication is a bit more involved. The rule is $c_{ij} = \sum a_{ik} b_{kj}$. The number of rows in the first matrix should be equal to the number of columns in the second matrix. Examine the program **mat-mult.py** listed below. For each element $c_{ij}$is calculated using another loop.

```
# 3 x 3 matrices
a = [ [1,2,3], [3,4,5], [10,11,12] ]
b = [ [5,6,7], [7,8,9], [12,13,14] ]
c = [ [0,0,0], [0,0,0], [0,0,0] ]
NR = len(a)     # number of rows
NC = len(b[0])  # number of columns

for row in range(NR):
    for col in range(NC):
        for i in range(len(a[0])):  # number of columns of A
```

```
            c[row][col] += a[row][i] * b[i][col]
print (c)

# 2 x 3 and 3 x 2 matrices
a = [ [1,2,3], [3,4,5] ]       # 2 x 3
b = [ [5,6], [7,8], [12,13] ]  # 3 x 2
c = [ [0,0], [0,0] ]           # 2 x 2

NR = len(a)     # number of rows
NC = len(b[0])  # number of columns
for row in range(NR):
    for col in range(NC):
        for i in range(len(a[0])): # number of columns of A
            c[row][col] += a[row][i] * b[i][col]
print (c)
```

### 2.21.3 Cross product two vectors

Vectors can be represented using matrices and cross product of two vectors is defined by

$$A \times B = \begin{vmatrix} i & j & k \\ A_1 & A_2 & A_3 \\ B_1 & B_2 & B_3 \end{vmatrix} = i(A_2B_3 - A_3B_2) + j(A_3B_1 - A_1B_3) + k(A_1B_2 - A_2B_1) \quad (2.1)$$

The program **cross.py** calculates it using pure python code.

*Example cross.py*

```
def mycross2(a,b):
    a1, a2, a3 = a
    b1, b2, b3 = b
    return [a2*b3 - a3*b2, a3*b1 - a1*b3, a1*b2 - a2*b1]

def mycross(a,b):
    return [a[1]*b[2]-a[2]*b[1], a[2]*b[0]-a[0]*b[2],
            a[0]*b[1]-a[1]*b[0] ]

a = array([1,2,3])
b = array([4,5,6])
print(mycross(a,b))
print(mycross2(a,b))
```

The output is [-3, 6, -3] in both cases. The function *mycross* and *mycross2* are given to practice Python coding. It is not wise to carry your own cross all the way up to the mountain of scientific computation. Better methods using Numpy will be discussed in the next chapter.

### 2.21.4 Determinant of a Matrix

Determinant is a scalar value that can be computed from the elements of a square matrix. For 2x2 and 3x3 matrices, they are defined by the equations

$$\mid A \mid = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$$

$$\mid A \mid = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix} = aei - afh + bfg - bdi + cdh - ceg$$

The python code given below calculates the determinant of a 3x3 matrix, represented using a list of lists.

```
m = [ [1.,1.,1.], [0.,2.,5.], [2.,5.,-1.] ]
m1, m2, m3 = m[0]
m4, m5, m6 = m[1]
m7, m8, m9 = m[2]
det = m1*(m5*m9-m6*m8) - m2*(m4*m9-m6*m7) + m3*(m4*m8-m5*m7)
```

### 2.21.5 Inverting a Matrix

There are many ways to invert a matrix. We are using the method of taking the transpose of the matrix of cofactors and dividing it by the determinant.

$$A^{-1} = \frac{1}{\mid A \mid} C^T$$

The code **mat-invert.py** listed below does it in a crude manner, applicable only for a 3x3 matrix.

```
def invert(m):
    m1, m2, m3 = m[0]
    m4, m5, m6 = m[1]
    m7, m8, m9 = m[2]
    d = m1*(m5*m9-m6*m8) - m2*(m4*m9-m6*m7) + m3*(m4*m8-m5*m7)
    return [[(m5*m9-m6*m8)/d, (m3*m8-m2*m9)/d, (m2*m6-m3*m5)/d],
            [(m6*m7-m4*m9)/d, (m1*m9-m3*m7)/d, (m3*m4-m1*m6)/d],
            [(m4*m8-m5*m7)/d, (m2*m7-m1*m8)/d, (m1*m5-m2*m4)/d] ]

a = [ [1.,1.,1.], [0.,2.,5.], [2.,5.,-1.] ]
print (invert(a))
```

The output is

```
[[1.2857142857142858, -0.2857142857142857, -0.14285714285714285],
[-0.47619047619047616, 0.14285714285714285, 0.23809523809523808],
[0.19047619047619047, 0.14285714285714285, -0.09523809523809523]]
```

You can verify the result by calculating $A^{-1}A$ using the matrix multiplication program discussed earlier. The result should be an identity matrix, that should look like

```
[[1.0, 0.0, 0.0],
[0.0, 0.9999999999999999, 0.0],
[2.220446049250313e-16, -5.551115123125783e-17, 1.0]]
```

The elements are ones and zeros within the precision of our computation.

Examples in this section demonstrates the difficulties in using the list data type for representing arrays and matrices. In the next chapter, we will use Numpy arrays to represent matrices, making coding easier.

## 2.22 Object Oriented Programming in Python

OOP is a programming paradigm that uses *objects* (Structures consisting of variables and methods) and their interactions to design computer programs. Python is an object oriented language but it does not force you to make all programs object oriented and there is no advantage in making small programs object oriented. In this section, we will discuss some features of OOP.

Before going to the new concepts, let us recollect some of the things we have learned. We have seen that the effect of operators on different data types is predefined. For example $2*3$ results in 6 and $2*'abc'$ results in $'abcabc'$. This behavior has been decided beforehand, based on some logic, by the language designers. One of the key features of OOP is the ability to create user defined data types. The user will specify, how the new data type will behave under the existing operators like add, subtract etc. and also define methods that will belong to the new data type.

We will design a new data type using the *class* keyword and define the behavior of it. In the program *point.py*, we define a class named *Point*. The variables *xpos* and *ypos* are members of *Point*. The __init__() function is executed whenever we create an instance of this class, the member variables are initialized by this function. The way in which an object belonging to this class is printed is decided by the __str__ function. We also have defined the behavior of add (+) and subtract (-) operators for this class. The + operator returns a new Point by adding the x and y coordinates of two Points. Subtracting a Point from another gives the distance between the two. The method dist() returns the distance of a Point object from the origin. We have not defined the behavior of Point under copy operation. We can use the copy module of Python to copy objects.

*Example* **point.py**

```
class Point:
  '''
  This is documentation comment.
  help(Point) will display this.
  '''
  def __init__(self, x=0, y=0):
    self.xpos = x
    self.ypos = y

  def __str__(self): # overloads print
   return 'Point at (%f,%f)'%(self.xpos, self.ypos)

  def __add__(self, other):  #overloads +
    xpos = self.xpos + other.xpos
    ypos = self.ypos + other.ypos
    return Point(xpos,ypos)

  def __sub__(self, other):  #overloads -
    import math
    dx = self.xpos - other.xpos
    dy = self.ypos - other.ypos
```

```
    return math.sqrt(dx**2+dy**2)

  def dist(self):
    import math
    return math.sqrt(self.xpos**2 + self.ypos**2)
```

The program **point1.py** imports the file point.py to use the class Point defined inside it to demonstrate the properties of the class. A self. is prefixed when a method refers to member of the same object. It refers to the variable used for invoking the method.

*Example point1.py*

```
from point import *
origin = Point()
print origin
p1 = Point(4,4)
p2 = Point(8,7)
print (p1)
print (p2)
print (p1 + p2)
print (p1 - p2)
print (p1.dist())
```

Output of program point1.py is shown below.

```
Point at (0.000000,0.000000)
Point at (4.000000,4.000000)
Point at (8.000000,7.000000)
Point at (12.000000,11.000000)
5.0
5.65685424949
```

In this section, we have demonstrated the OO concepts like class, object and operator overloading.

### 2.22.1 Inheritance, reusing code

Reuse of code is one of the main advantages of object oriented programming. We can define another class that inherits all the properties of the Point class, as shown below. The __init__ function of colPoint calls the __init__ function of Point, to get all work except initilization of color done. All other methods and operator overloading defined for Point is inherited by colPoint.

*Example* **cpoint.py**

```
class colPoint(Point):  #colPoint inherits Point
  color = 'black'
  def __init__(self,x=0,y=0,col='black'):
    Point.__init__(self,x,y)
    self.color = col

  def __str__(self):
    return '%s colored Point at (%f,%f)'% \
       (self.color,self.xpos, self.ypos)
```

*Example* **point2.py**

```
from cpoint import *
p1 = Point(5,5)
rp1 = colPoint(2,2,'red')
print (p1)
print (rp1)
print (rp1 + p1)
print (rp1.dist())
```

The output of point2.py is listed below.
Point at (5.000000,5.000000)
red colored Point at (2.000000,2.000000)
Point at (7.000000,7.000000)
2.82842712475

For a detailed explanation on the object oriented features of Python, refer to the online book http://openbookproject.net/thinkcs/python/english3e/

## 2.23 Turtle Graphics

Turtle Graphics have been noted by many psychologists and educators to be a powerful aid in teaching geometry, spatial perception, logic skills, computer programming, and art. The language LOGO was specifically designed to introduce children to programming, using turtle graphics. An abstract drawing device, called the Turtle, is used to make programming attractive for children by concentrating on doing turtle graphics. It has been used with children as young as 3 and has a track record of 30 years of success in education.

We will use the Turtle module of Python to play with Turtle Graphics and practice the logic required for writing computer programs. Using this module, we will move a *Pen* on a two dimensional screen to generate graphical patterns. The Pen can be controlled using functions like forward(distance), backward(distance), right(angle), left(angle) etc.[8]. Run the program turtle1.py to understand the functions. This section is included only for those who want to practice programming in a more interesting manner.

*Example* **turtle1.py**

```
from turtle import *
a = Pen()    # Creates a turtle in a window
a.forward(50)
a.left(45)
a.backward(50)
a.right(45)
a.forward(50)
a.circle(10)
a.up()
a.forward(50)
a.down()
a.color('red')
a.right(90)
```

[8] http://docs.python.org/library/turtle.html

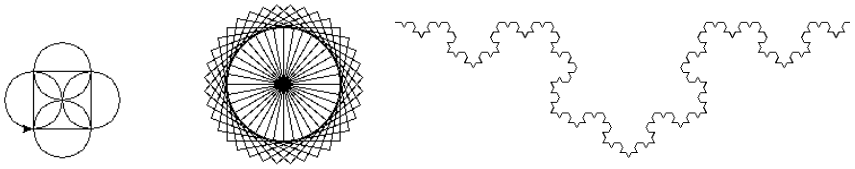


Figure 2.3: Output of turtle2.py (b) turtle3.py (c) turtle4.py

```
a.forward(50)
input('Press Enter')
```

*Example* **turtle2.py**

```
from turtle import *
a = Pen()
for k in range(4):
  a.forward(50)
  a.left(90)
  a.circle(25)
input('Press Enter')
```

Outputs of the program turtle2.py and turtle3.py are shown in figure 2.3. Try to write more programs like this to generate more complex patterns.

*Example* **turtle3.py**

```
from turtle import *
def draw_rectangle():
    for k in range(4):
        a.forward(50)
        a.left(90)

a = Pen()
for k in range(36):
  draw_rectangle()
  a.left(10)
input('Press Enter')
```

The program turtle3.py creates a pattern by drwaing 36 squares, each drawn tilted by 10° from the previous one. The program turtle4.py generates the fractal image as shown in figure2.3(c).

*Example* **turtle4.py**

```
from turtle import *

def f(length, depth):
  if depth == 0:
    forward(length)
  else:
    f(length/3, depth-1)
```

```
        right(60)
        f(length/3, depth-1)
        left(120)
        f(length/3, depth-1)
        right(60)
        f(length/3, depth-1)

f(500, 4)
input('Press any Key')
```

## 2.24 Writing GUI Programs

The programs we have written so far do not have any Graphical User Interface. Howto write such programs. First thing one should understand is other than the graphics elements there is some fundamental difference. In the text mode programs, the flow of program execution is almost decided by the code. It starts running and waits for user inputs as mentioned in the code. On the other hand, the GUI programs are event driven (movement of mouse, clicking a mouse button, pressing and releasing a key etc. are called events). The execution sequence of the program is decided by these events, generated mostly by the user. For example, when the user clicks on a Button, the code associated with that Button is executed.

Python has several tool kits for creating Graphical User Interfaces. A highly popular toolkit is Qt and PyQt5 is the current python module for that. For more details refer to[9]. GUI Programming is about creating Widgets like Button, Label, Canvas etc. on the screen and executing selected functions in response to events. After creating all the necessary widgets and displaying them on the screen, the control is passed on to PyQt toolkit by calling a function named *app.exec_()*. After that the program flow is decided by the events and associated callback functions. You can terminate the program by clicking on the 'x' at the top right corner, or by providing a Button for that purpose.

For writing GUI program using Qt toolkit, the first step is to call the function QApplication. After that we create various Widgets. The example program **qt-win.py** creates a window using QWidget, assigns a title to it and then calls show() to display it on screen. After that app_exec() takes over and waits for events generated within this window by the user. The output of qt-win.py is shown in figure2.4(a).

While we can code a simple window without using Object Oriented Programming (OOP), it becomes more difficult to as we improve the functionality of our application. We will rewrite the previous application in the object oriented programming styles. The class MyGUI inherits all the properties of the parent class QWidget. The __init__ function calls the super.__init__() to initialize the parent class. This method becomes handy while writing more complicated programs.

*Example qt-win.py*

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget

app = QApplication(sys.argv)
w = QWidget()
w.setWindowTitle('My First Qt Window')
```

[9] https://pythonspot.com/pyqt5/
https://likegeeks.com/pyqt5-tutorial/

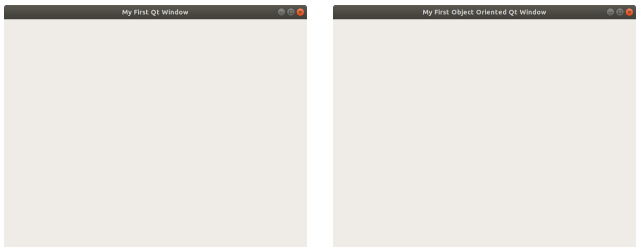



Figure 2.4: Outputs of (a)qt-win.py (b)qt-win-2.py

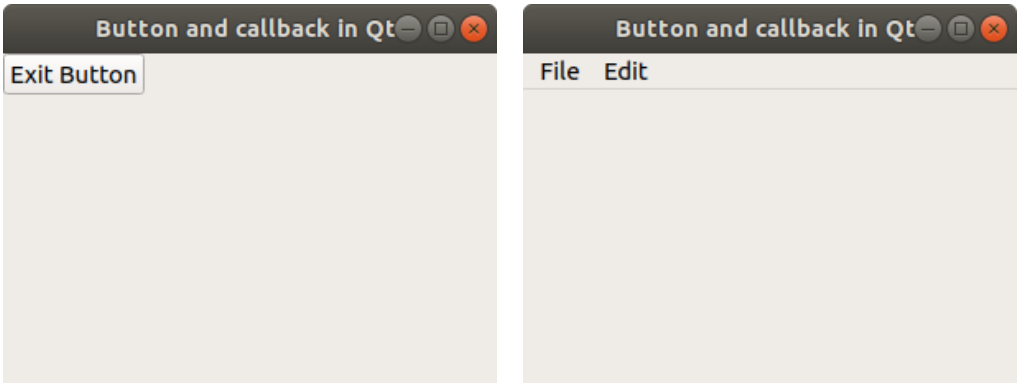



Figure 2.5: Outputs of (a) qt-button.py (b)qt-menu.py

```
w.show()
app.exec_()
```

*Example* ***qt-win-2.py***

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget

class MyGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('My First Object Oriented Qt Window')
        self.show()

app = QApplication(sys.argv)
win = MyGUI()
sys.exit(app.exec_())
```

## Button, Menu and Callback

The next example will show how to use a Button widget. A Button widget can have a callback function, *done*() in this case, that gets executed when the user clicks on the Button. The program will display a Button on the screen. When you click on it, the function *done()* will be executed. The output of the program is shown in figure 2.5(a).

*Example* **qt-button.py**

```
import sys from PyQt5.QtWidgets
import QApplication, QWidget, QPushButton
class MyGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('Button and callback in Qt')
        self.setGeometry(300, 300, 300, 200)
        btn = QPushButton('Exit Button', self)
        btn.clicked.connect(self.done)
        self.show()

def done(self):
    sys.exit(0)

app = QApplication(sys.argv)
win = MyGUI()
sys.exit(app.exec_())
```

Menu is another commonly used widget. First make a menu bar and add menus to it. Items are added to each menu after that. In order to use the menu widget the class MyGUI is subclassed from the QMainWindow widget. The output of the code listed below is shown in 2.5(b).

*Example* **qt-menu.py**

```
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QPushButton

class MyGUI(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('Button and callback in Qt')
        self.setGeometry(300, 300, 300, 200)
        mainMenu = self.menuBar()
        fileMenu = mainMenu.addMenu('File')
        fileMenu.addAction('Open',self.fileOpen)
        editMenu = mainMenu.addMenu('Edit')
        self.show()

    def fileOpen(self):
        print ('File Open called')
app = QApplication(sys.argv)
win = MyGUI()
sys.exit(app.exec_())
```

These examples are not sufficient to get one started in to programming GUIs using Qt, but plenty of documentation is available on the Internet. The objective of this book is different and we will be using a different toolkit for plotting graphs.

## 2.25 Exercises

1. Generate multiplication table of eight and write it to a file.
2. Write a Python program to open a file and write 'hello world' to it.
3. Write a Python program to open a text file and read all lines from it.
4. Write a program to generate the multiplication table of a number from the user. The output should be formatted as shown below
   1 x 5 = 5
   2 x 5 = 10
5. Write Python code to generate the sequence of numbers
   25 20 15 10 5
   using the range() function. Delete 15 from the result and sort it. Print it using a for loop.
6. Define a string *s = 'mary had a little lamb'*.
   a) print it in reverse order
   b) split it using space character as separator
7. Join the elements of the list ['I', 'am', 'in', 'pieces'] using + character. Do the same using a for loop also.
8. Create a window with five buttons. Each button should have some text on it.
9. Create a program that will put words in alphabetical order. The program should allow the user to enter as many words as he wants to.
10. Create a program that will check a sentence to see if it is a palindrome. A palindrome is a sentence that reads the same backwards and forwards ('malayalam').
11. A text file contains two columns of numbers. Write a program to read them and print the sum of numbers in each row.
12. Read a String from the keyboard. Multiply it by an integer to make its length more than 50. How do you find out the smallest number that does the job.
13. Write a program to find the length of the hypotenuse of a right triangle from the length of other two sides, get the input from the user.
14. Write a program displaying 2 labels and 2 buttons. It should print two different messages when clicked on the two buttons.
15. Write a program using for loop to reverse a string.
16. Write a program to print the values of sine function from 0 to $2\pi$ with 0.1 increments. Find the mean value of them.
17. Generate N random numbers using random.random() and find out how many are below 0.5 . Repeat the same for different values of N to draw some conclusions.
18. Use the equation $x = (-b \pm \sqrt{b^2 - 4ac})/2a$ to find the roots of $3x^2 + 6x + 12 = 0$
19. Write a program to calculate the distance between points (x1,y1) and (x2,y2) in a Cartesian plane. Get the coordinates from the user.

20. Write a program to evaluate $y = \sqrt{2.3a} + a^2 + 34.5$ for a = 1, 2 and 3.
21. Print Fibonacci numbers up to 100, without using multiple assignment statement.
22. Draw a chess board pattern using turtle graphics.
23. Find the syntax error in the following code and correct it.
    x=1
    while x <= 10:
    print x * 5
24. Write function to find the smallest of three numbers. Use it in a program that accepts inputs from the user.
25. Write a function that returns the sum of numbers in a list.
26. Write a function returns are area of a circle, default radius should be taken as 1.
27. Split 'hello world' and join back to get 'hello+world'.
28. Read a 5 digit integer from keyboard and print the sum of the digits.
29. Modify the list [1,2,3,4] to make it [1,2,3,10]
30. Print the value of $sin^2x + cos^2x$ for 10, 45 and 80 degrees.
31. print a formatted multiplication table of 8, up to 10
32. Write Python code to multiply $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ with $\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
33. Write python code to transpose $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$
34. Create a class named vehicle with for wheels. Subclass a colored car from it.
35. Draw a rectangle using Turtle graphics.
36. Draw an equilateral triangle using Turtle graphics.

# Chapter 3

# Arrays and Matrices

In the previous chapter, we have learned some essential features of Python language and used the *math* module to calculate trigonometric functions. Several matrix operations were carried out in pure Python, using the list data type to store the matrices. The list data type is not really suitable for representing arrays and matrices. In this chapter, we will use Numpy arrays to represent matrices. That will make code much simpler and easy to understand, eliminating unnecessary loops. If your main objective is to apply numerical methods to learn science or mathematics then you should learn to use the well tested and highly efficient modules like NumPy and SciPy.

The two different approaches, coding everything from scratch or using available libraries, is demonstrated by using two examples shown below. The program *sine1.py* generates the coordinates to plot a sine wave and *sine2.py* does the same and also plots it

*Example* **sine1.py**

```
import math
x = 0.0
while x < 2 * math.pi:
    print (x , math.sin(x))
    x = x + 0.1
```

The output to the screen can be redirected to a file as shown below, from the command prompt. You can plot the data using any plotting program.

$ python sine1.py > sine.dat

Listing of **sine2.py** below, along with the result in 3.1

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
plt.plot (np.sin(x))
plt.show()
```

The impact of these libraries is evident even from the simple example shown above. Big arrays of numbers and matrices are essential in scientific computation. *NumPy* is a package widely used for scientific computing using Python[1]. In the subsequent sections of this book, we will use Numpy arrays instead of Python lists to implement matrices. However, we will be writing code for many

[1] https://www.scipy.org/scipylib/faq.html

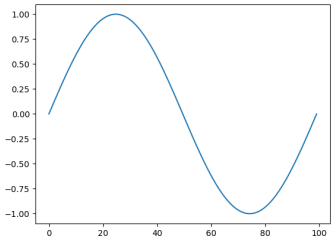



Figure 3.1: Plot of Sine curve

mathematical operations like inverting a matrix or extrapolating data, just to give some idea about the algorithms. Better versions of all those functions are available in Numpy and Scipy. For serious scientific computation one should use them.

## 3.1 NumPy Arrays

NumPy is a Python extension module that provides efficient operation on arrays of homogeneous data. *First thing to learn is how to create and manipulate arrays using numpy.* There are several functions that can be used for creating arrays.[2] The mathematical functions like sine, cosine etc. of numpy accepts array objects as arguments and return the results as arrays objects. NumPy arrays can be indexed, sliced and copied like Python Lists.

The following examples shows how to make a one and two dimensional arrays.

*Example* **1Darray.py**

```
import numpy as np
x = np.array( [1., 2, 3] ,dtype=float)   # Make array from list
print (x , type(x))
```

In the above example, we have created a numpy ndarray type object from a list. To make all members float, either you specify the datatype explicitly or make at least one member float. In the code, both are done, just for demonstration.

*Example* ***numpy2.py***

```
import numpy as np
a = [ [1, 2, 3] , [4, 5, 6] ] # make a list of lists
x = np.array(a)    # and convert to an array
print (x)
```

Other than than *array(),* there are several other functions that can be used for creating different types of arrays. Some of them are described below.

https://numpy.org/devdocs/user/quickstart.html
https://docs.scipy.org/doc/scipy/reference/tutorial/index.html
https://www.python-course.eu/numpy.php
http://www.u.arizona.edu/~erdmann/mse350/topics/basic_linear_algebra.html#matrix-powers
[2] https://docs.scipy.org/doc/numpy-1.13.0/reference/routines.array-creation.html

### 3.1.1 arange(start, stop, step)

Creates an evenly spaced one-dimensional array. Start, stop, step-size and datatype are the arguments. If datatype is not given, it is deduced from the other arguments. Note that, the values are generated within the interval, including start but excluding stop.

arange(2.0, 3.0, .1)
makes the array([ 2. , 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9])

### 3.1.2 linspace(start, stop, number of elements)

Similar to arange(). Start, stop and number of samples are the arguments.

linspace(1, 2, 11) is equivalent to array([ 1. , 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2. ])

### 3.1.3 zeros(shape)

Returns a new array of given shape and type, filled zeros. For example *zeros( [2,3])* generates a 2 x 3 array filled with zeros

```
[[0.,0.,0.]
 [0.,0.,0.]]
```

### 3.1.4 ones(shape)

Similar to zeros() except that the elements are initialized to 1.

### 3.1.5 random.random(shape)

Similar to the functions above, but the matrix is filled with random numbers ranging from 0 to 1, of *float* type. For example, random.random([3,3]) will generate the 3x3 matrix;

```
array([[ 0.3759652 , 0.58443562, 0.41632997],
       [ 0.88497654, 0.79518478, 0.60402514],
       [ 0.65468458, 0.05818105, 0.55621826]])
```

### 3.1.6 reshape(array, newshape)

We can also make multi-dimensions arrays by reshaping a one-dimensional array. The function *reshape()* changes dimensions of an array. The total number of elements must be preserved. Note the object oriented way of calling reshape(). Since the array **a** is an object belonging to the Class named Array, all the functions of that class are available under that object. Working of *reshape()* can be understood by looking at *reshape.py* and its result.

*Example* **reshape.py**

```
import numpy as np
a = np.arange(20)
b = a.reshape([4,5])
print (b)
```

The result is shown below.

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19]])
```

The program *array_funcs.py* demonstrates some of the functions for creating numpy arrays.

*Example* **array_funcs.py**

```
import numpy as np
a = np.arange(1.0, 2.0, 0.1)
print (a)
b = np.linspace(1,2,11)
print (b)
c = np.ones(5,'float')
print (c)
d = np.zeros(5, 'int')
print (d)
e = np.random.rand(5)
print (e)
```

Output of this program will look like;
[ 1. 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9]
[ 1. 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 2. ]
[ 1. 1. 1. 1. 1.]
[ 0. 0. 0. 0. 0.]
[ 0.89039193 0.55640332 0.38962117 0.17238343 0.01297415]

### 3.1.7 Copying

Numpy arrays can be copied using the copy method, as shown below.

*Example* **array_copy.py**

```
import numpy as np
a = np.zeros(5)
print (a)
b = a
c = a.copy()
a[0] = 10
print a, b, c
```

The output of the program is shown below. The statement $b = a$ does not make a copy of **a**. Changing the first element of **a** has changed **b** also, but **c** is a separate entity.
[ 0. 0. 0.]
[ 10. 0. 0.] [ 10. 0. 0.] [ 10. 0. 0.]

### 3.1.8 Saving and Restoring

There are many ways for reading from file and writing to data files in numpy. We will discuss only the ones saves data in the formatted text form, savetext() and loadtext() methods, as shown in the code *fileio.py*

*Example* **fileio.py**

```
import numpy as np
a = np.array([[2,3], [4,5]])
print(a)
np.savetxt('mydata.txt', a, fmt = '%2.3f', delimiter = ' ')
print('Data saved to text file')
b = np.loadtxt("mydata.txt", delimiter=" ")
print('Data loaded from text file')
print(b)
```

The output of the program will look like

```
[[2 3]
[4 5]]
Data saved to text file
Data loaded from text file
[[ 2. 3.]
[ 4. 5.]]
```

The text file 'mydata.txt' will look like
2.000 3.000
4.000 5.000

### 3.1.9 Slicing to extract elements, rows and columns

Since slicing is used for extracting columns from a two dimensional array, we need to understand it a bit better.

```
x = np.reshape(range(12),(4,3))
```

creates the array having 4 rows and 3 columns.

```
[[ 0,  1,  2],
 [ 3,  4,  5],
 [ 6,  7,  8],
 [ 9, 10, 11]]
```

and extract elements from it in different ways, python indexing starts from zero.

```
x[1] extracts the second row to give [3,4,5].
x[2,1] extracts 7, the element at third row and second column.
x[1:3, 1] extracts the elements [4,7] from second column.
x[ :, 1} gives the full second column [ 1, 4, 7, 10]
x[:, :] gives the complete array
[[0, 1],
 [3, 4]]
```

You should remember that a statement like $y = x[1]$ does not create a new array. It is only a reference to the second row or $x$ . The statement $y[0] = 100$ changes $x[1, 0]$to 100.

The program **slice.py**

```
import numpy as np
x = np.arange(12).reshape([4,3])
print(x[1])      # prints the second row
print(x[2,1])    # element at third row and second column
print(x[1:3,1])
print(x[:,1])
print(x[:2,:2])
```

will generate the output

```
[3 4 5]
7
[4 7]
[ 1 4 7 10]
[[0 1]
 [3 4]]
```

### 3.1.10 Arithmetic Operations

Arithmetic operations on array differs from that of ordinary numbers. Adding or multiplying an array object with an ordinary number will multiply all the elements by that number. However, operations involving more than one array could be done in different ways.

```
x = ones((2,2)) makes
[[ 1.,  1.],
 [ 1.,  1.]]
```

x * 5 multiplies all elements by 5 to give

```
[[ 5.,  5.],
 [ 5.,  5.]]
```

x * x will not perform a matrix multiplication. To perform a proper matrix multiplication try $x@x$ , $np.dot(x, x)$ or $x.dot(x)$ to get the result

```
[[ 2., 2.],
 [ 2., 2.]]
```

*Example* **aroper.py**

```
import numpy as np
x = np.ones( (2,2,))
print(x*5)
print(x * x)
print(x @ x)
```

Modifying this program for more operations is left as an exercise to the reader.

### 3.1.11 Pauli spin matrices

We use the Pauli spin matrices

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \text{ and } \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

to demonstrate some matrix operations using numpy, with the program **pauli.py** listed below

```
import numpy as np
sx = np.array( [ [0+0j, 1+0j], [1+0j, 0+0j] ])    # sigma x
sy = np.array( [ [0+0j, 0-1j], [0+1j, 0+0j] ])    # sigma y
sz = np.array( [ [1+0j, 0+0j], [0+0j, -1+0j] ])   # sigma z
print ('sx\n', sx)
print ('Transpose(sx)\n', np.transpose(sx))
commutexy = sx.dot(sy) - sy.dot(sx)
print ('[sx,sy]\n',commutexy)
sz2i = 2 * (0+1j) * sz
print ('2 i sz\n',sz2i)
print (sz2i == commutexy)
```

The output of the program is given below. The last result shows that $[\sigma_x, \sigma_y] = 2i\sigma_z$ . Refer to Wikipedia[3] for more information.

```
sx
[[ 0.+0.j  1.+0.j]
 [ 1.+0.j  0.+0.j]]
Transpose(sx)
[[ 0.+0.j  1.+0.j]
 [ 1.+0.j  0.+0.j]]
[sx,sy]
[[ 0.+2.j  0.+0.j]
 [ 0.+0.j  0.-2.j]]
2isz
[[ 0.+2.j  0.+0.j]
 [ 0.+0.j  0.-2.j]]
[[ True  True]
 [ True  True]]
```

## 3.2 Vectorizing Functions

The functions like sine, log etc. from NumPy are capable of accepting arrays as arguments. This eliminates the need of writing loops in our Python code.

*Example* **vfunc.py**

```
import numpy as np
a = np.array([1,10,100,1000])
print (np.log10(a))
```

[3] https://en.wikipedia.org/wiki/Pauli_matrices

The output of the program is [ 0. 1. 2. 3.] , where the log of each element is calculated and returned in an array. This feature simplifies the programs a lot. Numpy also provides a function to vectorize functions written by the user.

*Example* **vectorize.py**

```
import numpy as np

def spf(x):
    return 3*x

vspf = np.vectorize(spf)
a = np.array([1,2,3,4])
print (vspf(a))
```

The output will be [ 3 6 9 12] .

## 3.3 Exercises

1. Write code to make a one dimensional matrix with elements 5,10,15,20 and 25. make another matrix by slicing the first three elements from it.
2. Create a $3 \times 2$ matrix and print the sum of its elements using for loops.
3. Create a $2 \times 3$ matrix and fill it with random numbers.
4. Use linspace to make an array from 0 to 10, with step-size of 0.1
5. Use arange to make an 100 element array ranging from 0 to 10
6. Make an array a = [2,3,4,5] and copy it to $b$. change one element of $b$ and print both.
7. Make a 3x3 matrix and multiply it by 5.
8. Create two 3x3 matrices and add them.
9. Write programs to demonstrate the dot and cross products.
10. Using matrix inversion, solve the system of equations
    4x1 − 2x2 + x3 = 11
    −2x1 + 4x2 − 2x3 = −16
    x1 − 2x2 + 4x3 = 17
11. Find the new values of the coordinate (10,10) under a rotation by angle $\pi/4$.
12. Write a vectorized function to evaluate $y = x^2$ and print the result for x=[1,2,3].
13. Create a 4x3 matrix using range() and reshape().
14. Extract the second row from a 4x3 matrix
15. Extract the second column from a 3x3 matrix
16. create a matrix [ [1,2] , [3,4] ]. Save it to a file, read it back and print.

# Chapter 4

# Data visualization

A graph or chart can be used for presenting numerical data in visual form. A graph is one of the easiest ways to compare numbers. They should be used to make facts clearer and more understandable. Results of mathematical computations are often presented in graphical format. In this chapter, we will explore the Python modules used for generating two and three dimensional graphs of various types.

## 4.1 The Matplotlib Module

Matplotlib[1] is a python package that produces publication quality figures in a variety of hardcopy formats. You can generate plots, histograms, power spectra, bar charts, error-charts, scatter-plots, etc, with just a few lines of code and have full control of line styles, font properties, axes properties, etc. The data points to the plotting functions are supplied as Python lists or Numpy arrays.

Matplotlib creates a Figure object (representing your entire plot window). Inside this, it creates one or more Axes objects by using the add_subplot() method. Inside the Axes object, the drawing objects are created using the methods like plot() etc. The sequence is shown below:

```
import matplotlib.pyplot as plt
fig = plt.figure()
ax = fig.add_subplot()
ax.plot([1,2,3], [10,12,15])
plt.show()
```

However, in the case of 2D plots the plot() function can be called directly like:

```
import matplotlib.pyplot as plt
plt.plot([1,2,3], [10,12,15])
plt.show()
```

In this case the Figure and Axes objects are created and managed by Matplotlib. The first method is called using the explicit "Axes" interface. The second one is using the implicit "pyplot" interface. We will be using the implicit method for all the 2D plots. The explicit way of creating the objects will be used where the references to the Figure and Axes objects are required, for example creating 2D animations or 3D plots.

[1] https://www.tutorialspoint.com/matplotlib/matplotlib_quick_guide.htm
https://matplotlib.org/
https://matplotlib.org/tutorials/introductory/pyplot.html#sphx-glr-tutorials-introductory-pyplot-py

### 4.1.1 2D plots

Let us start with some simple plots to become familiar with matplotlib.

*Example* **plot1.py**

```
import matplotlib.pyplot as plt
data = [1,2,5]
plt.plot(data)
plt.show()
```

In the above example, the x-axis of the three points is assumed as [0,1,2]. We can specify both the axes as shown below.

*Example* **plot2.py**

```
import matplotlib.pyplot as plt
x = [1,2,5]
y = [4,5,6]
plt.plot(x,y)
plt.show()
```

By default, the color of first plot is blue and the line style is continuous. This can be changed by an optional argument after the coordinate data, which is the format string that indicates the color and line type of the plot. The default format string is 'b-' (blue, continuous line). Let us rewrite the above example to plot using red circles. We will also set the ranges for x and y axes and label them, as shown in *plot3.py*.

*Example* **plot3.py**

```
import matplotlib.pyplot as plt
x = [1,2,5]
y = [4,5,6]
plt.plot(x,y,'ro')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.axis([0,6,1,7])
plt.show()
```

The figure 4.1 shows two different plots in the same window, using different markers and colors. It also shows how to use legends.

*Example* **plot4.py**

```
import numpy as np
import matplotlib.pyplot as plt
t = np.arange(0.0, 5.0, 0.2)
plt.plot(t, t**2,'x', label='t^2')      # t^{2}
plt.plot(t, t**3,'ro', label='t^3')     # t^{3}
plt.legend(framealpha=0.5)
plt.show()
```

We can specify the markers and colors in the following manner also.

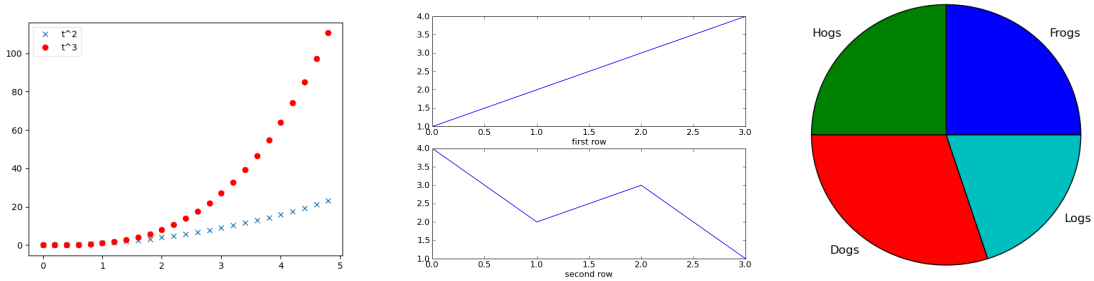



Figure 4.1: Output of (a) plot4.py (b) subplot1.py (c) piechart.py

```
plt.plot(t, t**2, color='red', marker='x')
```

The markers and colors available are,

```
markers = ["." , "," , "o" , "v" , "^" , "<", ">"]
colors = ['r','g','b','c','m', 'y', 'k']
```

### 4.1.2 Polar plots

Polar coordinates locate a point on a plane with one distance and one angle. The distance 'r' is measured from the origin. The angle $\theta$ is measured from some agreed starting point. Use the positive part of the $x-axis$ as the starting point for measuring angles. Measure positive angles anti-clockwise from the positive $x-axis$ and negative angles clockwise from it.

Matplotlib supports polar plots, using the polar$(\theta, r)$ function. Let us plot a circle using polar(). For every point on the circle, the value of *radius* is the same but the polar angle $\theta$ changes from 0to $\frac{\pi}{2}$. Both the coordinate arguments must be arrays of equal size. Since $\theta$ is having 100 points , $r$ also must have the same number. This array can be generated using the *ones*() function. The axis$([\theta_{min}, \theta_{max}, r_{min}, r_{max})$ function can be used for setting the scale.

*Example* **polar.py**

```
import numpy as np
import matplotlib.pyplot as plt
th = np.linspace(0, np.pi/2,100)
r = 5 * np.ones(100)           # radius = 5
plt.polar(th,r)
plt.show()
```

### 4.1.3 Pie Charts

An example of a pie chart is given below. The percentage of different items and their names are given as arguments. The output is shown in figure 4.1(c).

*Example* **piechart.py**

```
import matplotlib.pyplot as plt
labels = 'Frogs', 'Hogs', 'Dogs', 'Logs'
fracs = [25, 25, 30, 20]
plt.pie(fracs, labels=labels)
plt.show()
```

### 4.1.4 Multiple plots

Matplotlib allows you to have multiple plots in the same window, using the subplot() command as shown in the example subplot1.py, whose output is shown in figure 4.1(b).

*Example* **subplot1.py**

```
import matplotlib.pyplot as plt
plt.subplot(211)             # the first subplot
plt.plot([1,2,3,4])
plt.xlabel('first row')
plt.subplot(212)             # the second subplot
plt.plot([4,2,3,1])
plt.xlabel('second row')
plt.show()
```

The arguments to subplot function are NR (number of rows) , NC (number of columns) and a figure number, that ranges from 1 to $NR * NC$. The commas between the arguments are optional if $NR * NC < 10$, ie. subplot(2,1,1) can be written as subplot(211).

Another example of subplot is given is *subplot2.py*. You can modify the variable NR and NC to watch the results. Please note that the % character has different meanings. In *(pn+1)%5*, it is the reminder operator resulting in a number less than 5. The % character also appears in the String formatting.

*Example* **subplot2.py**

```
import numpy as np
import matplotlib.pyplot as plt
mark = ['x','o','^','+','>']
NR = 2  # number of rows
NC = 3  # number of columns
pn = 1
for row in range(NR):
   for col in range(NC):
      subplot(NR, NC, pn)
      a = np.random.rand(10) * pn
      plt.plot(a, marker = mark[(pn+1)%5])
      plt.xlabel('plot %d X'%pn)
      plt.ylabel('plot %d Y'%pn)
      pn = pn + 1
plt.show()
The output is shown in figure4.2.
```

## 4.2 Plotting mathematical functions

One of our objectives is to understand various mathematical functions by plotting them graphically. We will use the *arange*, *linspace* and *logspace* functions from *numpy* to generate the input data and also the vectorized versions of the functions. For the function arange(), the third argument is the step-size. The total number of elements is calculated from start, stop and step-size. In the case of linspace(), we provide start, stop and the total number of points. The step size is calculated from these three parameters. Please note that to create a data set ranging from 0 to 1 (including both) with a step-size of 0.1, we need to specify linspace(0,1,11) and not linspace(0,1,10).

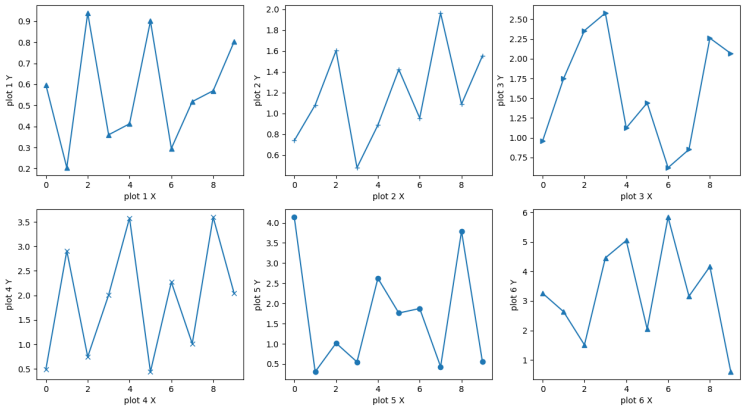



Figure 4.2: Multiple plots

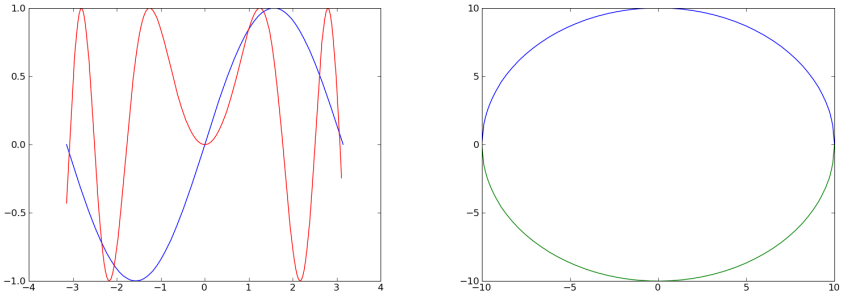


Figure 4.3: (a) Output of npsin.py (b) Output of circ.py .

### 4.2.1 Sine function and friends

Let the first example be the familiar sine function. The input data is from $-\pi$ to $+\pi$ radians[2]. To make it a bit more interesting we are plotting $\sin x^2$ also. The objective is to explain the concept of odd and even functions. Mathematically, we say that a function $f(x)$ is even if $f(x) = f(-x)$ and is odd if $f(-x) = -f(x)$. Even functions are functions for which the left half of the plane looks like the mirror image of the right half of the plane. From the figure 4.3(a) you can see that $\sin x$ is odd and $\sin x^2$ is even.

*Example* **npsin.py**

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-np.pi, np.pi , 200)
y = np.sin(x)
y1 = np.sin(x*x)
```

[2]Why do we need to give the angles in radians and not in degrees. Angle in radian is the length of the arc defined by the given angle, with unit radius. Degree is just an arbitrary unit.

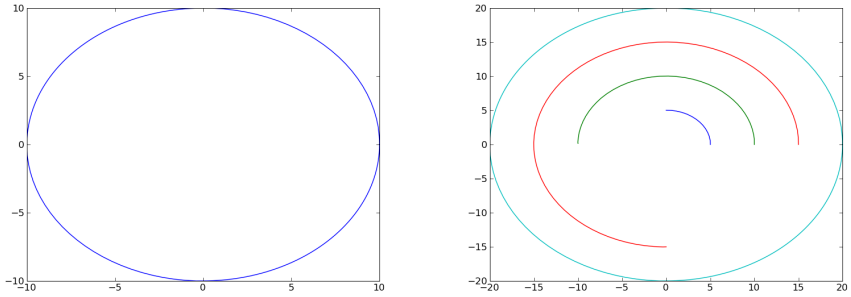


Figure 4.4: (a)Output of circpar.py. (b)Output of arcs.py

```
plt.plot(x,y)
plt.plot(x,y1,'r')
plt.show()
```

Exercise: Modify the program *npsin.py* to plot $\sin^2 x$ , $\cos x$, $\sin x^3$ etc.

### 4.2.2 Trouble with Circle

Equation of a circle is $x^2 + y^2 = a^2$ , where a is the radius and the circle is located at the origin of the coordinate system. In order to plot it using Cartesian coordinates, we need to express $y$ in terms of $x$, and is given by

$$y = \sqrt{a^2 - x^2}$$

We will create the x-coordinates ranging from $-a$ to $+a$ and calculate the corresponding values of y. This will give us only half of the circle, since for each value of x, there are two values of y (+y and -y). The following program *circ.py* creates both to make the complete circle as shown in figure 4.3(b). Any multi-valued function will have this problem while plotting. Such functions can be plotted better using parametric equations or using the polar plot options, as explained in the coming sections.

*Example* **circ.py**

```
import numpy as np
import matplotlib.pyplot as plt
a = 10.0
x = np.linspace(-a, a , 200)
yupper = np.sqrt(a**2 - x**2)
ylower = -np.sqrt(a**2 - x**2)
plt.plot(x,yupper)
plt.plot(x,ylower)
plt.show()
```

### 4.2.3 Parametric plots

The circle can be represented using the equations $x = a\cos\theta$ and $y = a\sin\theta$ . To get the complete circle $\theta$ should vary from zero to $2\pi$ radians. The output of circpar.py is shown in figure 4.4(a).

*Example* **circpar.py**

```
import numpy as np
import matplotlib.pyplot as plt
a = 10.0
th = np.linspace(0, 2*np.pi, 200)
x = a * np.cos(th)
y = a * np.sin(th)
plt.plot(x,y)
plt.show()
```

Changing the range of $\theta$ to less than $2\pi$ radians will result in an arc. The following example plots several arcs with different radii. The *for* loop will execute four times with the values of radius 5,10,15 and 20. The range of $\theta$ also depends on the loop variable. For the next three values it will be $\pi$, $1.5\pi$ and $2\pi$ respectively. The output is shown in figure 4.4(b).

*Example* **arcs.py**

```
import numpy as np
import matplotlib.pyplot as plt
a = 10.0
for a in range(5,21,5):
   th = np.linspace(0, np.pi * a/10, 200)
   x = a * np.cos(th)
   y = a * np.sin(th)
   plt.plot(x,y)
plt.show()
```

## 4.3 Plotting Error Bars

The programm **errbar.py** listed below shows how to add errorbars to a plot.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.arange(10)
y = 5 * np.sin(x / 5 )
dy = np.linspace(.1, 0.3, 10)
plt.plot(x,y)
plt.errorbar(x, y, yerr=dy)
plt.show()
```

## 4.4 Simple 2D animation

Matplotlib supports some animation functions. Program **animate.py** uses FuncAnimation() to do a basic animation of a sine wave. A matplotlib 'figure' object is created and a 'line' is drawn. Invoking the 'FuncAnimation' function sets up calling 'update' periodically, with frame number sequentially incremented from 0 to 99, that is used for adding a phase increment to the plot. You may change the variable 'interval' to adjust the update rate.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation as fa

fig1 = plt.figure()
ax = fig.add_subplot()
x = np.linspace(-2*np.pi, 2*np.pi, 100)
L, =  ax.plot(x, np.sin(x))
def update(fnum):
    L.set_data(x, np.sin(x+ 0.1 *fnum))
anim = fa(fig1, update, frames=100, interval=50)
plt.show()
```

## 4.5 Famous Curves

Connection between different branches of mathematics like trigonometry, algebra and geometry can be understood by geometrically representing the equations. You will find a large number of equations generating geometric patterns having interesting symmetries. We will select some of them and plot here. Exploring them further is left as an exercise to the reader.[3]

### 4.5.1 Astroid

The astroid was first discussed by Johann Bernoulli in 1691-92. It also appears in Leibniz's correspondence of 1715. It is sometimes called the tetracuspid for the obvious reason that it has four cusps. A circle of radius 1/4 rolls around inside a circle of radius 1 and a point on its circumference traces an astroid. The Cartesian equation is

$$x^{\frac{2}{3}} + y^{\frac{2}{3}} = a^{\frac{2}{3}} \tag{4.1}$$

The parametric equations are

$$x = a\cos^3(t), y = a\sin^3(t) \tag{4.2}$$

In order to plot the curve in the Cartesian system, we rewrite equation 4.1 as

$$y = (a^{\frac{2}{3}} - x^{\frac{2}{3}})^{\frac{3}{2}}$$

The program *astro.py* plots the part of the curve in the first quadrant. The program *astropar.py* uses the parametric equation and plots the complete curve. Both are shown in figure 4.5

*Example astro.py*

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
x = np.linspace(0,a,100)
y = ( a**(2.0/3) - x**(2.0/3) )**(3.0/2)
plt.plot(x,y)
plt.show()
```

[3] https://en.wikipedia.org/wiki/List_of_curves

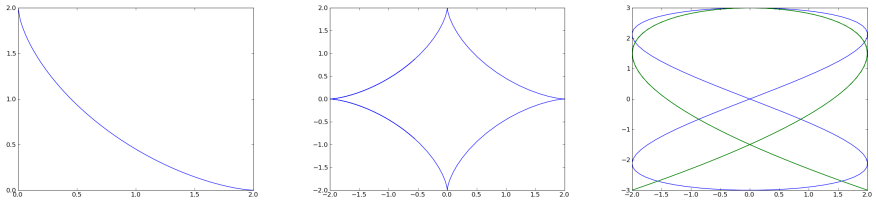


Figure 4.5: (a)Output of astro.py (b) astropar.py (c) lissa.py

*Example astropar.py*

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
t = np.linspace(-2*a,2*a,101)
x = a * np.cos(t)**3
y = a * np.sin(t)**3
plt.plot(x,y)
plt.show()
```

### 4.5.2 Ellipse

The Cartesian equation of ellipse is

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1 \tag{4.3}$$

The parametric equations are

$$x = a\cos(t), y = b\sin(t) \tag{4.4}$$

The program **ellipse.py** uses the parametric equation to plot the curve. Modifying the parametric equations will result in Lissajous figures. The output of lissa.py are shown in figure 4.5(c).

*Example ellipse.py*

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
b = 3
t = np.linspace(0, 2 * pi, 100)
x = a * np.sin(t)
y = b * np.cos(t)
plt.plot(x,y)
plt.show()
```

*Example* **lissa.py**

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
```

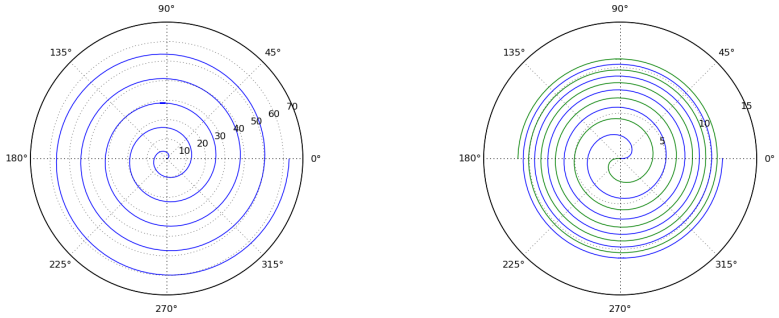



Figure 4.6: (a)Archimedes Spiral (b)Fermat's Spiral (c)Polar Rose

```
b = 3
t= np.linspace(0, 2*np.pi,100)
x = a * np.sin(2*t)
y = b * np.cos(t)
plt.plot(x,y)
x = a * np.sin(3*t)
y = b * np.cos(2*t)
plt.plot(x,y)
plt.show()
```

The Lissajous curves are closed if the ratio of the arguments for sine and cosine functions is an integer. Otherwise open curves will result, both are shown in figure 4.5(c).

### 4.5.3 Spirals of Archimedes and Fermat

The spiral of Archimedes is represented by the equation $r = a\theta$. Fermat's Spiral is given by $r^2 = a^2\theta$. The output of archi.py and fermat.py are shown in figure 4.6.

*Example archi.py*

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
th= np.linspace(0, 10 * np.pi,200)
r = a*th
plt.polar(th,r)
plt.axis([0, 2 * np.pi, 0, 70])
plt.show()
```

*Example fermat.py*

```
import numpy as np
import matplotlib.pyplot as plt
a = 2
th= np.linspace(0, 10 * np.pi,200)
r = np.sqrt(a**2 * th)
plt.polar(th,r)
plt.show()
```

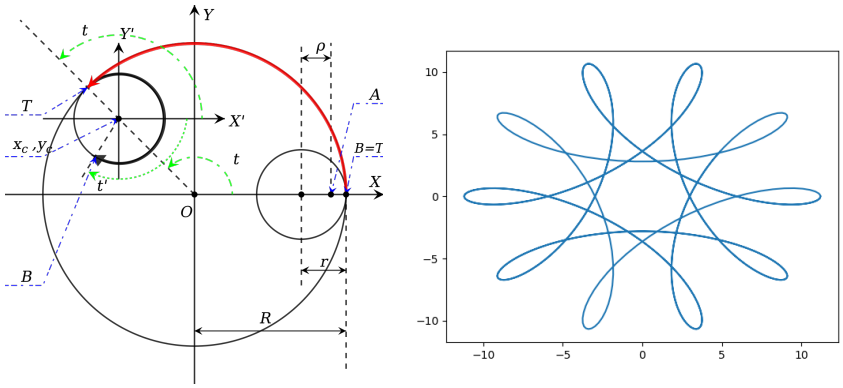



Figure 4.7: Spirograph.(a) mathematical basis (b) trajecory plot

### 4.5.4 Spirograph

Spirograph is a geometric drawing device that produces mathematical roulette curves of the variety technically known as hypotrochoids and epitrochoids.[4] Consider a fixed outer circle of radius R, centered at the origin, and a smaller inner circle of radius r inside it, as shown in figure4.7(a). When the small circle rolls inside the big circle, the trajectory of a point located inside the small circle at a distance $\rho < R$ from it's center is given by the equations,

$$x(t) = R\left[(1-k)\cos t + lk\cos\frac{1-k}{k}t\right]$$

$$y(t) = R\left[(1-k)\sin t + lksin\frac{1-k}{k}t\right]$$

where $k = \frac{r}{R}$; $l = \frac{\rho}{r}$ and $t$ is the angle of rotation. The python code **spiro.py** plots the output for some values of R, r and $\rho$ .4.7(b). You may try changing the values of these parameters.

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(0, 10 * np.pi, 500)
r = 3.
R = 10.0
k = r/R
l = 1.4
x = R*((1-k)*np.cos(t) + l*k*np.cos((1-k)*t/k))
y = R*((1-k)*np.sin(t) - l*k*np.sin((1-k)*t/k))
plt.plot(x,y)
plt.show()
```

## 4.6 2D plot using colors

A two dimensional matrix can be represented graphically by assigning a color to each point proportional to the value of that element. The program imshow1.py makes a $50 \times 50$ matrix filled

[4] https://en.wikipedia.org/wiki/Spirograph
https://www.geeksforgeeks.org/fractal-using-spirograph-python

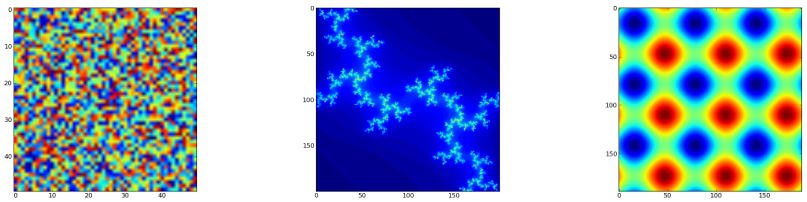


Figure 4.8: Outputs of (a) imshow1.py (b) julia.py (c) mgrid2.py

with random numbers and uses *imshow()* to plot it. The result is shown in figure 4.8(a).

*Example* **imshow1.py**

```
import numpy as np
import matplotlib.pyplot as plt
m = np.random.random([50,50])
plt.imshow(m)
plt.show()
```

### 4.6.1 Fractals

Fractals[5] are a part of fractal geometry, which is a branch of mathematics concerned with irregular patterns made of parts that are in some way similar to the whole (e.g.: twigs and tree branches). A fractal is a design of infinite details. It is created using a mathematical formula. No matter how closely you look at a fractal, it never loses its detail. It is infinitely detailed, yet it can be contained in a finite space. Fractals are generally self-similar and independent of scale. The theory of fractals was developed from Benoit Mandelbrot's study of complexity and chaos. Complex numbers are required to compute the Mandelbrot and Julia Set fractals and it is assumed that the reader is familiar with the basics of complex numbers.

To compute the basic Mandelbrot (or Julia) set one uses the equation $f(z) \rightarrow z^2 + c$ , where both z and c are complex numbers. The function is evaluated in an iterative manner, ie. the result is assigned to $z$ and the process is repeated. The purpose of the iteration is to determine the behavior of the values that are put into the function. If the value of the function goes to infinity (practically to some fixed value, like 1 or 2) after few iterations for a particular value of $z$ , that point is considered to be outside the Set. A Julia set can be defined as the set of all the complex numbers $(z)$ such that the iteration of $f(z) \rightarrow z^2 + c$ is bounded for a particular value of c.

To generate the fractal the number of iterations required to diverge is calculated for a set of points in the selected region in the complex plane. The number of iterations taken for diverging decides the color of each point. The points that did not diverge, belonging to the set, are plotted with the same color. The program *julia.py* generates a fractal using a julia set. The program creates a 2D array (200 x 200 elements). For our calculations, this array represents a rectangular region on the complex plane centered at the origin whose lower left corner is (-1,-j) and the upper right corner is (1+j). For 200x200 equidistant points in this plane the number of iterations are calculated and that value is given to the corresponding element of the 2D matrix. The plotting is taken care by the imshow function. The output is shown in figure 4.8(b). Change the value of $c$ and run the program to generate more patterns. The equation also may be changed.

[5] http://en.wikipedia.org/wiki/Fractal

*Example* **julia.py**

```
'''
Region of a complex plane ranging from -1 to +1 in both real
and imaginary axes is represented using a 2D  matrix
having X x Y elements.For X and Y equal to 200,the step-size
in the complex plane is 2.0/200 = 0.01.
The nature of the pattern depends much on the value of c.
'''
import numpy as np
import matplotlib.pyplot as plt
X = 200
Y = 200
MAXIT = 100
MAXABS = 2.0
c = 0.02 - 0.8j   # The constant in equation z**2 + c
m = np.zeros([X,Y])  # A two dimensional array

def numit(x,y):       # number of iterations to diverge
     z = complex(x,y)
     for k in range(MAXIT):
            if abs(z) <= MAXABS:
                z = z**2 + c
            else:
                return k     # diverged after k trials
     return MAXIT            # did not diverge,

for x in range(X):
    for y in range(Y):
        re = 0.01 * x - 1.0     # complex number for
        im = 0.01 * y - 1.0     # this (x,y) coordinate
        m[x][y] = numit(re,im)  # get the color for (x,y)
plt.imshow(m)  # Colored plot using the 2D matrix
plt.show()
```

The fern is one of the basic examples of self-similar sets, a mathematically generated pattern that can be reproducible at any magnification or reduction.(Fern). The program **fern.py** listed below generates a pattern as shown in the figure4.9(b).

```
import numpy as np
import matplotlib.pyplot as plt
NP = 2000000
x = np.zeros(NP)
y = np.zeros(NP)
for i in range(0, NP-1):
      p = np.random.randint(1, 100)
      if p == 1:
        x[i+1] = 0
        y[i+1] = 0.16*y[i]
      if p >= 2 and p <= 86:
        x[i+1] = 0.85*x[i] + 0.04*y[i]
```

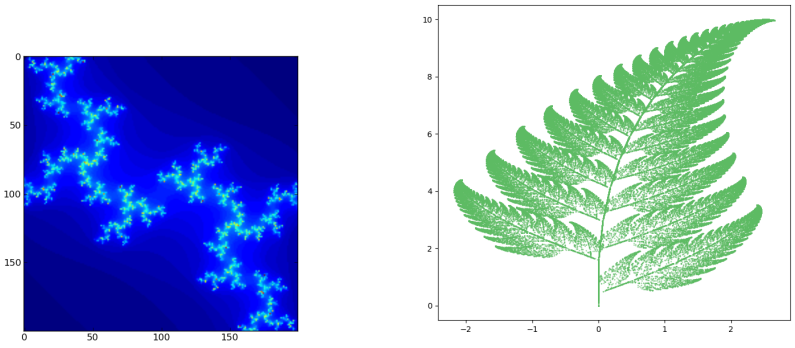


Figure 4.9: a)Julia fractal b)Barnsley Fern

```
        y[i+1] = -0.04*x[i] + 0.85*y[i]+1.6
      if p >= 87 and p <= 93:
        x[i+1] = 0.2*x[i] - 0.26*y[i]
        y[i+1] = 0.23*x[i] + 0.22*y[i]+1.6
      if p >= 94 and p < 99:
        x[i+1] = -0.15*x[i] + 0.28*y[i]
        y[i+1] = 0.26*x[i] + 0.24*y[i]+0.44
plt.scatter(x, y, s = 0.2, c ='#5dbb63')
plt.show()
```

## 4.7 3D Plots

Matplotlib may not be the best tool for 3D plotting but your existing knowledge about plotting 2D graphs can be extended to several types of 3D plots, using the mplot3d toolkit. The Axes3D class is used for making the 3D plots. First a matplotlib Figure object (representing your entire plot window) is created. Under this one or more Axes objects are created using the add_subplot() method. Inside the Axes object, the drawing objects are created using the methods like plot() etc. The example below demonstrates the procedure.

### 4.7.1 3D Line Plots

Example of a line plot is shown in line3d.py along with the output in figure 4.7.3(b).

*Example* **line3d.py**

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
z = np.linspace(0, 10, 10)
x = z
y = z
fig = plt.figure()
ax = fig.add_subplot(projection = '3d')
```

```
ax.plot3D(x,y,z)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.scatter3D(x,y,-z)
plt.show()
```

The result is a straight line starting from the origin to the point (10,10,10) and ten points where the value of 'z' goes from 0 to -10.

### 4.7.2 Meshgrids

in order to make contour and surface3D plots, we need to understand the concept of meshgrid. Consider a rectangular area on the X-Y plane. Assume there are m divisions in the X direction and n divisions in the Y direction. We now have a $m \times n$ mesh. A meshgrid is the coordinates of a grid in a 2D plane, x coordinates of each mesh point are held in one 2D matrix and y coordinates are held in another.

The NumPy function meshgrid() creates two 2x2 matrices from two 1D arrays, as shown in the example below. This can be used for plotting surfaces and contours, by assigning a Z coordinate to every mesh point.

*Example* **mgrid1.py**

```
import numpy as np
x = np.arange(0, 3, 1)
y = np.arange(0, 3, 1)
gx, gy = np.meshgrid(x, y)
print gx
print gy
```

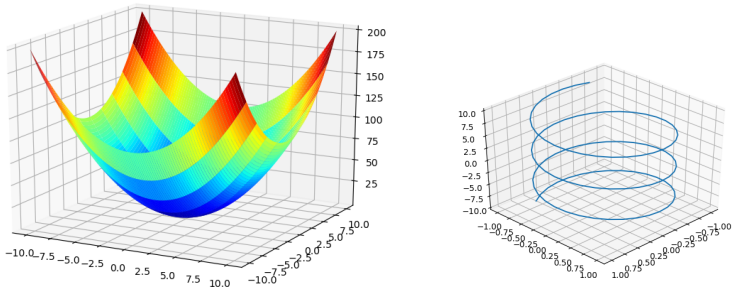
The outputs are as shown below, gx(i,j) contains the x-coordinate and gx(i,j) contains the y-coordinate of the point (i,j).

[[0 1 2]
[0 1 2]
[0 1 2]]
[[0 0 0]
[1 1 1]
[2 2 2]]

We can evaluate a function at all points of the meshgrid by passing the meshgrid as an argument. The program *mgrid2.py* plots the sum of sines of the x and y coordinates, using imshow to get a result as shown in figure 4.8(c).

*Example* **mgrid2.py**

```
from pylab import *
x = arange(-3*pi, 3*pi, 0.1)
y = arange(-3*pi, 3*pi, 0.1)
xx, yy = meshgrid(x, y)
z = sin(xx) + sin(yy)
imshow(z)
show()
```



Output of (a)surface3d.py (b)line3d.py

### 4.7.3 Surface3D Plots

The example *mgrid2.py* is re-written to make a surface plot using the same equation in *surface3d.py* and the result is shown in figure 4.7.3(a).

*Example* **sufrace3d.py**

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(projection = '3d')
x = np.linspace(-10, 10, 100)
y = np.linspace(-10, 10, 100)
xx, yy = np.meshgrid(x, y)
z = xx ** 2 + yy **2    # also try with z = sin(xx) + sin(yy)
ax.plot_surface(xx, yy, z, cmap=plt.cm.jet, cstride=1)
plt.show()
```

### 4.7.4 Spherical harmonics

The spherical harmonics are the angular portion of the solution to Laplace's equation in spherical coordinates where azimuthal symmetry is not present. Program ylm.py plots the spherical harmonics $Y_m^l$ and modulates the radius using it's value. The output is shown in figure4.10. It may be repeated with other l and m values

*Example* **ylm.py**

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm, colors
from mpl_toolkits.mplot3d import Axes3D
from scipy.special import sph_harm
phi = np.linspace(0, np.pi, 100)
theta = np.linspace(0, 2*np.pi, 100)
phi, theta = np.meshgrid(phi, theta)
l = 1    # repeat for different l,m combinations
```

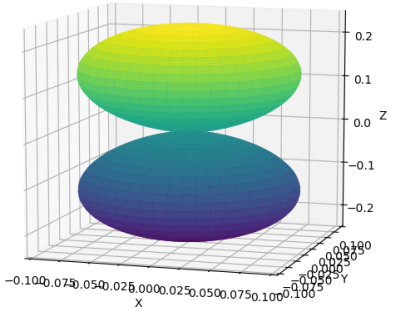



Figure 4.10: Output of ylm.py

```
m = 0
realSH = (sph_harm(m, 1, theta, phi).real)**2
radius = realSH   # Modulate the radius to visualize
x = radius * np.sin(phi) * np.cos(theta)
y = radius * np.sin(phi) * np.sin(theta)
z = radius * np.cos(phi)
fig = plt.figure(figsize=plt.figaspect(1.))
ax = fig.add_subplot(projection = '3d')
ax.plot_surface(x, y, z)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.show()
```

### 4.7.5 Animating 3D plots

The program **anim-line3d,py** listed below changes the viewing angle of a 3D lineplot, within the animation routine, to generate a continuously rotating image. The elevation and azimuth angle are updated.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation as fa
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(projection = '3d')
z = np.linspace(0, 20, 100)
x = np.sin(z)
y = np.cos(z)
ax.axis('off')
ax.plot3D(x,y,z)
def animate(phi):
ax.view_init(phi, phi) #both elevation and azimuth updated
```

```
anim = fa(fig, animate, frames=360, interval=10)
plt.show()
```

## 4.8 Exercises

1. Plot a sine wave using markers +, o and x using three different colors.
2. Plot $\tan\theta$ from $\theta$ from $-2\pi$ to $2\pi$, watch for singular points.
3. Plot a circle using the polar() function.
4. Generate a triangular wave using Fourier series.
5. Evaluate $y = \sum_{n=1}^{n=\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!}$ for 10 terms.
6. Write a Python program to calculate sine function using series expansion and plot it.
7. Write a Python program to plot $y = 5x^2 + 3x + 2$ (for x from 0 to 5, 20 points), with axes and title. Use red colored circles to mark the points.
8. Write a Python program to plot a Square wave using Fourier series, number of terms should be a variable.
9. Write a Python program to read the x and y coordinates from a file, in a two column format, and plot them.
10. Plot $x^2 + y^2 + z^2 = 25$ using matplotlib 3D.
11. Make a plot z = sin(x) + sin(y) using imshow() , from $-4\pi$ to $4\pi$ for both x and y.
12. Write Python code to plot $y = x^2$, with both the axes labeled.
13. Plot the following curves
    (a) $r^2 = a^2 \cos 2\theta$
    (b) $y = \sqrt{2\pi} e^{-x^2/2}$
    (c) $a\cosh(x/a)$
    (d) $\sin(a\theta)$ for a = 2, 3, and 4
14. Plot $x = \frac{a\sqrt{2}\cos\theta}{\sin^2\theta+1}$ , $y = \frac{a\sqrt{2}\cos\theta\sin\theta}{\sin^2\theta+1}$ for different values of a
15. Plot the curves described on https://www.matematica.pt/en/useful/list-curves.php[6]

[6] You may manipulate the equations to express $y$ in terms of $x$. Otherwise search the curve by name and you will find it listed in the parametric form. Use that for plotting.

# Chapter 5

# Symbolic Computation, SymPy

Before computers appeared on the scene, problem solving usually consisted of solving the relevant algebraic equations and evaluating the results by numerical computation. The variables are substituted with their numeric values in the final stage of the calculation. With the advent of computers, it bacame possible to tackle problems lacking analytic solutions also, by using various numerical methods. Numerical computation uses floating point representation of numbers, which is not exact. The precesion depends on the number of bits used for representing one number. Repeated arithmetic operations result in accumulated errors, affecting the accuracy of the results. In spite of these difficulties, numerical methods have made highly impressive contributions to various branches of science and engineering. Modern computers made numerical calculations much easier and it became commonplace to do enormous calculations, which in some cases made it possible to avoid laborious algebraic manipulations. As a result, scientifc computation become synonymous with numerical computation.

The advances in symbolic computation software is making the symbiosis between symbolic and numeric computation possible again. The principal focus is on restating algebraic formulas and methods in a form suitable for the problem at hand, so that the eventual numeric computation can be cheaper, more accurate, and less sensitive to errors. Symbolic computation aims at the automation of the steps of mathematical problem solving that precede evaluating numerical models and that, to a large extent, are still the domain of human problem solvers. Used together, symbolic and numeric computation enhance each other.

To illustrate the difference that can be made by Symbolic Computation, let us take a simple example of finding the area under a curve using integration. We will use the equation of a circle

$$y = \sqrt{r^2 - x^2} \tag{5.1}$$

We assume unit radius and the area under the curve from $x = 0$ to $x = 1$ can be obtained by integrating the equation 5.1. This has been explained in section 7.3.2 on Simpson's 1/3-Rule. In this case we already know the analytical result given by $A = \pi r^2/4$=0.7853981633974483. The area calculated using Simpson's 1/3 rule (area divided into 100 slices) is 0.7852833013348068. The error is around 0.015 percent.

Analytical solutions are better (if one is available) but doing them manually is tedius and error prone. This is were Symbolic Computation becomes useful. Symbolic computation software uses algorithms to manipulate the symbols representing mathematical objects. The symbols include the usual representation systems for numbers like the integers and rationals. But it also includes polynomials, rational and trigonometric functions, algebraic numbers, groups, ideals, and tensors, among others. In fact, it is possible, in theory, to carry out any heuristic or algorithmic

| | |
|---|---|
| **Python** | A computer programming language. A way to describe algorithms to a computer. |
| **IPython** | A Python *interpreter:* A computer application that provides a convenient, interactive mode for executing Python commands and programs. |
| **Spyder** | An *integrated development environment* (IDE): A computer application that includes IPython, a tool to inspect variables, a text editor for writing and debugging programs, and more. |
| **NumPy** | A standard library that provides numerical arrays and mathematical functions. |
| **PyPlot** | A standard library that provides visualization tools. |
| **SciPy** | A standard library that provides scientific computing tools. |
| **Anaconda** | A *distribution*: A single download that includes all of the above and provides access to many additional libraries for special purposes. It also includes a *package manager* that helps you to keep everything up to date. |

Table 1.1: Elements of the Python environment described in this tutorial.

Appendix A describes how to install and launch Python. Most of the code that follows will run with any Python distribution. However, since we cannot provide instructions for every available version of Python and every integrated development environment (IDE), we have chosen the following particular setup:

- The Anaconda distribution of Python 3.4, available at `store.continuum.io/cshop/anaconda/`. Many scientists use an earlier version of Python (such as version 2.7). Appendix C discusses the two minor changes needed to adapt the codes in this tutorial for earlier versions.
- The Spyder IDE, which comes with Anaconda or can be downloaded at `code.google.com/p/spyderlib/`.
  Any programming task can be accomplished with a different IDE—or with no IDE at all—but this tutorial will assume you are using Spyder. Other IDEs are available, such as IDLE, which comes with every distribution of Python, and Web-based IPython Notebooks.

The choice of distribution is a matter of personal preference. We chose Anaconda because it is simple to install, update, and maintain, and it is free. You may find a different distribution is better suited to your needs. Enthought Canopy is another free, widely used Python distribution.

### 1.2.1 IPython console

Upon launch, Spyder opens a window that includes several *panes*. See Figure 1.1.

There is an Editor pane on the left for editing program files (*scripts*). There are two panes on the right. The top one may contain Object Inspector, Variable Explorer, and File Explorer tabs. If necessary, click on the Variable Explorer tab to bring it to the front. The bottom-right pane should include a tab called "IPython Console"; if necessary, click it now.[6] It provides the command line interpreter that allows you to execute Python commands interactively as you type them.

If your window layout gets disorganized, do not worry. It is easy to adjust. The standard format for Spyder is to open with a single window, divided into the three panes just described. Each pane can have multiple tabs. If you have unwanted windows, close them individually by clicking on their "Close" buttons. You can also use the menu `View>Panes` to select panes you want to be visible and deactivate those you do not want. `View>Reset window layout` will restore the standard layout, but

[6] If no IPython console tab is present, you can open one from the menu at the top of the screen: `Consoles>Open an IPython console`.

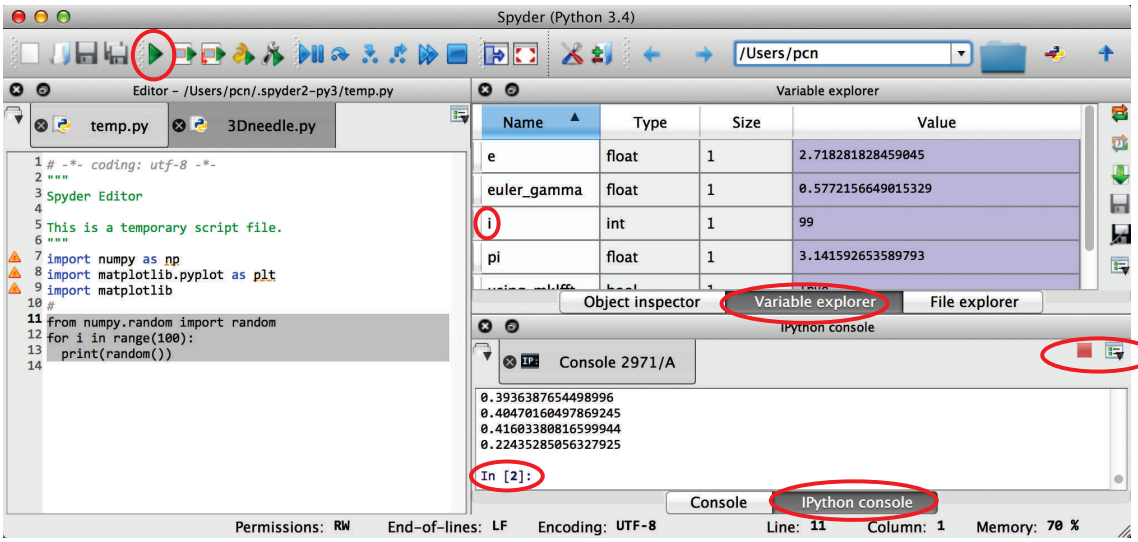



Figure 1.1: The Spyder display. Red circles have been added to emphasize (*from top to bottom*) the RUN button, a variable in the Variable Explorer, the tab that brings the Variable Explorer to the front in its pane, the Interrupt buttons, the IPython command prompt, and the tab that brings forward the IPython Console pane. Two scripts are open in the Editor; `temp.py` has been brought to the front by clicking its tab at the top of the Editor pane.

it will not activate panes that have been closed or deactivated, so perform this step after activating and deactivating panes.

Click in the IPython console. Now, things you type will show up after the *command prompt*. By default, this will be something like

```
In[1]:
```

Try typing the lines of code in Section 1.1.3, hitting `<Return/Enter>` after each line. Python responds immediately after each `<Return/Enter>`, attempting to perform whatever command you entered.[7]

*Python code consists entirely of plain text, with no formatting.*

Any formatting shown in this tutorial, including fonts and coloring, was added for readability. It is not something you need to worry about while entering code. Similarly, the line numbers shown on the left of code samples are there to allow us refer to particular lines. Don't type them. Python will assign and show line numbers when you work in the Editor, and will use them to tell you where it thinks you have made errors. They are not part of the code.

This tutorial uses the following color scheme in displaying code:

- Built-in functions and reserved words are displayed in boldface green type: `print("Hello, world!")`. You do not need to import these functions.
- Python errors and runtime exceptions are displayed in boldface red type: `SyntaxError`. These are also part of basic Python.

[7] This tutorial uses the word "command" to mean any Python statement that can be executed by an interpreter. Assignments like `a=1`, function calls like `plt.plot(x,y)`, and special instructions like `%reset` are commands.

- Functions and other objects from NumPy and PyPlot are displayed in boldface black type: `np.sqrt(2)`, or `plt.plot(x,y)`. We will assume that you import NumPy and PyPlot at the beginning of each session and program you write.[8]
- Functions imported from other libraries are displayed in blue boldface type:
  `from scipy.special import factorial`
- Strings are displayed in red type: `print("Hello, world!")`
- Comments are displayed in oblique blue type: `# This is a comment.`
- Keywords in function arguments are displayed in oblique black type:
  `np.loadtxt('data.csv', delimiter=',')`. Keywords must be entered exactly as shown.
- Keystrokes are displayed within angled brackets: <Enter/Return> or <Ctrl-C>.
- Buttons you can click with a mouse are displayed in small caps within a rectangle: RUN. Some buttons in Spyder have icons rather than text, but hovering the mouse pointer over the button will display the text shown in this tutorial.
- Most other text is displayed in normal type.

Click on the Variable Explorer tab. Each time you enter a command and hit <Return/Enter>, the contents of this pane will reflect any changes in Python's state: Initially empty, it will display a list of all your variables and a summary of their values. When a variable contains many values (like an array), you can double-click its entry in this list to open a spreadsheet that contains all the values of the array. You can copy from this spreadsheet and paste into other applications.

At any time, you can reset Python's state by quitting and relaunching it, or by executing the command

```
%reset
```

Since you are about to delete everything that has been created in this session, you will be asked to confirm this irreversible operation.[9] Type `y` and hit <Return/Enter> to proceed. (Commands that begin with a `%` symbol are **magic commands**, that is, commands specific to the IPython interpreter. They will not work in a basic Python interpreter, or in scripts that you write. To learn more about these, type `%magic` at the IPython command prompt.)

***Example:*** Use the `%reset` command, then try the following commands at the prompt. Explain everything you see happen:

```
q
q == 2
q = 2
q == 2
q == 3
```

*Solution:* Python complains about the first two lines: Initially, the symbol `q` is not associated with any object. It has no value, and so expressions involving it cannot be evaluated. Altering Python's state in the third line above changes this situation, so the last two lines do not generate errors.

[8] See Section 1.3 (page 11).
[9] If IPython does not seem to respond to `%reset`, try scrolling the IPython console up manually to see the confirm query.

***Example:*** Now clear Python's state again. Try the following at the prompt, and explain everything that happens. (It may be useful to refer to Section 1.1.4.)

```
a = 1
a
b = a**2 - a
b
a = 2
print(a)
print(b)
b = a**2 - a
a, b
print(a, b)
```

*Solution:* The results from the first four lines should be clear: We assign values to the variables `a` and `b`. In the fifth line, we change the value of `a`, but because Python only remembers the *value* of `b` and not its relation to `a`, its value is unchanged until we update it explicitly in the eighth line.

When entering code at the command prompt, you may run into a confusing situation where Python seems unresponsive.

> *If a command contains an unmatched* `(`, `[`, *or* `{`, *then Python continues reading more lines, searching for the corresponding* `)`, `]`, *or* `}`.

If you cannot figure out how to match up your brackets, you can abort the command by pressing `<Esc>`.[10] You can then retype the command and proceed with your work.

The examples above illustrate an important point: An assignment statement does not display the value that it assigns to a variable. To see the value assigned to a variable in an IPython session, enter the variable name on a line by itself. Alternatively, the `print()` command can be used to display values.[11] Note that "print" does not cause anything to come out of a printer; instead, it displays the requested information in the IPython console. If you need hard copy output, you can have your program write the desired information to a plain text file (see Section 3.2.2), then print that file in the usual way.

The last two lines of the example above illustrate how to see the values of multiple objects at once. Notice that the output is not exactly the same.

You can end a command by starting a new line. Or, if you wish, you can end a command with a semicolon (`;`) and *no* new line, then input another command on the same line. Sometimes this is convenient, but it's best not to make too much use of this ability. Your code may take up fewer lines, and Python won't care, but human readability will suffer.

It is also possible to make multiple assignments with a single = command. This is an alternative to using semicolons. Both of the following lines assign the same values to their respective variables:

```
a = 1; b = 2; c = 3
A, B, C = 1, 2, 3
```

Either side of the second command may be enclosed in parentheses without affecting the result.

---

[10] `<Esc>` cancels the current command in Spyder. In another IDE or interpreter, you may need to use `<Ctrl-C>` instead.

[11] In scripts that you write, Python will evaluate an expression *without* showing anything to the screen; if you want output, you must give an explicit `print()` command. Scripts will be discussed in Section 2.6.

These lines illustrate ways to save space and reduce typing with Python. However, you should try to make the *meaning* of your code as clear as possible. Clarity is worth a few extra seconds of typing or a few extra lines in a file.

In some situations, you may wish to use a very long command that doesn't fit on one line. For such cases, you can end a line with a backslash (\). Python will then continue reading the next line as part of the same command. Try this:

```
q = 1 + \
2
q
```

A single command can even stretch over multiple lines:

```
xv\
a\
l\
= 1 + \
2
```

This will create a variable `xval` assigned to the value 3. To write clear code, you should use this option sparingly.

### 1.2.2 Error messages

You should already have encountered an error message by now. When Python detects an error, it tells you where it encountered the error, provides a fragment of the code surrounding the statement that caused the problem, and tells you which general kind of error it detected among the many types it recognizes. For example, you will receive a **`NameError`** if you try to evaluate an undefined variable. See Appendix B for a description of common Python errors and some hints for interpreting the resulting messages.

### 1.2.3 Sources of help

The definitive documentation on Python is available online at `www.python.org/doc`. However, in many cases you'll find the answers you need more quickly by other means, such as asking a friend, searching the Web, or visiting `stackoverflow.com`.

Suppose that you wish to evaluate the square root of 2. You type `2**0.5` and hit `<Return/Enter>`. That does the job, but Python is displaying 16 digits after the decimal point, and you only want 3. You think there's probably a function called **`round`** in Python, but you are not sure how to use it or how it works. You can get help directly from Python by typing **`help`**`(`**`round`**`)` at the command prompt. You'll see that this is indeed the function you were looking for:

```
round(2**0.5, 3)
```

gives the desired result.

In Spyder, there are additional ways to get help. Type **`round`** at the command prompt, but do not hit `<Return/Enter>`. Instead hit `<Cmd-I>` or `<Ctrl-I>` (for "Information"). The information that was displayed in the IPython console when you issued the **`help`** command now shows up in the Object Inspector, and in a format that is easier to navigate and read, especially for long entries. You can also use the Object Inspector without entering anything at the command prompt: Try entering

**pow** in the "Object" field at the top of the pane. The Object Inspector will provide information about an alternative to the ** operation for raising a number to a power.

In IPython, you can also follow or precede the name of a module,[12] function, or variable by a question mark to obtain help: **round?** or **?round** provides the same information as **help**(**round**) and is easier to type.

When you type **help**(...), Python will print out the information it has about the expression in parentheses if it recognizes the name. Unfortunately, Python is not as friendly if you don't know the name of the command you need. Perhaps you think there ought to be a way to take the square root of a number without using the power notation. After all, it is a pretty basic operation. Type **help**(sqrt) to see what happens when Python does not recognize the name you request.

To find out what commands are currently available to you, you can use Python's **dir**() command. This is short for "directory," and it returns a list of all the modules, functions, and variable names that have been created or imported during the current session (or since the last **%reset** command). Ask Python for help on **dir** to learn more. Nothing in the output of **dir** looks promising, but there is an item called __builtin__. This is the collection of all the functions and other objects that Python recognizes when it first starts up. It is Python's "last resort" when hunting for a function or variable.[13] To see the list of built-in functions, type

```
dir(__builtin__)
```

There is no sqrt function or anything like it. In fact, *none* of the useful mathematical functions like sin, cos, or exp show up!

Python cannot help you any further at this point. You now have to turn to outside resources. Good options include books about Python, search engines, friends who know more about Python than you do, and so on.

> *In the beginning, a lot of your coding time will be spent using a search engine to get help.*

The **sqrt** function we seek belongs to a library. Later we will discuss how to access libraries of useful functions that are not automatically available with Python.

**Your Turn 1A**

Before proceeding, try a Web search for
`how to take square roots in python`

### 1.2.4 Good practice: Keep a log

As you work through this tutorial, you will hit many small roadblocks—and some large ones. How do you label graph axes? What do you do if you want a subscript in a graph axis label? The list is endless. Every time you resolve such a puzzle (or a friend helps you), *make a note of how you did it* in a notebook or in a dedicated file somewhere on your computer. Later, looking through that log will be much easier than scanning through all the code you wrote months ago (and less irritating than asking your friend over and over).

[12] Modules will be discussed in Section 1.3.
[13] See Section D.4 for an explanation of how Python searches for variables and other objects.

## 1.3 PYTHON MODULES

We found earlier that Python does not have a built-in `sqrt` function. Even your calculator has that! What good is Python? Think for a moment about how, exactly, your calculator knows how to find square roots. At some point in the past, someone came up with an algorithm for computing the square root of a number and stored it in the permanent memory of your calculator. Someone had to create a *program* to calculate square roots.

Python is a programming language. A Python interpreter understands a basic set of commands that can be combined to perform complex tasks. Python also has a large community of developers who have created entire libraries of useful functions. To gain access to these, however, you need to **import** them into your working environment.

> *Use the* **`import`** *command to access functions that do not come standard with Python.*

### 1.3.1 `import`

At the command prompt, type

```
import numpy
```

and hit `<Return/Enter>`. You now have access to a large number of useful functions. You have imported the NumPy module, a collection of tools for numerical calculation using Python: "Numerical Python." To see what has been gained, type **`dir(numpy)`**. You will find nearly 600 new options at your disposal, and one of them is the `sqrt` function you originally sought. You can search for the function within NumPy by using the command **`numpy.lookfor('sqrt')`** (This will often return more than you need, but the first few lines can be quite helpful.) Now that you have imported NumPy, try

```
sqrt(2)
```

What's going on? You just imported a square root function, but Python tells you that `sqrt` is not defined! Try instead

```
numpy.sqrt(2)
```

The `sqrt` function you want "belongs" to the **`numpy`** module you imported. Even after importing, you still have to tell Python where to find it before you can use it.

> *After you have imported a module, you can call its functions by giving the module name, a period, and then the name of the desired function.*

### 1.3.2 `from ... import`

There is another way to import functions. For example, you may wish access to all of the functions in NumPy without having to type the "**`numpy.`**" prefix before them. Try this:

```
from numpy import *
sqrt(2)
```

This is convenient, but it can lead to trouble when you want to use two different modules simultaneously. There is a module called **`math`** that also has a `sqrt` function. If you import all of the

functions from **math** and **numpy**, which one gets called when you type `sqrt(2)`? (This is important when you are working with arrays of numbers.) To keep things straight, it is best to avoid the "**from** `module` **import** ⋆" command. Instead, import a module and explicitly call **numpy.sqrt** or **math.sqrt** as appropriate. However, there is a middle ground. You can give a module any *nickname* you want. Try this:

```
import numpy as np
np.sqrt(2)
```

Now we can save typing and still avoid confusion when functions from different modules have the same name.

There may be times when you only want a specific function, not a whole library of functions. You can ask for specific functions by name:

```
from numpy import sqrt, exp
sqrt(2)
exp(3)
```

We now have just two functions from the NumPy module, which can be accessed without the "**numpy.**" prefix. Notice the similarity with the "**from numpy import** ⋆" command. The asterisk is a "wildcard" that tells the import command to grab everything.

There is one more useful variant of importing that allows you to give the function you import a custom nickname:

```
from numpy.random import random as rng
rng()
```

We now have a random number generator with the convenient name `rng`.

This example also illustrates a module within a module: **numpy** contains a module called **numpy.random**, which in turn contains the function **numpy.random.random**. When we typed **import numpy**, we imported many subsidiary modules. Instead, we can import just one function by using **from** and providing a precise specification of the function we want, where to find it, and what to call it.

### 1.3.3 NumPy and PyPlot

The two modules we will use most often are called NumPy and PyPlot. NumPy provides the numerical tools we need for generating and analyzing data, and PyPlot provides the tools we need to visualize that data. PyPlot is a subset of the much larger Matplotlib library. From now on, we will assume that you have issued the following commands:

```
import numpy as np
import matplotlib.pyplot as plt
```

This can also be accomplished with the single command

```
import numpy as np, matplotlib.pyplot as plt
```

You should execute these commands at the start of every session. You should also add these lines at the beginning of any scripts that you write. You will also need to reimport both modules each time you use the **%reset** command.

Give the **%reset** command, then try importing these modules now. Explore some of the functions available from NumPy and PyPlot. You can get information about any of them by using **help()** or any of the procedures described in Section 1.2.3. You will probably find the NumPy help files considerably more informative than those for the built-in Python functions. They often include examples that you can try at the command prompt.

Now that we have these collections of tools at our disposal, let's see what we can do with them.

## 1.4 PYTHON EXPRESSIONS

The Python language has a **syntax**—a set of rules for constructing expressions and statements that it can interpret. In this section, we will look at some simple expressions to get an idea of how to communicate with Python. The basic building blocks of expressions are literals, variable names, operators, and functions.

### 1.4.1 Numbers

You can enter explicit numerical values (**literals**) in various ways:

- `123` and `1.23` mean what you might expect. When entering a large number, however, don't separate groups of digits by commas. (Don't type `1,000,000` if you mean a million.)
- `2.3e5` is convenient shorthand for $2.3 \cdot 10^5$.
- `2+3j` represents the complex number $2 + 3\sqrt{-1}$. (Engineers may find the name j for $\sqrt{-1}$ familiar; mathematicians and physicists will have to adjust to Python's convention.)

Python stores numbers internally in several different formats. However, it will usually convert from one type to another when necessary. Beginners generally don't need to consider this. Just be aware that a number must be an integer for some purposes. If you need to force a number to be an integer (for example, when indicating an entry in a list), you can use the function **int**.

### 1.4.2 Arithmetic operations and predefined functions

Python includes basic arithmetic operators, for example, `+`, `-`, `*` (multiplication), `/` (division), and `**` (exponentiation).

*Python uses two asterisks, `**`, to denote raising a number to a power.*

For example, `a**2` means "`a` squared." The notation `a^2` used by some other math software means something quite different to Python.

Unlike standard mathematics notation, you may not omit multiplication signs. Try typing

```
(2)(3)
a = 2; a(3)
3a
3 a
```

Each of these commands produces an error message. None, however, generate a message like, "`You forgot a '*'!`" Python used its evaluation rules, and these expressions didn't make sense. Python doesn't know what you were trying to express, so it can't tell you exactly what is wrong. *Study these*

*error messages*; you'll probably see them again. See Appendix B for a description of these and other common errors.

Arithmetic operations have the usual precedence (ordering).

*You can use parentheses to override operator precedence.*

Unlike math textbooks, Python only recognizes parentheses (round brackets) for ordering operations. Square and curly brackets are reserved for other purposes. We have already seen that parentheses can also have another meaning (enclosing the arguments of a function). Yet another meaning will appear later: specifying a tuple. Python uses context to figure out which meaning to use.

For example, if you want to use the number $\frac{1}{2\pi}$, you might type `1/2*np.pi`. (Basic Python does not know the value of $\pi$, but NumPy does.) Try it. What goes wrong, and why? You can fix the expression by inserting parentheses. Later we'll meet other kinds of operators like comparisons and logical operations. They, too, have a precedence ordering, which you may not wish to memorize. Instead, use parentheses liberally to specify exactly what you mean.

To get used to Python arithmetic operations, figure out what famous math problem these lines solve, and check that Python got it right:

```
a, b, c = 1, -1, -1
(-b + np.sqrt(b**2 - 4*a*c))/(2*a)
```

Recall that **`np.sqrt`** is the name of a *function* that Python does not recognize when it launches, but which becomes available once we import the NumPy module. When Python encounters the expression in the second line, it does the following:

1. Evaluates the **argument** of the **`np.sqrt`** function—that is, everything inside the pair of parentheses that follows the function name—by substituting values for variables and evaluating arithmetic operations. (The argument may itself contain functions.)
2. Interrupts evaluation of the expression and executes a piece of code named **`np.sqrt`**, handing that code the result found in step **1**.
3. Substitutes the value returned by **`np.sqrt`** into the expression.
4. Finishes evaluating the expression as usual.

How do you know what functions are available for you? See Section 1.2.3 above: Type **`dir`**`(`**`np`**`)` and **`dir`**`(__builtin__)` at the IPython console prompt.

A few symbols in Python and NumPy are predefined. These do not require any arguments or parentheses. Try **`np.pi`** (the constant $\pi$), **`np.e`** (the base of natural logarithms e), and **`1j`** (the constant $\sqrt{-1}$). NumPy also provides the standard trig functions, but be alert when using them:

*The trig functions* **`np.sin`**, **`np.cos`**, *and* **`np.tan`** *all treat their arguments as angles expressed in radians.*

### 1.4.3 Good practice: Variable names

Note that Python offers you no protection against accidentally changing the value of a symbol: If you say **`np.pi`**`=22/7`, then until you change it or reset Python, **`np.pi`** will have that value. It is even possible to create a variable whose name supplants a built-in function, for example, **`round`**`=3`.[14]

[14] This can be undone by deleting your version of **`round`**: Type **`del`**`(`**`round`**`)`. Python will revert to its built-in definition.

This illustrates another good reason for using the "`import numpy as np`" command instead of the "`from numpy import *`" command: You are quite unlikely to use the "`np.`" prefix and name your *own* variables `np.pi` or `np.e`. Those variables retain their standard values no matter how you define `pi` and `e`.

When your code gets long, you may inadvertently reuse variable names. If you assign a variable with a generic name like `x` in the beginning, you may later choose the same name for some completely different purpose. Later still, you will want the original `x`, having forgotten about the new one. Python will have overwritten the value you wanted, and puzzling behavior will ensue. You have a **name collision**.

It's good practice to use longer, more meaningful names for variables. They take longer to type, but they help avoid name collisions and make your code easier to read. Perhaps the variable you were planning to call `x` could instead be called `index`, because it indexes a list. Perhaps the other variable that caused the name collision could logically be called `total`. Later, when you ask for `index`, there will be no problem.

Variable names are case sensitive, and many predefined names are lowercase. Thus, you can avoid some name collisions by including capital letters in variable or function names you define.

Blank spaces and periods are not allowed in variable names. Some coders use capitalization in the middle of variable names ("camel humps") to denote word boundaries—for example, `whichItem`. Others use the underscore, as in `which_item`. Variable names may contain digits (`myCount2`), but they must start with a letter.

Some variable names are forbidden. Python won't let you name variables `if`, `for`, `print`, or a handful of other **reserved words**. You can find them with a Web search for `python reserved words`.

### 1.4.4 More about functions

You may be accustomed to thinking of a function as a machine that eats exactly one number (its argument) and spits out exactly one number (its result). Some Python functions do have this character, but Python has a much broader notion of function. Here are some illustrations. (Some involve functions that we have not seen yet.)

- A function may have a single argument, multiple arguments separated by commas, or no arguments at all. Try typing `np.random.random()` at the command prompt.
- A function may allow a *variable* number of arguments, and behave differently depending on how many you supply. For example, we will see functions that allow you to specify certain options using *keyword arguments*. Each function's help text will describe the allowed ways of using it.
- A function may also *return* more than one value. You can capture the returned values by using a special kind of assignment (see Section 5.3). The number of values returned can even vary depending on the arguments you supply.[15]
- A function may change Python's state in ways other than by returning a result. For example, `plt.savefig` saves a plot to a file on your computer's hard drive. Other possible side effects include writing text into the IPython console: `print('hello')`.

If a function accepts two or more arguments, how does it know which is which? In mathematical notation, the *order* of arguments conveys this information. For example, if we define $f(x, y) = x\,\mathrm{e}^{-y}$,

[15] T2 Technically, a Python function always returns a single object. However, this object may be a tuple that contains several values.

then later $f(2, 6)$ means $2 \cdot e^{-6}$: The first given value (2) gets substituted for the first named variable in the definition ($x$), and so on. This scheme is also the standard one used by Python. But when a function accepts many arguments, relying on order can be annoying and prone to error. For this reason, Python has an alternative approach called **keyword arguments**:

```
f(y=6, x=2)
```

instructs Python to execute a function named `f`, initializing a variable named `y` with the value 6 and another named `x` with the value 2. With this syntax, you need not adhere to any particular order in giving the arguments. (However, you must use their correct names, which you can learn from the function's documentation.) Many functions will even let you *omit* specifying values for some or all of their arguments; if you omit them, the function supplies default values. Keyword arguments will be discussed in Section 5.1.3.

# CHAPTER 2

# Structure and Control

Heisenbug: *A computer bug that disappears or alters its characteristics when an attempt is made to study it.*
—Wikipedia

Much of the power of computation comes from the ability to do repetitive tasks extremely rapidly. You need to understand how to formulate instructions for this sort of task, so that your electronic assistant can do all the steps without your supervision. This chapter describes two key elements:

- Grouping data into *data structures* and
- Grouping code into repeated blocks (looping) and contingent blocks (branching).

## 2.1 OBJECTS AND THEIR METHODS

In Python, everything is an **object**. An object is a combination of data and functions. Even simple things like integers are objects in Python. (Type `dir(1)` to see all the data and functions associated with what you might have thought was "just a number.") When processing an assignment statement, Python attaches, or **binds**, a **name** (a symbol like `x` or `filename`) to an object.[1] Later, you can refer to the object by this name, or you can reassign the name to a different object. Names used in this way are also called **variables**.

Let's look at a few examples to see how this works.

When you type `f=2.5`, Python begins by evaluating the right-hand side of the assignment statement. The only thing it finds is a numeric **literal**, that is, an expression whose value is the expression itself. (Thus, `1+1` is an expression, but it is not a literal because its value is `2`. In contrast, `2` is a literal because its value is `2`.) Continuing with the assignment, Python creates a `float` object to store the number 2.5. The value of a `float` object is a **floating-point number**: As in scientific notation, the decimal point can "float" to give the same number of significant figures, regardless of how large or small the number is. To complete the assignment, Python binds the name `f` to the new `float` object. If no variable with the name `f` exists, then Python will create one. If `f` already exists, Python reassigns it to the value of the current expression.

Similarly, writing `s='Hello'` creates a `str` object, whose value is the string literal `'Hello'`, then binds the name `s` to that object. (Section 2.3 will discuss strings.) It does not matter if `s` was previously bound to a `float` or some other sort of object. Types belong to objects, not to the variables that point to them.

Writing `L=[1,2,3]` creates a `list` object whose value is a sequence of three `int` objects. The value of an `int` object is restricted to be an integer. (Section 2.2 will discuss lists.)

[1] T2 Appendix D describes in greater detail how Python handles assignment statements.

There is more to an object than just its value. In general, objects consist of both **attributes** (data) and **methods** (functions). The value of an object is one of its attributes. A method of an object is a specialized function that can act on its attributes. A method may also accept additional arguments. To access a method, you have to provide the name of the object, followed by a period, the name of the method, and a pair of parentheses enclosing any arguments. Try the following with the `float` object `f`, the `str` object `s`, and the `list` object `L` introduced above:

1. `f.is_integer()` : Every `float` object has a method called **`is_integer`** that determines whether its value is equivalent to an integer. The method returns **`True`** if the fractional part of the value is zero. In this case, the method returns the value **`False`**. The function `f.is_integer` does not require any additional arguments, but we must nevertheless place an empty pair of parentheses after its name.

   > *Providing a function name with no parentheses will not raise an error. Instead, it will return the function object.*

   For example, `f.is_integer` returns `<function float.is_integer>`, rather than the result of evaluating the function.
2. `s.swapcase()` : Every `str` object has a method called **`swapcase`** that returns a new string whose value is the value of the original string with the case of every letter reversed.
3. `L.reverse()`: Every `list` object has a method called **`reverse`** that does not return a value but does modify the value of `L`. The output of **`print`**`(L)` will show that the order of the list has been reversed.
4. `L.pop(1)` : Every list also has a method called **`pop`** that will return the item at a specified location and remove it from the list. Thus, this method returns a value *and* modifies the object's data. When called with no argument, `L.pop()` will remove the first item in the list.

The examples illustrate that every object possesses a suite of attributes and methods. Even literals have the standard methods appropriate to their type: Try `'Hello'.swapcase()`, or `(5.0).is_integer()`. You can see all the methods associated with an object by using the **`dir`** function.

A method can use its parent object's data (examples **1**–**4** above). It can modify that data (examples **3, 4**) or not (examples **1, 2**). It can return a value (examples **1, 2, 4**) or not (example **3**). It can accept additional arguments in parentheses (example **4**) or not (examples **1**–**3**). The documentation of a method will describe which of these behaviors apply to that particular method.

There are some objects in Python whose data is fixed once the object is created. They are called **immutable objects**.

> *The methods of an immutable object do not change its attributes.*

String and numeric objects are immutable, which is why none of the examples above change their values.[2] Lists are **mutable**: Their methods can modify their data. Examples **3** and **4** illustrated this. Shortly, we'll introduce NumPy arrays, which are also mutable.

Try creating all the example objects mentioned in this section and see how they appear in Spyder's Variable Explorer pane. Experiment with some of their methods.

Many objects have attributes that describe properties of the object other than its value. These are stored as part of the object and require no computation. Python just looks up the attribute and returns its value (see Section 2.2.2). The syntax for accessing an attribute is `object.attribute`. No

[2] The method **`swapcase`** returns a modified copy of the original string; it does not affect the original string.

parentheses are needed because an attribute is not a function. The expression starts with the name of the broader thing (`object`), followed by a period and the name of a specific thing (`attribute`).

You can create your own objects, and give them whatever attributes and methods you like, but that goes beyond the scope of this tutorial. To learn more, try a Web search for "`python classes`."

## 2.2 LISTS, TUPLES, AND ARRAYS

Much of the power of computer programming comes when we handle numbers in batches. A batch of numbers could represent a single mathematical object like a force vector, or it could be a set of points at which you wish to evaluate a function. To process a batch of numbers, we first need to collect them into a single data structure. The most convenient Python data structure for our purposes is the NumPy array, described below. Lists and tuples are also useful.

### 2.2.1 Creating a list or tuple

Python comes with a built-in `list` object type. Any set of objects enclosed by square brackets and separated by commas creates a list:[3]

```
L = [1, 'a', max, 3 + 4j, 'Hello, world!']
```

A **tuple** is a type of immutable object similar to a list. To create a `tuple` object, type `t=(2,3)`. Note the use of round parentheses, `()`, in contrast to the square brackets, `[]`, of a list. Python knows you want a tuple because it scans the text between the parentheses and finds a comma. Without any commas, parentheses are interpreted as specifying the order of operations.[4] Like a list, a tuple consists of an ordered sequence of objects. Unlike a list, however, a tuple is immutable. Its elements cannot be reassigned, nor can their order can be modified. We will use tuples to specify the shape of an array in Section 2.2.2 and later sections.

### 2.2.2 NumPy arrays

A list is a sequence (ordered collection) of objects, endowed with methods that can perform certain operations on its contents like searching and counting. You can do numerical calculations with lists; however, a `list` object is generally *not* the most convenient data type for such work. What we usually want is a *numerical array*—a grid of numbers, all of the same type. The NumPy module provides an `array` class ideally suited to our needs. Knowing that all of the elements in an array are numbers of the same type allows NumPy to do efficient calculations on an entire array. Because Python lists can contain any mix of object types, processing their elements is far less efficient.

> *A Python list is* not *the same as a NumPy array.*
> *We will almost always use NumPy arrays.*

You can create a one-dimensional array by using the command

```
a = np.zeros(4)
```

[3] A list can contain just one object, but note that `2.71`, a numeric object, is not the same thing as `[2.71]`, a list containing one numeric object. A list can even contain *no* objects: `L=[]`.
[4] If you want a tuple with just one element, write `(0,)`.

The function **np.zeros** requires one argument that describes the shape of the array—in this case, the integer 4—and creates an array with four elements. Then, it sets all four entries to zero.

In mathematics, we often need arrays of numbers with two or more dimensions. Try

```
a = np.zeros( (3, 5) )
```

and see what you get. Here **np.zeros** again accepts a single argument. In this case, it is the tuple `(3,5)`. The function creates an array with 15 entries, all equal to zero, arranged in three rows and five columns. The name `a` now refers to a `numpy.ndarray` object (an "*n*-dimensional NumPy array") that conforms to the mathematical notion of a $3 \times 5$ matrix.

**Your Turn 2A**

Create `a` by using the preceding code. Then, find it in the Variable Explorer. Double-click its value to look inside the array. Repeat with the function **np.ones** in place of **np.zeros** and see what you get. Finally, try **np.eye**`(3)`.

A *row vector* is a special case of a two-dimensional array with just one row: **np.zeros**`((1,5))`. Similarly, a *column vector* is a two-dimensional array with just one column: **np.zeros**`((3,1))`. Python will give you exactly what you ask for, so be aware of what you are requesting:

> *If you want a column vector of N zeros, use* **np.zeros**`((N,1))`*, not* **np.zeros**`(N)`*. If you want a row vector of N zeros, use* **np.zeros**`((1,N))`.

The functions **np.ones** and **np.random.random** can also create two-dimensional arrays, using the same syntax.[5]

Python can also handle arrays with more than two dimensions. Try

```
A = np.zeros( (2, 3, 4) )
B = np.ones( (2, 3, 4, 3) )
```

and inspect the resulting arrays.

Python can report how big an array is. After setting up `a`, use the command **np.size**`(a)` and see what you get. Also try **np.shape**`(a)`. This information is also displayed in the Variable Explorer.

A NumPy array is an object with its own attributes and methods. For example, it can report its own size and shape (`a.`**shape** and `a.`**size**). A NumPy array also has methods for acting on its data. Try `a.`**sum**`()`, `a.`**mean**`()`, and `a.`**std**`()`. (Of course, these methods are more useful when the array contains something other than zeros!)

Compare the output of **np.zeros**`(3)`, **np.zeros**`((1,3))`, and **np.zeros**`((3,1))`. Note, in particular, the shape of each. NumPy regards the first as neither a row nor a column vector. It is a one-dimensional array—an array with only one index. In some cases, NumPy will treat all three types of arrays the same. However, in certain matrix and vector operations, the shape of the array is important. Be certain you have arrays of the shape you need when performing these operations. You can reshape a one-dimensional array into a column vector or row vector if necessary. (See Section 2.2.9.)

[5] There is also a function for generating random numbers that requires a series of integers separated by commas rather than a tuple: **np.random.rand**`(2,3)` is equivalent to **np.random.random**`((2,3))`.

Don't confuse the concept of "three-dimensional array" with that of "three-dimensional vector." The variable `A` defined earlier is a three-dimensional array; it could represent grid points filling a volume of space, each holding a single quantity (a *scalar*). In contrast, a three-dimensional vector is just a set of three numbers (the components of the vector); it could be represented by an array like **`np.ones`**`(3)` or **`np.ones`**`((3,1))`.[6]

### 2.2.3 Filling an array with values

Perhaps you'd like to set up an array with more interesting values. The command

```
a = [2.71, 3.14, 3000]
```

creates a Python list—*not* a NumPy array—with the specified values. It contains the values we want, but we cannot perform the operations we would like with the list. To create a NumPy array from these values, we can call the function **`np.array`** on the list:

```
a = np.array( [2.71, 3.14, 3000] )
```

Making each element of the list its own list—that is, enclosing each entry in square brackets—returns a column vector, a two-dimensional array with three rows and one column:

```
a = np.array( [ [2.71], [3.14], [3000] ] )
```

**Your Turn 2B**

Try

```
a = np.array( [ [2, 3, 5], [7, 11, 13] ] )
```

and explain the result.

Usually you don't want to specify each entry in an array explicitly. For example, we frequently want to create a NumPy array of evenly spaced values over some range. NumPy provides two useful functions to do this: **`np.arange`** and **`np.linspace`**.

The function **`np.arange`**`(M,N)` creates a one-dimensional array with `M` as its first entry, `M+1` as the second, and so on, stopping just before it reaches a value that equals or exceeds `N`.

***Example:*** Try the following commands. Inspect and describe the resulting arrays.

```
a = np.arange(1, 10)
a = np.arange(5)
a = np.arange(2.1, 5.4, 0.1)
```

The last command creates a series, but each entry is incremented by 0.1 from its predecessor, not by the default increment of 1.

> *The* **`arange`** *syntax is* **`np.arange`**`(start,end,increment)`*. The increment and starting value are optional. The default start value is 0; the default increment is 1. The end value* is not included *in the array.*

Type **`help`**`(`**`np.arange`**`)` at the command prompt to learn more.

[6] T2 A three-dimensional grid of three-dimensional vectors can be represented by a *four*-dimensional NumPy array!

The function **np.linspace**(A,B,N) does a similar job. It creates a one-dimensional array with exactly N evenly spaced entries. The first entry equals A. Unlike the **np.arange** function, however, the last entry equals B exactly. You don't get to specify the spacing; Python determines what is needed to get from A to B in exactly N steps.

> *The* **linspace** *syntax is* **np.linspace**(start,end,num_steps)*. The end value* is included *in the array.*

**Your Turn 2C**

a. Try

```
a = np.arange(0, 10, 2)
b = np.linspace(0, 10, 6)
```

and explain the results. See if you can use **np.arange** and **np.linspace** to create identical arrays. [*Hint:* What happens when you add the increment to the end value in **np.arange**?]

b. Now try

```
a = np.arange(0, 10, 1.5)
b = np.linspace(0, 10, 7)
```

Observe that a and b are not the same. Explain why not, and decide which form you should use if you want to evaluate a function over the range 0 to 10.

When creating a series to compute the values of a function over a range, **np.linspace** is the appropriate function. It allows you to explicitly choose the start and end of the range (and the number of points in the series). However, when the exact spacing between points is important, **np.arange** is the appropriate function.

You *can* specify the end point using **np.arange**. The following construction will do the job:

```
x_min = 0
x_max = 10
dx = 0.1
x_array = np.arange(x_min, x_max + dx, dx)
```

Python also has a built-in function called **range**, but it does not create a numerical array of values. It useful in loops, as described in Section 2.4.

### 2.2.4 Concatenation of arrays

NumPy offers two useful methods for building up an array from smaller ones. Each takes a single argument, a list or tuple whose entries are the arrays to be combined:

- **np.hstack** (horizontal stack): The resulting array has the same number of rows as the original arrays. The arrays to be stacked must have the same number of rows.
- **np.vstack** (vertical stack): The resulting array has the same number of columns as the original arrays. The arrays to be stacked must have the same number of columns.

Try these examples, and describe the results:

```
a = np.zeros( (2, 3) )
b = np.ones( (2, 3) )
h = np.hstack( [a, b] )
v = np.vstack( [a, b] )
```

Look at the contents of `h` and `v`, and compare their shapes with those of `a` and `b`.

### 2.2.5 Accessing array elements

Once you have created an array, you can access each of its entries individually. Try the following code at the command prompt:

```
A = np.array( [2, 4, 5] )
A[0]
A[1] = 100
print(A)
```

*Array indices (offsets) are enclosed in square brackets, not parentheses.*

Using round parentheses when requesting array elements is a common error for Python beginners.

The third line above changes just one entry in the array, but it may not be the entry you expect. In Python, the indices of lists, tuples, arrays, and strings all start with 0. Thus, `A[1]` is the *second* element of `A`. The same indexing scheme can access individual characters in strings. This is not the convention that math texts use for vectors and matrices.[7]

> *Indices in Python start at 0. If $A$ is a one-dimensional array with $N$ elements, the first element is* `A[0]` *and the $N$th element is* `A[N-1]`*. Asking for element* `A[N]` *will result in an error.*

A math text would refer to `A[N-1]` as $A_N$. The notation $A_{-1}$ would probably not appear in most math texts, but Python interprets negative indices as an offset from the *end* of a list or array. For instance, `A[-1]` will access the last entry of an array. More generally, `A[-n]` refers to the $n$th element from the end of an array.

To understand how to access elements in a two-dimensional array, try the following:

```
A = np.array( [ [2, 3, 5], [7, 11, 13] ] )
A[0]
A[0][1]
A[1][2] = 999
```

In the second line above, `A[0]` returns an array whose elements are `[2,3,5]`. In the third line, `A[0][1]` then asks for the *second* item in *that* array, which is the number 3. Python also understands an abbreviated indexing scheme:

```
A = np.array( [ [2, 3, 5], [7, 11, 13] ] )
A[0, 1]
A[1, 2] = 999
```

[7] T2 The convention in Python is inherited from an older programming language called C. In that language, the name of an array is linked to the location in memory where its first value is stored. An index is interpreted as an offset. Thus, to get the *first* entry in the array, we need an offset of *zero*: `A[0]`.

Both `A[i][k]` and `A[i,k]` return the entry at the intersection of row `i+1` and column `k+1`. A mathematics text would call this entry $A_{i+1,k+1}$. Again, Python's behavior is easier to understand if you think in terms of offsets: `A[i,k]` is the entry `i` steps down and `k` steps to the right from the upper-left corner of the array.

### 2.2.6 Arrays and assignments

There appears to be little difference between an assignment statement for a variable, such as `f=2.5`, and an assignment statement for an element of an array, such as `A[1]=2.5`. However, these assignment statements are handled differently in Python, and it is important to understand the distinction.

When we access and assign elements of an array, we are using methods of an `array` object to view and modify its data.[8] In contrast, when we assign a value to a variable, as in `f=2.5`, Python binds the variable name to a new object. There is little difference in behavior until we have two names bound to the same object.

Suppose that we create two variables that point to the same `float` object and two variables that point to the same `array` object:

```
f = 2.5
g = f
A = np.zeros(3)
B = A
```

Inspect all four variables, `A`, `B`, `f`, and `g`, before and after these commands:

```
g = 3.5
A[0] = 1
B[1] = 3
```

There is no effect on `f`, even though the value of `g` is changed. However, `A` and `B` remain equal to each other. The reason is that `A` and `B` are bound to the same `array` object, and each used a method of that object to modify its data.

> *If multiple variables are bound to the same array, they can* all *use its methods to modify its data.*

This includes assignment of individual elements.

Lists behave in a similar way. Tuples are immutable objects that do not allow reassignment of their elements, so `A[0]=1` results in an error if `A` is a tuple.

There is another important difference between variable assignment and array assignment. If you type `A=1`, Python does not raise an error, even if `A` was previously undefined. However, if you try `A[0]=1` before defining `A`, Python returns an error message. The reason is that `A[0]=1` is attempting to use a method of an object called `A` to modify its data. Since no such object exists, Python cannot proceed. If you need to set the entries of an array one by one, you should first create an empty array of the appropriate size, for example, `A=np.zeros(100)`.

[8] $T_2$ The expression `A[1]` is shorthand for a method called `A.__getitem__(1)`, and `A[1]=2.5` calls the method `A.__setitem__(1,2.5)`.

### 2.2.7 Slicing

Often you will wish to extract more than one element from an array. For example, you may wish to examine the first ten elements of an array or the 17th column of a matrix. These operations are possible using a technique called **slicing**.

In Python, the colon indicates slicing when it is part of an index. The most general command for slicing a one-dimensional array is `a[start:end:stride]`. This returns a new array whose entries are

```
a[start], a[start + stride], a[start + 2*stride], ..., a[start + M*stride]
```

where `M` is the largest integer for which `start + M*stride < end`.

> *In slicing, the stride comes last. If it is omitted, the default is* `1`*. If* `start` *or* `end` *are omitted, their defaults correspond to the first and last entries, respectively.*

If the stride is omitted, the second colon may also be omitted.

Thus, if you want to see the first ten entries in `a`, you could use the expression `a[0:10:1]`, or more concisely `a[:10]`. The same syntax can be used to slice along each dimension of a multidimensional array. For example, to get a slice of the third column of a matrix, you could use `A[start:end:stride,2]`.

Suppose that you have been given experimental data in an array `A` with two columns. For your analysis, you need to divide the two columns of data into separate arrays. If you don't know the number of rows in `A`, you could start by finding it, and then assign the slices to separate variables:

```
N = np.size(A, 1)
x = A[0:N:1, 0]
y = A[0:N:1, 1]
```

A colon all by itself represents every allowed value of an index (that is, `start=0`, `end=-1`, `stride=1`). We can use this shortcut to specify our slices more concisely:

```
x = A[:, 0]
y = A[:, 1]
```

**Your Turn 2D**

Try the following commands to familiarize yourself with slicing:

```
a = np.arange(20)
a[:]
a[::]
a[5:15]
a[5:15:3]
a[5::]
a[:5:]
a[::5]
```

Explain the output you get from each line. Can you construct a slicing operation that returns only the odd entries of `a`?

Negative index values can also be used in slicing. This can be especially useful if you only want to see the last few entries in an array whose size is unknown. For example, `a[-10:]` will return the last ten elements stored in `a` (if `a` has at least ten elements).

An array slice can also appear in an assignment statement:

```
A = np.zeros(10)
A[0:3] = np.ones(3)
```

This code replaces a block of values in `A`, leaving the rest unchanged. (See Section 2.2.6.)

### 2.2.8 Flattening an array

An array may have any number of dimensions. In some cases, you may wish to repackage all of its values as a one-dimensional array. The function **`np.ravel`** does exactly this. It can be accessed as a NumPy function or an array method. NumPy arrays can also *flatten* themselves. Try the following commands, and inspect the results:

```
a = np.array( [ [1, 2], [2, 1] ] )
b = a.flatten()
c = np.ravel(a)
d = a.ravel()
```

Notice, in particular, that neither **`ravel`** nor **`flatten`** changes `a`. Each returns a one-dimensional array that contains the same elements as `a`. There is a significant difference between the methods, however. The **`flatten`** method returns a new, independent array; the **`ravel`** method returns an `array` object with access to the *same* data as `a`, but with a different shape. To see this, compare the effects of `b[1]=11` and `c[2]=22` on all four arrays.

### 2.2.9 Reshaping an array

Flattening an array is just one way to change its shape. An array can be recast into any shape consistent with its number of elements by using **`np.reshape`** or an array's own **`reshape`** method. Try the following, and inspect each array:

```
a = np.arange(12)
b = np.reshape(a, (3, 4) )
c = b.reshape( (2, 6) )
d = c.reshape( (2, 3, 2) )
```

The **`reshape`** method takes a tuple of numbers, like `(3,4)` or `(2,3,2)`, as its argument. As long as the product of these numbers is equal to the number of elements in the original array, the command will return a new array. Like **`ravel`**, these **`reshape`** methods return an `array` object with access to the same data as `a`, but with a different shape. Modifying the elements of any of the arrays in this example will affect all four of them.

### 2.2.10 T2 Lists and arrays as indices

Python has an even more flexible method for slicing an array: You can use a list where an index is expected. Using a list of integers will return an array that contains just the elements of the original array at the offsets specified by the list. Try

```
a = np.arange(10, 21)
b = [2, 4, 5]
a[b]
```

This method can be extremely useful if the list `b` was itself generated automatically.

You can also use a **Boolean** array (an array whose entries are either `True` or `False`) to select entries from another array of the same shape. This technique is called **logical indexing**. Try the following example:

```
a = np.arange(-10, 11)
less_than_five = (abs(a) < 5)
b = a[less_than_five]
```

The comparison in the second line returns an array with the same shape as `a` whose entries are `True` or `False`, depending on whether the particular element in `a` satisfies the comparison. [9] When `less_than_five` is used as an index to `a`, Python returns an array containing only those elements of `a` for which the corresponding element in `less_than_five` is `True`.

It is not necessary to create a named array to use as an index. The following line is equivalent to the last two lines of the example above:

```
b = a[abs(a) < 5]
```

## 2.3 STRINGS

Python can manipulate text as well as numbers. Next to an array, the second most important data structure for our purposes is the **string**. A string may contain any number of characters. You can create one with

```
s = 'Hello, world!'
```

Notice how `s` looks in the Variable Explorer.

The expression on the right side of the equals sign above is a **string literal**. The equals sign assigns this short sentence (without the quotes) as the value of `s`. Python allows you to use either a single quote (`'`) or a double quote (`"`) to define a string; however, you must begin and end the string with the same character.

*A string starts and ends with a single quote or a double quote.*

A single quote is the same as the apostrophe on your keyboard. Elsewhere on your keyboard there's a different key, the grave accent. You may be tempted to use it, as in `` `string' ``. Python won't understand that.

If you need an apostrophe inside a string, you can enclose the whole string in double quotes: `"Let's go!"` If you need both an apostrophe *and* double quotes, you will have to use a backslash (`\`) before the symbol that encloses the string. Write either `"I said, \"Let's go!\""` or `'I said, "Let\'s go!"'` More generally, inside a string, the backslash is an **escape character** that means, "Interpret the next character literally." In these examples, it instructs Python that the double quotes and the apostrophe, respectively, are part of a string, not the beginning or end of a string.[10]

[9] This is an example of *vectorized* computation, which will be discussed in Section 2.5.1.
[10] Another use of backslash is to code special characters, such as newline (`\n`) and tab (`\t`). See Section 3.2.2.

A string may contain a collection of digits that looks like a number, for example, `s='123'`. However, Python *still considers such a value to be a string*, not the number 123. Try typing

```
a = '123'
b = a + 1
```

Python issues a **TypeError** when you try to add a number to a string. However, you can convert a string to a number. (This may be necessary if you are getting input from a keyboard.) Python can do sensible conversions like creating an integer from `"123"` or a floating point number from `"3.14159"`, but it does not know how to create an integer from a string that contains a decimal point. Try the following:

```
s = '123'
pie = '3.141592653589'
x = int(s) + 1
y = float(pie) + 1
z_bad = int(s) + int(pie)
z_good = int(s) + int(float(pie))
```

Python does know how to "add" two strings together. Try this:

```
"2" + "2"
```

The result is not what you learned in kindergarten. Python uses the plus sign (+) to join (*concatenate*) two strings:

```
s = 'Hello, world!'
t = 'I am Python.'
s + t
```

**Your Turn 2E**

> The result of the last evaluation doesn't look quite right. Replace the last line with `s+' '+t` and explain the operations that lead to the output.

Some Python functions require arguments that are strings. For example, graph titles and legends must be specified as strings. (See Section 3.3.) You may wish to include the value of a variable in such a string. If the value is not already a string, one option is to convert it. Just as Python can convert strings to integers and floating point numbers, it can convert numbers to strings. The built-in function **str** will try to create a string using whatever input you provide:

```
s = "Poisson distribution for $\\mu$ = " + str(mu_val)
```

This string can now be used by PyPlot to generate a graph title. In this expression,

- The + sign joins two strings.
- The first string is a literal. It contains some special characters that PyPlot can use to produce the Greek letter $\mu$.[11]
- The second string is obtained from the current value of a variable called `mu_val`.

[11] T2 Python will interpret the text between dollar signs (`$...$`) as LaTeX typesetting instructions. The first backslash escapes the second one and prevents it from being interpreted as an escape character.

Python will display up to 16 digits when printing floating point numbers, and this is what the **str** command will generate in the example above if $\mu$ has that many digits. You can control how numbers appear in strings by using special commands involving the percent sign (`%`) or the **format** method that every string possesses. The percent sign works in all versions of Python and is widely used, so we will briefly describe it below. However, the **format** method behaves more like other functions in Python, so we will discuss it first and use it throughout the remainder of the tutorial.

### 2.3.1 Formatting strings with the `format()` method

Let's look at some examples that use the **format**`()` method:[12]

```
# string_format.py     [get code]
"The value of pi is approximately " + str(np.pi)
"The value of {} is approximately {:.5f}".format('pi', np.pi)
s = "{1:d} plus {0:d} is {2:d}"
s.format(2, 4, 2 + 4)
"Every {2} has its {3}.".format('dog', 'day', 'rose', 'thorn')
"The third element of the list is {0[2]:g}.".format(np.arange(10))
```

When evaluating a string's **format** method, Python interprets a pair of curly brackets (`{}`) as a placeholder, where values will be inserted. The argument of **format** is a series of expressions whose values will be inserted at the designated points. They can be strings, numbers, lists, or more complicated expressions. These examples illustrate several properties of the **format** method:

- An empty set of curly brackets will insert the next item in the list, as illustrated in the third line. The item can be formatted if the brackets contain a colon (`:`) followed by a formatting command. For example, "`{:.5f}`" means, "Format the current item in the argument list as a floating point number with 5 digits after the decimal point."
- Items can also be explicitly referenced by their location in the list passed to **format**. For example, the fourth line in the examples above defines `s` as a string with three placeholders to be filled later. It will take the second, then the first, then the third arguments and insert them at the designated locations. The syntax "`{1:d}`" means, "Insert the second element of the list here, and display it as a decimal (base 10) integer." (Python can also represent integers as binary, octal, or hexadecimal by using `{:b}`, `{:o}`, and `{:x}`, respectively.)
- Line 6 shows that not all items in the list need to be used.
- The last line demonstrates how you can reference an individual item of an array passed to **format**: The *replacement field* `0[2]` refers to the third element of the first argument, and the *format specifier* `:g` then instructs Python to display a number using the fewest characters possible (general format). It may choose exponential notation, and it will leave off trailing zeros in floating point numbers.

Python's **help**`(`**str**`)` information provides basic descriptions of the available string methods. There are also many references online if you wish to explore string processing beyond the rudimentary introduction provided here. However, it can be quicker and more fun to

*Experiment in the console to discover how things work.*

[12] The hash symbol # introduces a comment. (See Section 2.6.4.) The comment here tells you the name of this code snippet, so that you can find it in the book's online resources.

### 2.3.2 T2 Formatting strings with %

Try the following at the command prompt to see how the % syntax works:

```
# string_percent.py     [get code]
"The value of pi is approximately " + str(np.pi)
"The value of %s is approximately %.5f" % ('pi', np.pi)
s = "%d plus %d is %d"
s % (2, 4, 2 + 4)
```

When a string literal or variable is followed by the % operator, Python expects that you are going to provide values to be formatted and inserted into that string. The desired insertion point(s) are indicated by % characters within the string. These can be followed by information about how each value should be formatted, as described in Section 2.3.1. As these examples show, you can insert multiple values into a string, and each value can be the result of evaluating an expression. You must provide the values in the order they are to appear in the string.

Thus, in the third line of the example above, "%s" means, "Insert a string here." Similarly, "%.5f" means, "Insert a floating point number here with 5 digits after the decimal point." In the third line of the example, "%d" means, "Insert a decimal (base 10) integer here."

In addition to formatting strings, the percent sign can also function as an arithmetic operator in Python. In an expression like 5%2, the percent sign represents the **modulo operation**. It returns the remainder upon division. (Try it.) Although this particular operation is not used frequently in physical modeling, it can cause puzzling behavior when you have a bug that involves a percent sign.

## 2.4 LOOPS

So far, we have described Python as a glorified calculator with some string processing capabilities. However, we can continue to build on what we have learned and do increasingly complex tasks. In this section, we will explore two **control structures** that will allow us to repeat a set of operations as many times as we need: **for** loops and **while** loops.

### 2.4.1 for loops

Instead of solving a quadratic equation, let's go a step further and create a *table* of solutions for various values of a, holding b and c fixed. To do this, we will use a **loop**. Try typing the following in the IPython console:

```
# for_loop.py     [get code]
b, c = 2, -1
for a in np.arange(-1, 2, 0.3):
    x = (-b + np.sqrt(b**2 - 4*a*c)) / (2*a)
    print("a= {:.4f}, x= {:.4f}".format(a, x))
```

(Hit <Return/Enter> on a blank line to terminate the loop and run it.)

The keyword **for** instructs Python to perform a block of code repeatedly. It works as follows:

1. The function **np.arange**(-1,2,0.3) indicates a series of values with which to execute the indented block of code. Python starts with the first entry and cycles over each value in the array.

**2.** Python initially assigns `a` the value −1. Then, it evaluates a solution to the quadratic equation and prints the result. (Without the **`print`** statement, nothing would be displayed as the calculation proceeds.)

**3.** The end of the indented code block tells Python to move back to the nearest incomplete **`for`** command, update the value of `a`, and execute the block *again*. Eventually, Python reaches the end of the array. Python then jumps to the next block of unindented code (in this case, nothing). We say that it "exits the loop."

Note that `a` is an ordinary variable whose value can be accessed in calculations. It is generally not good practice to modify its value within the loop, however. When it reaches the end of an indented block, Python will continue on to the next value in the array and discard any changes you made to the value of `a`.

The example above illustrates a very important feature of Python:

*Blocks of code are defined* only *by their indentation.*

In many programming languages, special characters or commands separate blocks of code. In Java, C, or C++, the instructions in a **`for`** loop are enclosed in curly brackets (`{...}`). In other languages, the end of a block of code is designated by a keyword (for example, `end` in MATLAB). In Python, the indentation of a statement—and nothing else—determines whether it will be executed inside or outside a loop. This forces you to organize the text of your programs exactly as you intend the computer to execute it. You can tell at a glance which commands are inside a loop and where a block of code ends, and your code is not cluttered with dangling curly brackets or a pyramid of `end` statements at the end of a complex loop.

How much should you indent? The only requirements in Python are the following:

(*i*) Indentation consists of blank spaces or tabs.

(*ii*) Indentation must be consistent within a block. (For example, you cannot use four spaces on one line and a single tab on the next, even if the tab appears to indent the block by four spaces.)

(*iii*) The indentation level must increase when starting a new block, and go back to the previous level when that block ends.

Try this modified set of instructions and explain why the output differs from before:

```
b, c = 2, -1
for a in np.arange(-1, 2, 0.3):
    x = (-b + np.sqrt(b**2 - 4*a*c)) / (2*a)
print("a= {:.4f}, x= {:.4f}".format(a, x))
```

We can tell by looking that this version of the loop will cycle over the values of `a`, and *then* display the final values of `a` and `x` once Python exits the loop.

Note the colon after the **`for`** statement. This is essential. It tells Python that the next lines of indented code are to be associated with the **`for`** loop. The colon is also used in **`while`** loops and **`if`**, **`elif`**, and **`else`** statements, which will be discussed shortly.

IPython's command line interpreter aids you in setting up indentation. When you typed the first line of the **`for`** loop and hit `<Return/Enter>`, the next line was automatically indented. When you write your own programs, you will have to make sure statements are indented properly. (Many semi-intelligent editing programs, including the Editor in Spyder, will help you.)

If the body of a loop is very brief, you can just write it on the same line as the **`for`** statement after the colon, and forget about indentation:

```
for i in range(1, 21): print(i, i**3)
```

### 2.4.2 `while` loops

A second type of control structure, called a "**while** loop," is useful when you wish to repeat a block of code as long as some general condition holds but you do not know how many iterations will be required. A **while** loop will exit the first time its condition is **False**. For example, you could calculate solutions to a quadratic equation until the discriminant changes sign as follows:

```
# while_loop.py     [get code]
a, b, c = 2, 2, -1
while (b**2 - 4*a*c >= 0):
    x = (-b + np.sqrt(b**2 - 4*a*c)) / (2*a)
    print("a = {:.4f}, x = {:.4f}".format(a, x))
    a = a - 0.3
print("done!")
```

Note that we now need to change the value of `a` explicitly within the loop (line 6), because we are not iterating over an array. Also, be aware that a different choice of values for `a`, `b`, and `c` could result in an infinite loop. (See Section 2.4.4 below.)

As with a **for** loop, the indentation of statements tells Python which commands to execute inside a **while** loop; it also tells Python at what point to pick up again after the loop has completed. In the example above, the code prints out a short message after finishing the loop.

### 2.4.3 Very long loops

Some calculations take a long time to complete. While you're waiting, you may be wondering whether your code is really working. It's a good idea to make your code provide updates on its progress. If you have a loop like `for ii in range(10**6):`, then inside the loop you may wish to place

```
if ii % 10**5 == 0: print("{:d} percent complete ...".format( (100*ii)/10**6))
```

Here the percent sign is being used to determine a remainder.

You may want to do some other task while waiting for your code to complete. In that case, it's convenient to get audible bulletins. The simplest is `print("\a")`. Try it!

### 2.4.4 Infinite loops

When working with loops, there is the danger of entering an **infinite loop** that will never terminate. This is usually a bug, and it is useful to know how to halt a program with an infinite loop without quitting Spyder itself.

The easiest way to halt a Python program is to issue a **KeyboardInterrupt** by typing `<Ctrl-C>`. This will work on most programs and commands. Try it now. At the IPython command prompt, type

```
while True: print("Here we go again ...")
```

When you hit `<Return/Enter>`, Python will enter an infinite loop. Halt this loop by typing `<Ctrl-C>`.

`<Ctrl-C>` can also be used to halt commands or scripts that are taking too long, or lengthy calculations that you realize are using the wrong input. Just click on the IPython console tab, and type `<Ctrl-C>`. There is also a red STOP button in the upper-right corner of the IPython console that will issue a **KeyboardInterrupt** from within Spyder (see Figure 1.1, page 6). Run the infinite loop command again, and halt it with the STOP button.

Jump to Contents Jump to Index

It may take a while for Python to respond, depending on what it is doing, but <Ctrl-C> or the STOP button will usually halt a program or command. However, if Python is completely unresponsive, you may have to exit Spyder or force it to quit with the RESTART KERNEL button on the IPython console pane (again see Figure 1.1, page 6). If this happens, you could lose everything you have not saved, so be sure to save your work frequently.

## 2.5 ARRAY OPERATIONS

One reason to use arrays is that NumPy has a very concise syntax for handling repetitive operations on arrays. NumPy can calculate the square root of every element in an array or compute the sum of each column in a matrix much more rapidly than a **for** loop designed to do the same thing. Applying an operation to an entire array instead a single number (scalar) is called **vectorization** of the operation. An operation that combines elements of an array to create a smaller array—like summing the columns of a matrix—is called **array reduction**. Let's look at examples of both of these operations.

### 2.5.1 Vectorizing math

We can calculate the same solutions to the quadratic equation in Section 2.4.1 by using the following code:

```
# vectorize.py     [get code]
b, c = 2, -1
a = np.arange(-1, 2, 0.3)
(-b + np.sqrt(b**2 - 4*a*c)) / (2*a)
```

We type the last command exactly as we would for a single value of `a`, but Python evaluates the operation for every element of `a` and returns the results in an array.

To evaluate the expression on the fourth line, Python essentially carries out the following steps:[13]

1. Python starts with the innermost subexpression, first calculating `b**2` and saving the result. In evaluating `4*a*c`, it notices that the array `a` is to be multiplied by 4. Python interprets this as a request to multiply *each element* by 4. This is the standard interpretation in math: When you multiply a vector by a scalar, each component is multiplied by that quantity. Similarly, Python then multiplies the resulting array by `c`.
2. After evaluating `4*a*c`, Python sees that you've asked it to combine this result with the single quantity `b**2`. *Unlike* standard math usage (where it does not make sense to add a vector and a scalar together), Python interprets this as a request to create a new array, in which the entry indexed by $k$ is equal to `b**2 - 4*a[k]*c`. That is, addition and subtraction of an array and a scalar are also evaluated item by item.
3. Python then passes the array generated in step **2** to the **np.sqrt** function as an argument. NumPy's square root function will accept a single number, but like many functions in the NumPy module (including **sin**, **cos**, and **exp**), it can also act on an array. The function acts item by item on each entry and returns a new array containing the results.[14]

[13] Python may actually use a more efficient procedure.

[14] The module we are using is important: The `sqrt` function in the **math** module will not accept an array as an argument.

**4.** Python adds `-b` to the array from **3** using the same rule as in **2**.

**5.** Python divides the array from **4** by `2*a`. This operation involves two arrays and will be discussed in a moment. It is also evaluated item by item.

The code in `vectorize.py` is a *vectorized* form of the earlier **`for`** loop. Vectorized code often runs much faster than equivalent code written with explicit **`for`** loops, because clever programmers have optimized NumPy functions to run "behind the scenes" with compiled programs written in C and FORTRAN, avoiding the usual overhead of Python's interpreter.

Another advantage of vectorized code is that it can help you to write better programs because it is so concise. Long, rambling code can be hard to read, making it difficult to spot bugs. Of course, extremely dense code is also hard to read. Your coding style will evolve as you get more experienced. Certainly if a code runs fast enough with **`for`** loops, then there's no need to go back and vectorize it.

Not every mathematical operation on an array will act item by item, but most common operations do. Suppose that you wanted to graph the function $y = x^2$. You could set up an array of $x$ values by `x=np.arange(21)`. Then, `y=x**2` or `y=x*x` gives you what you want. This is *not* true of Python's native `list` objects. It is one reason why NumPy arrays are more useful for numerical work. Execute the following commands to see why we do not use Python's `list` data structure:

```
x = [1, 2, 3, 4, 5, 6]
2 * x
x + 2
x * x
x - x
```

Python's `list` objects are useful in many computing applications, but fast mathematical calculation on large collections of numbers is not one of them.

We can now finish explaining the example at the start of this section. The variable `a` is an array; therefore, `-b + np.sqrt(b**2 - 4*a*c)` and `2*a` are arrays, too. When two NumPy arrays of the same shape and size are joined by `+`, `-`, `*`, `/`, or `**`, Python performs the operation on each pair of corresponding elements, and returns the results in a new array of the same length.

**Your Turn 2F**

a. We often wish to evaluate the function $y = e^{-(x^2)}$ over a range of $x$ values. Figure out how to code this in vectorized notation.
b. We often wish to evaluate the function $e^{-\mu}\mu^n/(n!)$ over the integer values $n = 0,\ 1, \ldots, N$. Here, the exclamation point denotes the factorial function. Figure out how to code this in vectorized notation for $N = 10$ and $\mu = 2$. (You may want to import the **`factorial`** function from SciPy's collection of special functions, **`scipy.special`**.)

The item-by-item operations work equally well with multidimensional arrays. Again, the arrays to be combined must have the same shape. The expressions `a+b` and `a*b` generate error messages unless `a.shape==b.shape` is **`True`** or `a` and `b` can be "broadcast" to a common shape. (For example, you can always add a number to an array of any shape. For more information, try a Web search for `numpy broadcasting`.)

*Most math operations on NumPy arrays are performed item by item.*

Jump to Contents Jump to Index

Sometimes, instead of item-by-item operations, you may want to combine arrays by the rules of matrix math. For example, the "dot product" of two vectors (or, more generally, matrix multiplication) requires a special function call. Compare the output of the two operations below:

```
a = np.array( [1, 2, 3] )
b = np.array( [1, 0.1, 0.01] )
a*b
np.dot(a, b)
```

The number of elements in `a` equals the number of elements in `b`, so the dot product is defined. In this case it is the single number $\boldsymbol{a} \cdot \boldsymbol{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$.

### 2.5.2 Reducing an array

In contrast to ordinary functions like **`sin`**, which act item by item when given an array, some functions combine elements of an array to produce a result that has fewer elements than the original array. Sometimes the result is a single number. Such operations are said to **reduce** an array.

A common array reduction is finding the sum of the elements in each row or column, or the sum of all the elements in the array. Try

```
a = np.vstack( (np.arange(20), np.arange(100, 120)) )
b = np.sum(a, 0)
c = np.sum(a, 1)
d = np.sum(a)
```

The function **`np.sum`**`(a,n)` takes an array `a` and an integer `n` and creates a new array. Each entry in the new array is the sum of the entries in `a` over all allowed values of index number $n-1$, holding the other indices fixed. Thus, in the example above, `b` contains the sums over each column of `a`, and `c` contains sums over each row. When no index is given, as for `d`, **`np.sum`** adds up *all* of the elements in the array.

Python also has a built-in **`sum`** function that does not work the same way. Try the example above using **`sum`** without the "**`np.`**" prefix to see what happens, then look at **`help`**`(`**`sum`**`)` and **`help`**`(`**`np.sum`**`)` to understand the difference between the two functions.

You can use experimentation and the **`help`** command to explore the useful related functions **`np.prod`**, **`np.mean`**, **`np.std`**, **`np.min`**, and **`np.max`**. Alternatively, every array comes with methods that evaluate these functions on the array's own data. For instance, the example above could also have been written as follows:

```
a = np.vstack( (np.arange(20), np.arange(100, 120)) )
b = a.sum(0)
c = a.sum(1)
d = a.sum()
```

See if you can explain how the following code calculates 10!:

```
ten_factorial = np.arange(1, 11).prod()
```

## 2.6 SCRIPTS

You can do many useful things from the IPython Console, which is how we have been using it so far. However, retyping the same lines of code at the command prompt over and over is tedious, and many tasks involve code that is far more complex than the examples above. Your code will go through many versions as you get it right, and it's tiresome to keep retyping things and making new errors. You'll want to work on it, take a break, return to it, close Spyder, move to a different computer, and so on. And you will wish to share code with other people in a "pure" form, free from all the typos, missteps, and output made along the way. For all these reasons, you will want to do most of your coding in the form of scripts.

A **script** is simply a text file that contains a series of commands. You edit a script in a text editor, then execute it by calling it with a single command or a mouse click.

### 2.6.1 The Editor

To create a script within Spyder, you can open a new document from the File menu, you can click on the blank page icon at the upper left, or you can use the keyboard shortcut `<Cmd-N>`.[15]

Write a script that contains any of the code fragments above. If the Editor isn't open, you can type **`%edit`** at the command prompt or open a new file. As you enter text in a script, hitting `<Return/Enter>` only moves the cursor to the next line. No commands are executed because Python has no knowledge of your script yet. It is just a text file somewhere in your computer's memory.

When you finish editing, you can execute your file by clicking the RUN button (the green "Play" button), selecting "Run" from the `Run` menu at the top of the window, or by using the shortcut key `<F5>`. Python executes the code in the Editor's active tab and displays any output in the IPython console. Before it does that, however, it saves the code, prompting you for a file name if necessary. It is customary to end Python code file names with the extension `.py`. You can also execute the code by typing **`%run`** followed by your file name (with or without the `.py` extension) at the command prompt, but you will need to import modules and define variables within the script instead of relying on what you may have already done in the IPython console.

> *It's good practice to type* **`%reset`** *at the command prompt before running any script.*

That way, you know exactly what state Python is in when the script begins. Keep in mind that **`%reset`** will delete all imported modules. Thus, you will probably want to include the following lines at the top of *every* script you write:

```
import numpy as np
import matplotlib.pyplot as plt
```

You can have Spyder add these lines to all of your new scripts by default. See Section A.2 (page 110).

### 2.6.2 T2 Other editors

If you prefer another text editor, you can edit and save files outside of Spyder and then run them within Spyder. Give your code file the extension `.py` (not `.txt`).

[15] On Windows systems, use `<Ctrl-N>`. You can also use IPython to run scripts that you create outside of Spyder. See Section 2.6.2.

You can use **%run** to execute any plain text Python file in the current directory. Formatted text files such as `.rtf`, `.doc`, or `.docx` files won't work, however. It's best to use a plain text editor that has three key features:[16]

- Your editor should have an *autoindent* feature that will ensure consistent spacing in indented blocks of code. Many text editors also allow you to convert tabs into an equivalent number of spaces. If you do not use either feature, you could accidentally mix tabs with spaces in indented blocks. The code may look correctly indented on the screen, but it won't run properly.
- Your editor should also have *syntax highlighting.* This useful feature displays Python commands and keywords in different colors or fonts, allowing you to spot some coding errors at a glance. If your editor supports syntax highlighting, it will operate automatically whenever you open a file with extension `.py`.
- Your editor should support *bracket matching,* which identifies unmatched brackets such as a "`(`" without a "`)`", or an extra "`)`".

IPython can also accommodate you in editing files outside of Spyder. You can open a script for editing by typing **%edit** `my_script.py`. This will open `my_script.py` in your default text editor. Once you have created or edited the file, save it and quit the editor. IPython may automatically execute your file. You can also execute the script by typing **%run** `my_script.py`.

Finally, you can execute a Python script directly from a command line or Windows command prompt, outside of any IDE, with the command

```
>> python my_script.py
```

### 2.6.3 First steps to debugging

If you make an error in your code while you are typing it in the Editor, Spyder may catch it before you even click RUN. Lines with potential errors are flagged by yellow triangles and red diamonds with an exclamation point inside. A red diamond means your script will not run; a yellow triangle means it may not run.[17] Type the following lines in the Editor to see the two types of errors:

```
s = str 3
prnt(s)
```

(You may need to save the file before Spyder checks it for errors.) Spyder recognizes that you are using the known function **str** incorrectly in the first line (a *syntax error*) and flags it with a red diamond. Move the mouse cursor over the red diamond to get a brief explanation. The information provided is sometimes cryptic, but just knowing the general location of an error is quite helpful.

Fix the first line so that `s` is assigned the string representation of `3`. Now, a yellow triangle will appear next to the second line. (If there is a fatal error, Spyder will not bother warning you about other potential trouble. If there is more than one fatal error, Spyder will usually only recognize the first.) In this case, Spyder is not aware of any function called `prnt`, but you may have defined it somewhere else. If you click RUN, Spyder will attempt to execute the code. When Python encounters the `prnt` command, it discovers that no such name is defined, and so it *raises an exception* called

[16] Classic examples are the command line text editors Vim and Emacs, included with most Unix systems. Other options include Notepad++ for Windows (`notepad-plus.org`), TextWrangler for Mac (`www.barebones.com`), and gedit for Linux (`wiki.gnome.org/Apps`), all of which are free.

[17] T2 Spyder's syntax checking is performed by the `pyflakes` module, which can be run independently of Spyder. Another popular syntax checker is `pylint`.

**NameError**. That is, it prints a *runtime error message* in the IPython console and aborts any further processing.

> *A lot of information may print out when Python encounters an error. Generally, the last few lines are the most helpful.*

The last portion of the error message will cite the line number where Python first noticed something wrong, display a portion of the script near this line, and describe the type of error encountered. The script may contain other errors, but Python quits when it encounters the first one. Fix it and try again.

> *The most common cause of* **NameError** *is misspelling.*

Variable names are case sensitive, so inconsistent case is a form of spelling mistake. For example, if you define `myVelocity=1` and later attempt to use `myvelocity`, Python treats the second instance as a totally new, undefined variable. Scanning the Variable Explorer can help you find such errors.

> *Another common cause of* **NameError** *is forgetting to prefix a function name with the name or nickname of the module containing it.*

Runtime errors generate messages in the IPython console when you run the code. Most cause Python to halt immediately. A few, however, are "nonfatal." For example, try

```
for x in np.arange(-1, 8):
    print(x, np.log(x))
```

Python finds nothing syntactically incorrect here, but NumPy recognizes a problem when it attempts to calculate the first two values. It gives these values names in this case (**nan** for "not a number" and **-inf** for "infinity"), but it knows these values could cause problems later, so it issues two **RuntimeWarning** messages, and even explains what happened.[18]

Still other situations generate no message at all—you just get puzzling results. You will encounter many situations of this sort, and then you'll need to *debug* your code.

Whole books are written about the art of debugging. When a code generates messages or output that you don't understand, you need to look for clues. Here are some ideas:

- Read your code carefully. This is often the quickest route to insight.
- Build your code slowly. Most scripts are designed to execute a complicated task by breaking it into a series of simple tasks. Make sure each step does exactly what it is supposed to. If every step does exactly what it is supposed to do and your code still does not work, you have a *theory bug,* not a *programming bug.*
- Start with an easier case. You're using a computer because you can't do the problem by hand, but maybe there's another case that you *can* do by hand. Adapt your code for that case (perhaps just a matter of changing the parameters), and compare the output to the answer you know to be correct.
- Probe your variables. After a code finishes (or terminates with an error), all of its variables retain their most recent values; check them to see if anything seems amiss.
- Insert diagnostics. Somewhere prior to where you suspect there's an error, you can add a few lines that cause Python to print out the values of some variables *at that moment* in the code's execution. When you run the program again, you can check whether they're what you expect.

[18] Not all modules are as forgiving. Try importing the **math** module and using its `log` function inside the loop instead.

- Be proactive as you write. Make sure that what you think *should be* true actually *is* true when a command is executed. Python offers a very useful tool for this: the **assert** statement. At any point in your code, you can insert a line of the form

```
assert (condition), "Message string"
```

  If the condition is **True**, then Python will continue executing your code. If it is **False**, however, Python will stop the code, give an **AssertionError**, then print your message. For example, in the code above, we could add the line above the **print** statement:

```
assert (x > 0), "I do not know how to take the log of {}!".format(x)
```

  When the code runs, it will not only stop when it tries to take the logarithm of a bad value—it will also print out the value that caused the problem. Of course, you can't foresee every possible exceptional case. But you will develop a sense of what bugs are likely after some practice.
- Explain the code line-by-line out loud to another person or inanimate object. The latter practice is often called *Rubber Duck Debugging.* Professional developers commonly force themselves to explain malfunctioning code to a rubber duck or similar totem to avoid having to involve another developer. The act of describing what the code is *supposed* to do while examining what it actually does quickly reveals discrepancies.
- Ask a more experienced friend. This may be embarrassing, because almost all coding errors appear "stupid" once you've found them. But it's not as embarrassing as asking your instructor. And either one is better than endlessly banging your head on the temple wall. You need to become self-reliant, eventually, but it doesn't happen all at once.
- Ask an online forum. An amazing, unexpected development in human civilization was the emergence of sites like `stackoverflow.com` where people pose queries at every level, and total strangers freely help them. The turnaround time can be rather slow; however, your question may already be asked, answered, archived, and available.
- Learn about *breakpoints.* This topic goes beyond the scope of this tutorial, but there may come a day when your code is complex enough to need it.

Maybe the single most important point to appreciate about debugging is that *it always takes longer than you expected.* The inevitable corollary is

*Don't wait till the day before an assignment is due to start.*

Some puzzles don't resolve until your subconscious has had time to unravel them. If you need help from a friend, lab partner, or instructor, that takes time too. Respect the subtlety of coding, and give yourself enough time.

### 2.6.4 Good practice: Commenting

Another key advantage of writing scripts is that you are free to include as many remarks to your reader—and yourself—as you like.

Let's apply our example of calculating solutions to the quadratic equation by giving it some physical meaning. Suppose you are planning a prank on a friend and need to know how long a snowball remains in the air when thrown upward with a particular initial speed. You recall from introductory physics that the height of a ball thrown upward is

$$y(t) = y_0 + v_0\, t - \frac{1}{2}\, g\, t^2.$$

After you recall the meaning of all the symbols, you realize that you need the value of $t$ for which $y(t) = 0$. Rearranging things a bit, you obtain

$$\frac{1}{2}gt^2 - v_0 t - y_0 = 0.$$

This is the quadratic equation we already solved, but now the parameters all have physical meanings.

You can use the following code to plan your prank:

```
# projectile.py     [get code]
# Jesse M. Kinder -- 2014 Nov 11
"""
Calculate how long an object is in the air when thrown from a specified height
with a range of initial speeds assuming constant acceleration due to gravity:
    0.5 * g * t**2 - v0 * t - y0 = 0
"""
import numpy as np
import matplotlib.pyplot as plt

#%% Initialize variables.
initial_speed = 0.0          # v0 = initial vertical speed of ball [m/s]
impact_time = 0.0            # t = time of impact [s] (computed in loop)

#%% Initialize parameters.
g = 9.8066                   # Gravitational acceleration [m/s^2]
initial_height = 2.0         # y0 = height ball is thrown from [m]
speed_increment = 5.0        # Speed increment for each iteration [m/s]
cutoff_time = 10.0           # Stop computing after impact time exceeds cutoff.

#%% Calculate and display impact time.  Increment initial speed each step.
#   Repeat until impact time exceeds cutoff.
while impact_time < cutoff_time:
    # Use quadratic equation to solve kinematic equation for impact time:
    impact_time = (np.sqrt(initial_speed**2 + 2 * g * initial_height) \
            + initial_speed) / g
    print("speed= {} m/s; time= {:.1f} s".format(initial_speed, impact_time))
    initial_speed += speed_increment
print("Calculation complete.")
```

(The syntax `x+=1` is a concise and descriptive shorthand for `x=x+1`. Other arithmetic operations can be called in a similar way. For example, `x*=2` will double the value of `x`.)

Compare this code to what was written in Section 2.4.1. The code here accomplishes a similar task, but the purpose and function of the script are now easy to understand because of the comments, whitespace, and meaningful variable names.

- The opening lines now tell us who wrote the program, when, and why.
- The first two commands import NumPy and PyPlot (which will be described later).
- Lines 11–19 assign values and meaningful names to the parameters and variables. (By "parameter" we simply mean a variable whose value is not going to change throughout the execution of the code.) Comments describe the role of each.

- The first parts of lines 12–13 and lines 16–19 are code, but everything after the hash signs (`#`) is commentary explaining the meaning of the assignments.
- Lines 21–22 introduce the main loop of the program and explain the condition for exiting the loop.

The script is certainly longer than the bare code introduced in Section 2.4.1, but which is easier to read? Which would *you* rather read, if you wanted to reuse this code after a month or two of not working on it?

> *Every coder eventually learns that good comments save time in the long run... as long as comments are updated to reflect changes in the code.*

You will probably ignore this advice until you return to a script you wrote several weeks earlier, cannot figure out what any of it means, and have to rewrite the whole thing from scratch. At least you were warned.

The script makes use of two different types of comments: One type is enclosed within triple quotes (`"""`), and the other is preceded by the hash sign (`#`). The hash sign is an *inline* comment character. Python will ignore everything that follows a hash sign on a single line. Comments can start at the beginning of a line or in the middle. Placing a short comment on the same line as the code being discussed is a good way to ensure that you'll remember to update the comment if you change the line later.

Some comments begin with the hash sign followed by two percent signs (`#%%`). These mean nothing special to Python, but they divide the code into logical units called **cells**. Besides looking good on the screen (try it), this structuring allows you to run individual cells separately within Spyder. To see how this works, click anywhere inside such a cell, then click the Run cell button (immediately to the right of the Run button) or use the shortcut `<Ctrl-Enter>`. Also try the alternative Run cell and advance (immediately to the right of the Run cell button), which has the shortcut `<Shift-Enter>`. Breaking code into cells by using comments is another useful debugging tool.

Triple quotes provide a convenient way to create comments or strings that span multiple lines. They can also be used to create special kinds of comments called documentation strings, or **docstrings** for short. A string enclosed in triple quotes becomes a docstring when it is placed at the beginning of a file or immediately after a function declaration.[19] Anywhere else in the file, a string enclosed in triple quotes is treated as an ordinary string.

Python uses docstrings to provide help about modules and functions you write. For example, the docstring in a function can tell a user what arguments are required, what the function will return, and so on. The script above is not a very useful module, but to see how the `help` feature works with the docstrings you write, you can import it anyway. Type `import projectile` (the file name without the `.py` extension) at the command prompt. Now type `help(projectile)`. You should see the text between the triple quotes.

Here is a final comment about comments: As you hack together a workable code, you may want to try out some temporary change, but preserve the option to revert your code to its preceding version. A simple method is simply to *keep* the old lines of code, but "comment them out" so that they become invisible to Python. If you want them back later, simply "uncomment" them. Spyder (like most other editors) offers a menu item to comment or uncomment a selected region of text, or you can use the shortcut `<Cmd-1>`.

The sample script above also illustrates proper use of indentation and other forms of *whitespace*.

[19] Section 5.1 discusses user-defined functions.

- Proper indentation is crucial to Python. It also makes the code easier to read and understand. There is no ambiguity about which lines of code belong to the `while` loop and which do not.
- Python ignores blank lines, but they make the code much easier to read and interpret by dividing it into logical units.

### 2.6.5 Good practice: Using named parameters

Why did we clutter the code above with a variable name like `initial_height` instead of `y0`? Why did we bother to name the fixed parameters at all? We could have "hard coded" everything, replacing `g`, `initial_height`, and `speed_increment` by the numerical values `9.8066`, `2.0`, and `5.0` everywhere. There are at least four reasons:

1. We have separated what the code does from its input. This type of abstraction allows you to easily adapt your code for other purposes. You can run the same calculation with different values for the fixed parameters, embed the main loop of this code *inside* a loop in another program that evaluates *many* values of `initial_height`, or use the main loop inside a *function* that accepts the parameter values as input. Keeping them symbolic helps.
2. Using meaningful names for quantities improves your code's readability and clarity, even if those quantities stay fixed. Would you be as likely to understand the purpose of the program if there were no comments and we had just used short, uninformative variable names like `x1`, `x2`, and `x3`? You can probably recall taking notes in a physics or math class and momentarily forgetting what quantity a particular symbol stands for. It is much faster to write $y_0$ on the chalkboard or in your notebook, but the meaning is not always clear. There is no ambiguity about what the variable `initial_height` represents.
3. Naming parameters also keeps their values distinct, even when they are numerically equal. What if there were two parameters with different physical meanings that had the same numerical value of `5.0`? If you give variables and parameters different, meaningful names, then you won't get confused by such coincidences.
4. Using named parameters to control the *size* of a calculation is extremely useful. For instance, you may intend to set up ten arrays with five million entries each, and perform a thousand calculations on the collection of arrays. Such a code could take a long time to execute. It would waste a lot of time to wait for it to execute every time you fix a small bug or add a feature. It's better to set up some parameters (perhaps called `num_elements` and `max_iterations`) near the beginning of the script that control how big the calculation is going to be. You can choose small values for the development phase, and switch to the larger desired values only when you near the final version. Moreover, you can be certain that every array in the calculation will have the same number of elements.

These points are part of a bigger theme of coding practice:

> *Don't duplicate. Define once, and reuse often.*

That is, if the same parameter appears twice, give it a name and set its value *once.* If you don't implement this principle, then when you wish to change a value, inevitably you will find and change all but one instance. The one you missed will cause problems, and it could be hard to find. Worse yet, you may accidentally change something else!

Later we will see how "don't duplicate" can be applied not just to parameter values but also to code itself when we discuss functions in Section 5.1.

### 2.6.6 Good practice: Units

Most physical quantities carry units, for example, 3 cm. Python doesn't know about units; all its values are pure numbers. If you are trying to code a problem involving a quantity $L$ with dimensions of length, you'll need to represent it by a variable `length` whose value equals $L$ *divided by some unit.* That's fine, as long as you are consistent everywhere about what unit to use.

You can make things easier on yourself if you include a block of comments at the head of your code declaring (to yourself and others) the variables you'll use, and the units chosen to make them pure numbers:

```
# Variables:
# ------------------------------------------------------
#   length          =   length of the microtubule [um]
#   velocity        =   velocity of motor [um/s]
#   rate_constant   =   rate constant [1/days]
...
```

(The notation `um` is an easy-to-type substitute for $\mu$m.) As you work on the code, you can refer back to this section of your comments to keep yourself consistent. Such fussy hygiene will save you a lot of confusion someday.

Later on, when you start to work with data files, you need to learn from whoever created the file what units are being used to express quantities. Ideally this will be spelled out in a text file (perhaps called `README.txt`) accompanying the data file, or in the opening lines of the data file. (You should document data files that you create in this way.)

## 2.7 CONTINGENT BEHAVIOR: BRANCHING

The **while** loop above modified its behavior on the fly when it decided whether to execute its block of code again depending on the result of an intermediate calculation. Every programming language, including Python, has a more general mechanism for specifying contingent behavior, called **branching**. Try entering this code in the Editor and running it:[20]

```
# branching.py     [get code]
""" This script illustrates branching. """

import numpy as np, matplotlib.pyplot as plt

for trial in range(5):
    userInput = input('Pick a number: ')
    number = float(userInput)
    if number < 0:
        print('Square root is not real')
    else:
        print('Square root of {} is {:.4f}'.format(number, np.sqrt(number)))
    userAgain = input('Another [y/n]? ')
    if userAgain != 'y':
```

[20] In Python 2, **input** attempts to evaluate its argument as a Python expression. To run this script, users of Python 2 should replace **input** with `raw_input` or redefine **input** as `raw_input`. See Appendix C.

```
15          break
#%% Check whether the loop terminated normally.
if trial == 4:
    print('Sorry, only 5 per customer.')
elif userAgain == 'n':
20      print('Bye!')
else:
    print('Sorry, I did not understand that.')
```

### 2.7.1 The `if` statement

The code above illustrates several new ideas:

- The **`input`** function displays a prompt in the IPython console, then waits for you to type something followed by `<Return/Enter>`. Its return value is a string containing that input.[21] In this case, we assign that string to a variable called `userInput`, then convert it to a `float` for use later in the program.
- Line 9 contains an **if statement**. The keyword **`if`** is followed by a logical (*Boolean*) expression and a colon. If Python evaluates the expression and finds that it's **`True`**, then it executes the next indented block of code. If the expression is **`False`**, Python skips the block of code.
- The conditional block of the first **`if`** statement is followed by the keyword **`else`** and a colon. If the condition in the **`if`** statement is **`True`**, then Python skips the indented block following the **`else`** statement on line 11. If the condition is **`False`**, then Python executes the indented block following the **`else`** statement.
- The **`input`** function appears again in line 13. This time, the string returned by the function is what we want, so no conversion is necessary.
- The notation `!=` in line 14 means "is not equal to." You can also use the keyword **`not`** to negate a logical statement, so we could have written **`if not`**`(userAgain=='y')`.
- Sometimes a loop should terminate prematurely (before the condition in its **`for`** or **`while`** statement is satisfied). In line 15, the keyword **`break`** instructs Python to exit the indented block of the **`for`** loop and proceed at line 16, regardless of whether the loop is done.
- Line 17 starts a three-way branch whose behavior depends on how the loop terminated. This is accomplished using the **`elif`** statement, short for "else if." There can be as many **`elif`** statements as you like. Python will drop down from the opening **`if`** statement through the **`elif`** statements until it finds one that evaluates to **`True`** or it reaches an **`else`** statement. In any circumstance, only one of lines 18, 20, or 22 will be executed.

In short,

> *A branch construction starts with an* **`if`** *statement. This can be followed by a single block of indented code or multiple blocks separated by* **`elif`** *or* **`else`** *statements.*

Include an **`else`** statement when you want *something* to happen, even when none of the **`if`** or **`elif`** statements are true.

[21] See footnote 20.

The code listed above used the relation operators <, ==, and != to generate **True** or **False** values. Other operators that return Boolean values include >, <=, and >=. You can generate more complex conditionals with the Boolean operators **and**, **or**, and **not**.

### 2.7.2 $T_2$ On truth

Python defines **True** rather broadly. Any nonzero numerical value evaluates to **True** upon conversion to type `bool`, as does any nonempty list, string, tuple, or array. The following expressions evaluate to **False**: **False**, **None**, `[]`, `()`, `{}`, `0`, `0.0`, `0j`. Almost everything else evaluates to **True**.

## 2.8 NESTING

In many cases, we may wish to embed one **for** loop inside another **for** loop, embed a **for** loop within a **while** loop, or create some other combination. For example, when dealing with probability, we sometimes wish to create an array `A` where the value of `A[i,k]` depends on the indices `i` and `k`. We can do this with a code like the following:

```
# nesting.py     [get code]
rows = 3
columns = 4
A = np.zeros( (rows, columns) )
for i in range(rows):
    for k in range(columns):
        A[i, k] = i**2 + k**3
```

Notice that

- Before the loops, the code creates an array with **np.zeros**. NumPy does not build arrays on the fly. It sets aside a block in the computer's memory to store the array, so it needs to know how big that block should be.[22] Thus, we create an empty array, then later fill it up with the values we want.
- Python's **range** function is used to iterate over values of `i` and `k`. It is similar to **np.arange**, but it creates an object that generates values as they are needed instead of an array.[23]

*Use* **np.arange** *when you need an array. Use* **range** *in* **for** *loops.*

- Line 7 sits inside *two* **for** loops, so it is executed $3 \times 4 = 12$ times. One loop inside another is called a **nested loop**. Note that **if** statements and **while** loops can also be nested.
- Line 7 must be executed inside the inner loop, so it is indented twice.
- Descriptive variable names and Python's forced use of indentation make comments nearly superfluous in this code fragment. A reader might only wonder why you are computing $A_{i+1,k+1} = i^2 + k^3$ and not some other function.

[22] You can later resize the array if you need to, but this usually indicates poor planning. If you need a data structure with a variable size, you might use a Python list instead.

[23] $T_2$ The **range** function in Python 3 is equivalent to the `xrange` function in Python 2. In Python 2, **range** creates an actual list of integers. It will work in **for** loops, but `xrange` is often preferable, especially for large loops. There is no function called `xrange` in Python 3.

# CHAPTER 3

# Data In, Results Out

*Computers in the future may... perhaps only weigh 1.5 tons.*
—*Popular Mechanics* magazine, 1949

Most data sets are too big to enter by hand, often because they were generated by automated instruments. You need to know how to bring such a data set into your Python computing session (*import* it). You will also want to save your own work to files (*export* it) so that you do not have to repeat complex calculations. Python offers simple and efficient tools for reading and writing files.

Also, most results are too complex to grasp if presented as tables of numbers. You need to know how to present results in a graphical form that humans (including you) can understand. The PyPlot module provides an extensive collection of resources for visualizing data sets.[1]

In this chapter, you will learn to

- Load data from a file;
- Save data to a file; and
- Create plots from a data set.

## 3.1 IMPORTING DATA

Much scientific work involves collections of experimental data, or *data sets*. In order to crunch a data set, Python must first import it. Many data sets are available in plain text files, including

- *Comma-separated value* (`.csv`) files: Each line of the file represents a row of an array, with entries in that row separated by commas.
- *Tab-separated value* (`.tsv`) files: Each line represents a row of an array, with entries in that row separated by tabs, spaces, or some combination of the two. Spacing does not have to be consistent. Python treats any amount of whitespace between entries as a single separator.

The extensions `.txt` and `.dat` are also used for data files with comma or whitespace separation. Python has built-in support for reading and writing such files, and NumPy includes additional tools for loading and saving array data to files.

The SciPy library provides a module called **`scipy.io`** that will allow you to read and write data in a variety of formats, including MATLAB files, IDL files, and `.wav` sound files. See `docs.scipy.org`.

[1] T2 All of the data processing described in this chapter—and much more—can be accomplished with the `pandas` module. Though it falls outside the scope of this tutorial, `pandas` provides an extensive suite of tools for working with large data sets. It combines the functionality of NumPy, PyPlot, spreadsheets, and databases into a single interface. The `pandas` library is part of the Anaconda distribution. See `pandas.pydata.org`.

### 3.1.1 Obtaining data

Before you can import a data set, you must obtain the file of interest and place it in a location where Python can find it. It is good practice to keep all of the files associated with a project—data, scripts, supporting information, and the reports you produce—in a single folder (also called a **directory**).

You can tell Spyder to use a particular folder by setting the **global working directory** in the Preferences menu. Tick the box next to "At startup the global working directory is the following directory." Then, choose a folder,[2] and click APPLY then OK. Close Spyder and relaunch. Now each time you launch Spyder, it will start in this folder. This is also where Spyder will attempt to load and save all data files by default. The IPython command `%run myfile.py` will also look in this folder.

It is possible to switch to a different working directory during a session in Spyder. This can even happen by accident if you click the RUN button for a script that is not located in the current working directory. We recommend keeping things simple by working in a single folder.

Once you have set up your working directory, you can place data files into it. To download the data sets described in this tutorial, go to

`press.princeton.edu/titles/10644.html`

Here, you will find a link to a zipped file that contains a collection of data sets.[3] Follow the instructions to save the zipped file to your computer. It will probably be saved into your `Downloads` folder unless you specify a different destination. Double-click on the file to unzip it. This will create a new folder called `PMLSdata` in the current folder. You will find all of the data sets described in this tutorial, plus several more. Move `PMLSdata` to a permanent location. If you place `PMLSdata` in your working directory, you will be able to access it easily from within Spyder.

You do not have to limit yourself to the data sets described here. Someday, you'll use data from another source. You may have an instrument in the lab that creates it, or you may get it from a public repository or a coworker. Scientific publications that contain quantitative data in the form of graphs are another source. A graph is a visual representation of a set of numbers, and can be converted back to numbers with a special purpose application. Applications that convert graphs to numbers include

- DataThief (`www.softpedia.com/get/Science-CAD/DataThief.shtml`),
- Plot Digitizer (`plotdigitizer.sourceforge.net`),
- GetData (`www.getdata-graph-digitizer.com`), and
- GraphClick (`www.arizona-software.ch/graphclick/download.html`).

### 3.1.2 Bringing data into Python

After you have obtained the data sets, find the entry called `01HIVseries`. This folder contains a file called `HIVseries.csv`. There is also an information file called `README.txt` that describes the data set. Copy these two files into your working directory.

#### Arrays

You are now ready to launch Python and load the data set. First, inspect the data using the Editor by typing `%edit HIVseries.csv`. (You can also open files in the Editor by using the `File>Open` menu

[2] We recommend a generic name, like `scratch` or `curr`. After you are done with a project, you can archive the files in a different folder, and start the next one in the same folder.

[3] Resources mentioned here are also maintained at a mirror site: `www.physics.upenn.edu/biophys/PMLS/Student`.

option or <Cmd-O>, but you may need to tell Spyder to look for "all files" instead of just Python files.) You should see 16 lines of data, each containing two numbers separated by a comma. You can open the information file README.txt to find out what the numbers mean.

To load the data into a NumPy array, you can use the **np.loadtxt** command. This command attempts to read a text file and transform the data into an array. By default, it assumes that the data file is a .tsv file. To load a .csv file, you must instruct it to use a comma, not whitespace, as the separator. A look at the help file (try typing **np.loadtxt?**) shows that you can do this by including a *keyword argument* called *delimiter*. Keywords are optional arguments you can pass to a function to modify its behavior (see Section 1.4.4). To load the data, use the command

```
data_set = np.loadtxt("HIVseries.csv", delimiter=',')
```

Notice how this command is used:

1. The function returns an array. We assign the variable name data_set to this array.
2. We must give **np.loadtxt** the file name as a string.
3. We must specify the delimiter as a string, in this case ','.

Python will look for the file in the current working directory. You can see the working directory in a text box in the upper-right corner of the screen. If the file fails to load, either the file is located in the wrong folder or Spyder is looking in the wrong folder. If Spyder is no longer in its default working directory, you may wish to restart Spyder and try again. If the file is in the wrong location, move it to the working directory and try to load it again. An alternative to moving the file is to specify the complete **path** to the file. For example, on a Mac where the PMLSdata folder is still in the Downloads folder, you could write[4]

```
data_file = "/Users/username/Downloads/PMLSdata/01HIVseries/HIVseries.csv"
data_set = np.loadtxt(data_file, delimiter=',')
```

Path names are formatted differently in Windows:

```
data_file = r"c:\windows\Downloads\PMLSdata\01HIVseries\HIVseries.csv"
```

The modifier r before the string tells Python that this is a *raw string*. In raw strings, the backslash is interpreted literally, not as an escape character.

If you are working with multiple files outside the current working directory, specifying the complete path can lead to a lot of typing. Also, you might move the data to a different folder at some point in the future. For these reasons, it is often convenient to "define once, reuse often" in scripts. For example, with the following two commands near the top of the script, you can easily load multiple data files from the same folder:

```
home_dir = "/Users/username/"
data_dir = home_dir + "Downloads/PMLSdata/01HIVseries/"
data_set = np.loadtxt(data_dir + "HIVseries.csv", delimiter=',')
```

Later, if you work on a different computer (where home_dir will change) or move the data on your own computer (where data_dir will change), you will only have to modify one or two lines to get your code running again.

Once you have successfully imported the data, look at the array. It has two columns. You can see its size and its entries in the Variable Explorer, or by using the IPython Console. There is no description of what the numbers mean. For this, you need to consult README.txt.

[4] If you do not know a file's full path in Mac OS X, open a Terminal window. Find the file in Finder and drag it into the Terminal window. The file's path appears as plain text, which you can copy and paste into your code.

### T2 Other kinds of text

The data file you need to load may not be in any conveniently delimited format. One way to handle such files is to process the file line by line. The following will generate the same array as **np.loadtxt**, but it can be adapted to more exotic formats.

```
# import_text.py     [get code]
my_file = open("HIVseries.csv")
temp_data = []
for line in my_file:
    print(line)
    x, y = line.split(',')
    temp_data += [ (float(x), float(y)) ]
my_file.close()
data_set = np.array(temp_data)
```

The second line creates an object that can read the data file. During each iteration of the loop, the variable `line` is assigned a string value equal to the next line in the input file. The third line creates an empty list to store the data. (We use a list instead of an array because we do not know how many lines the file contains.) Inside the loop, Python displays the line being processed, and then uses the string method `line.split(',')` to break the line into numbers using the comma as a separator. Next, it converts the text into numbers and stores the data from each line as a new ordered pair at the end of the list of data points. The loop will terminate once it reaches the end of the file. After the loop exits, we close the file and convert the list of data points to a NumPy array.

The block of code within the **for** loop that processes the data file can be modified to extract data from other types of text files.

### T2 Direct import from the Web

Instead of saving a data set to your computer and then reading that file, you can combine these steps and read a file directly from the Web. Try the following:

```
import urllib
web_file = urllib.request.urlopen( "http://www.physics.upenn.edu/biophys/" + \
                                   "PMLS/Datasets/01HIVseries/HIVseries.csv" )
data_set = np.loadtxt(web_file, delimiter=',')
```

After the call to **urlopen**, Python can read the data in `web_file` by using **np.loadtxt** (or any other method for processing text files), as if the data set were located in a local file.[5]

## 3.2 EXPORTING DATA

Saving your work is always important when working on a computer. It is even more important when you are working in the IPython console.

*Data on the screen is fleeting. Data saved in files is permanent.*

If you have been working for hours and finally finish crunching numbers and making plots, great! But the moment you quit Spyder, all that work is gone. Only data you have saved to files will outlast your session.

[5] T2 In Python 2, the relevant command is **urllib.urlopen**.

### 3.2.1 Scripts

Scripts are a good way to save your work. If you keep adding lines to a script as you work through a problem, then you will have a record of your work *and* a program that will reconstruct the state of the system right up to the last line of the script. It can build and fill arrays, load data, carry out complicated analyses, create plots, and much more.

Each time you run a script, Spyder first saves it. Unless you have modified the global working directory in the Preferences, however, the default folder is probably not what you want. You can use `File>Save As...` to specify the destination. Be sure you know where your script is saved. (See Section 3.1.1 and Appendix A.)

You can also store all or part of your script in other files. Because scripts are plain text files, you can easily copy and paste code into a word processor, a homework assignment, your personal coding log, or an e-mail.

### 3.2.2 Data files

In contrast to scripts, Python does *not* automatically save its state or the values of any variables created during a session. Hence, the importance of writing scripts: They can reconstruct the state from scratch. However, with large data sets and complex analyses, running the script could take a long time. If you have already generated the data you need, you can save it to a file and load it the next time you want to use it.

#### Data to be read by Python

NumPy provides three convenient functions for saving data stored in arrays to a file:

- **`np.save`** – save a single array as a NumPy archive file (not human readable), with extension `.npy`
- **`np.savez`** – save multiple arrays as a single NumPy archive file (not human readable), with extension `.npz`
- **`np.savetxt`** – save a single array as a text file (human readable), with any extension

For example, the following code will store two arrays in five different files:

```
# save_load.py     [get code]
x = np.linspace(0, 1, 1001)
y = 3*np.sin(x)**3 - np.sin(x)

np.save('x_values', x)
np.save('y_values', y)
np.savetxt('x_values.dat', x)
np.savetxt('y_values.dat', y)
np.savez('xy_values', x_vals=x, y_vals=y)
```

Explore the resulting files, which have been created in the current working directory. Those created by **`np.save`** and **`np.savez`** have the extensions `.npy` and `.npz`, respectively, whereas those created by **`np.savetxt`** have the `.dat` extension we provided. (They contain tab-separated values. To generate comma-separated values, you must use the *`delimiter`* keyword again.)

You can view any of these files by using commands like **`%edit`** `x_values.npy` at the command prompt. Do not save any changes to a `.npy` or `.npz` file when you close it—that will corrupt the data. If you need to show your results to someone else, or use them outside of Python, use **`np.savetxt`**. As you can see, `.npy` and `.npz` files are not easy reading.

Jump to Contents Jump to Index

To recover the saved data at any point—in the current session or in the future—use **np.load** or **np.loadtxt**, depending on the file type:

```
x2 = np.load('x_values.npy')
y2 = np.loadtxt('y_values.dat')
w = np.load('xy_values.npz')
```

The variables `x2` and `y2` now point to arrays containing the same data as `x` and `y`. In the third example, `w` is not an array at all; however, our data is stored inside it.[6] If you type `w.keys()`, Python will return a list with two elements: the two strings `'x_vals'` and `'y_vals'`. These are the names we provided in the strange-looking assignment above when we called the **np.savez** command. We can access our data from `w` by using the keys. Executing the following lines demonstrates that the data have been faithfully saved and loaded:

```
x2 == x
y2 == y
w['x_vals'] == x
w['y_vals'] == y
```

From these examples, you can see that **np.savez** offers a convenient way to save several arrays in a single object. *Be sure you provide descriptive names for the data.* If you do not, NumPy will provide you with the unenlightening names "`arr_0, arr_1, ...`"

### Data to be read by humans

Not all the data you generate will be in arrays. Even with array data, sometimes you will need to write information to a text file, and you may want more control over its format than **np.savetxt** offers. (Perhaps the file will be read by a colleague, or an application other than Python.)

Recall Section 2.6.4, where snowball impact times were simply displayed on the console. When the session ends, this information will be lost unless you write it to a file. For small amounts of information, you can simply copy from the IPython console and paste into a text editor or word processor.

For large data sets, you will want to write data to a file *without* displaying it on the screen. Python's built-in function **open** will allow you to write directly to a file.[7] To illustrate the method, let's display the first ten powers of 2 and 3 *and* store this data for later use. The following script accomplishes both tasks:

```
# print_write.py        [get code]
my_file = open('power.txt', 'w')
print( " N \t\t2**N\t\t3**N" )           # Print labels for columns.
print( "---\t\t----\t\t----" )           # Print separator.
my_file.write( " N \t\t2**N\t\t3**N\n" )# Write labels to file.
my_file.write( "---\t\t----\t\t----\n" )# Write separator to file.
#%% Loop over integers from 0 to 10 and print/write results.
for N in range(11):
    print( "{:d}\t\t{:d}\t\t{:d}".format(N, pow(2,N), pow(3,N)) )
    my_file.write( "{:d}\t\t{:d}\t\t{:d}\n".format(N, pow(2,N), pow(3,N)) )
my_file.close()
```

[6] T2 It is a special object whose contents can be accessed like those of a Python **dictionary**. This tutorial won't discuss dictionaries, but they are a useful data type. See, for example, Libeskind-Hadas & Bush, 2014, or Jones, 2013.

[7] There is no `file` object type in Python 3, but `open('temp.txt')` returns an object similar to a Python 2 `file` object.

Compare the output displayed to the screen with the content of `power.txt`.

*Writing to a file is similar to printing to the display.*

The differences are that

- You must open a file before writing to it: `my_file=open('file.txt','w')`. (The `'w'` option means this file will be opened for writing. This erases any existing file of the same name *without* any warning or request for confirmation.)
- You must explicitly tell Python where you want a line to start and end. The character `'\n'` inserts a new line wherever it occurs in a string. (The escape sequence `'\t'` inserts a tab.)
- When you are done writing, you should close a file with the command `my_file.close()`.

Python is a powerful tool for reading, writing, and manipulating files, but we will not explore these capabilities any further. You can already accomplish quite a lot using the basic commands above in combination with the string formatting methods of Section 2.3.

What should you name your file? Python will accept any valid string you type, but your operating system may not be as flexible. It is good practice to restrict filenames to letters, numbers, underscore (_), hyphen (-), and period.

## 3.3 VISUALIZING DATA

With data in hand, we are ready for graphics. Python has no built-in graphing functions. Almost all of the graphing tools we discuss come from the PyPlot module, which is part of the larger Matplotlib module. To gain access to these functions, you must import the PyPlot module:

```
import matplotlib.pyplot as plt
```

### 3.3.1 The `plot` command and its relatives

PyPlot will make an ordinary, two-dimensional graph for you if you supply it with a set of $(x, y)$ pairs. It's up to you to space those points appropriately. Try

```
# simple_plot.py     [get code]
import numpy as np, matplotlib.pyplot as plt
num_points = 5
x_min, x_max = 0, 4
x_values = np.linspace(x_min, x_max, num_points)
y_values = x_values**2
plt.plot(x_values, y_values)
# plt.show()
```

The last line may be necessary in your Python environment. If no figure window opens, try uncommenting the last line or typing **`plt.show()`** at the command prompt.

The figure window may appear automatically, but it can still be hard to find. Try hiding your other applications and moving the Spyder window around. Your plot may be behind it. Changing to a different graphics *back end* may help.[8] (See Section A.2.2, page 110.)

[8] We describe a way to raise a specific figure window to the foreground using the Qt back end at `press.princeton.edu/titles/10644.html`.

The code given above is a basic plotting script. Notice what happens when you execute it:

- A new window appears. (See Section A.2 if it does not.) This is now the *current figure*. It will be called "Figure 1" if no other figure windows are open.
- Inside the current figure, PyPlot has drawn and labeled some axes. These are now the *current axes*.
- PyPlot has automatically drawn a region of the $xy$ plane that contains all the points you supplied.
- The **plot** function takes the items in its first argument one at a time, combines them with the corresponding items in the second argument to make $(x, y)$ pairs, and joins those points by a solid blue line. This graph is a bit jagged, but you can improve that by specifying a larger value for `num_points`. Try it.
- With some back ends, the figure window remains "live" until you close it. That is, you can continue to modify the figure by issuing additional commands at the IPython prompt. Other back ends may force you to close a figure before you can type any more commands.[9] See Section A.2.2 for instructions on how to change the back end.
- Changing the data in `y_values` will not automatically update your plot. You must issue another **plot** command to see such changes.
- You can close a figure window manually in the usual way (clicking on a red dot or "X" in the corner of the figure window), or from the IPython command prompt with the command **plt.close**`()`. You can close all open figure windows with the command **plt.close**`('all')`.

The two arrays that you supply to **plt.plot** must be of exactly the same length. That may sound obvious, but it's surprising how often you can get just one extra element in one of them. If you're not sure exactly how many elements you'll get from `a=`**np.arange**`(0,26,0.13)`, just take a few seconds to type **len**`(a)` at the IPython console prompt and find out. The function **len** is Python's built-in function for counting the number of elements in a data structure. You can also use **np.shape** or ask an array its own shape, as in `a.`**shape**.

*Get in the habit of checking the lengths of your arrays.*

Python's **assert** command provides a useful test. For the example above, you could insert a single statement before the plot command:

```
assert len(x_values) == len(y_values), \
    "Length-mismatch: {:d} versus {:d}".format(len(x_values), len(y_values))
plt.plot(x_values, y_values)
```

PyPlot will issue its own error message if you try to plot lists of unequal lengths, but an **assert** statement provides you with more information about the problem. To see the difference, rewrite the script to change the length of `x_values` after defining `y_values` and run the script with and without the **assert** statement.

You can avoid many of these "off-by-one" errors by adhering to the principle of "Define once, and reuse often." For example, if the length of all arrays is determined by the single parameter `num_points`, then they will all have the same length.

[9] The command **plt.show**`(block=`**False**`)` may prevent this, but you will need to type **plt.**`draw()` to update the figure after making changes from the command line.

### Plotting options

Every visual aspect of a graph can be changed from its default. You can specify these when you create a plot; most can also be changed later. Unless you only want to look at a rough plot of some data once and then discard the graph, you should write a script to generate your plot. Otherwise, you will find yourself retyping a lot of commands just to change a label or the color of a single curve in the plot. Here are some of the more common adjustments you can make:

- You can change the color of a curve when you first plot it by adding an optional argument: `plt.plot(x_values,y_values,'r')` produces a solid red line. Other line styles include
  - `'b'` (blue), `'k'` (black), `'g'` (green), and so on.
  - `':'` (join data points with a dotted line), `'--'` (dashed line), `'-'` (solid line), and so on. Remember that red, blue, and other colors look similar on a grayscale printout. Dotted and dashed lines are much easier to distinguish.
  - `'.'` (draw a small dot at each data point), `'o'` (open circle), and so on.

  To some extent, options can be combined: For example, `plt.plot(x_values,y_values,'r--o')` plots a red dashed line with red circles at each point. Python's default is to draw a solid line. If you specify a point style, but no line style, no line will be drawn.
- You can change Python's choice of plot region *after* creating the plot. Use the command `plt.xlim(1,6)` to display the region $1 \le x \le 6$, using the default for $y$. Similarly, `plt.ylim` adjusts the vertical region. Both functions will also accept keyword arguments,[10] or a list, tuple, or array containing the lowest and highest values to display on the axis: `plt.xlim(xmax=6, xmin=1)`, `plt.xlim([1,6])`, `plt.xlim((1,6))`.
- The command `plt.axis('tight')` effectively issues `xlim` and `ylim` commands to make the axes just fit the range of the data, without extra space around it.
- PyPlot automatically determines the height and width of the graph, and then scales the $x$ and $y$ values by different amounts to make everything fit into a frame with standard height and width. You may not like the resulting distortion of your figure. The command `plt.axis('equal')` forces each axis to use the same scaling: a circle will look like a circle, not an ellipse.

The documentation on `plt.plot` is quite helpful. Type `help(plt.plot)` or `plt.plot?` at the command prompt to read it. It describes the available marker and line styles, describes other adjustable parameters, and provides several examples.

Some of the commands just mentioned illustrate common themes: Plot attributes can be set by passing strings as arguments to `plt.plot`. Some must appear in a specific position in the list of arguments, like `'r'`. Alternatively, any attribute can be modified by pairing a keyword with the desired value, as in *linestyle*`='r'` or *ls*`='r--'`.

If you don't want to join your plotted points with a line, there is a separate function called `plt.scatter` that can be called instead of using the options listed above. It gives you more control over the size, style, and color of individual symbols than `plt.plot`.

### Log axes

If you'd prefer a logarithmic vertical axis, use `plt.semilogy` in place of `plt.plot`. You can also call `plt.semilogy()` after creating a plot. For a logarithmic horizontal axis, use `plt.semilogx`; for a log-log plot, use `plt.loglog`. In the two latter cases, you may want to evaluate your function

[10] Keyword arguments will be discussed in Section 5.1.3.

at a set of values that are uniformly spaced on a logarithmic scale. See `help(np.logspace)` for a function that creates such arrays.

### 3.3.2 Manipulate and embellish

There is no limit to the time you can spend improving your graphs. Here we introduce a few more useful adjustments. You can locate many others by using your favorite search engine. (Try searching the Web for "`python graph triangle symbol`".) Just remember: When you close a figure, your graph is gone. If you write a script to construct the figure and add all of your embellishments, you can easily reconstruct it later. The commands below may look strange at first, but you will get used to them with practice, especially if you record the ones you use in your coding log (see Section 1.2.4).

You see a plot as an image on your screen, but to Python, a plot is an *object* with many associated *attributes*. The object has *methods* for modifying those attributes. To adjust the properties of an existing figure, first create a variable linked to the plot object:

```
# graph_modifications.py     [get code]
ax = plt.gca()
```

In this command, `plt.gca()` (get current axes) returns the object that controls many of the attributes of the plot in the current figure. (It is called an `Axes` object.) It is fairly complex, with over 450 attributes and methods! However, you can accomplish a lot by using only a few of these. The commands fall into two general categories: *getting* data from the object and *setting* values of attributes that you want to adjust. These operations have fairly descriptive names, like `ax.get_xticks()` and `ax.set_xlabel()`. Many of these operations can also be accomplished with functions from the PyPlot module itself.

Here are a few useful operations:[11]

**Title:** You can add a title with `ax.set_title("My first plot")`. If you don't like the default font, you can change it by using a keyword and value:

```
ax.set_title("My first plot", size=24, weight='bold')
```

Alternatively, you may use

```
plt.title("My first plot", size=24, weight='bold')
```

**Axis labels:** You can—and should—label the axes with

```
ax.set_xlabel("speed")
ax.set_ylabel("kinetic energy")
```

or

```
plt.xlabel("speed")
plt.ylabel("kinetic energy")
```

Even better, you can—and should—include units: `ax.set_xlabel("speed [um/s]")`. Here "`um`" is an informal abbreviation for micrometer. But it's almost as easy to write `$\\mu$m`

[11] The following commands were tested on the `Qt4Agg` back end. They may not work correctly with all graphics back ends.

instead, which will produce the standard abbreviation μm. (See Section 2.3.) Even if you are reporting a dimensionless quantity, like concentration relative to its value at time zero, help your readers by stating "`c/c(0) [unitless]`" or "`concentration [a.u.]`."

**Font:** You can change the font and font size for the numbers labeling the axes by first creating the plot, then giving the commands

```
ax.set_xticklabels(ax.get_xticks(), family='monospace', fontsize=10)
ax.set_yticklabels(ax.get_yticks(), family='monospace', fontsize=10)
```

**Line style:** You can change the attributes of the plotted lines after the initial plot command has been executed. First, we need to gain access to the objects Python uses to draw the lines, then set their properties using PyPlot's **`plt.setp`** command:

```
lines = ax.get_lines()       # Lines is a list of line objects.
# Make the first line thick, dashed, and red.
plt.setp(lines[0], linestyle='--', linewidth=3, color='r')
```

**Legend:** You can add a **legend**—a descriptive label for each line. You can do this in three ways: Give each line a label when it is created by using the *`label`* keyword, use the **`set_label`** method of a line object, or use the **`legend`** method of an `Axes` object.

```
# Use "label" keyword to set labels when plotting.
plt.plot(x_values, y_values, label="Population 1")
plt.plot(x_values, x_values**3, label="Population 2")
plt.legend()

# Use line objects to set labels after plotting.
ax = plt.gca()
lines = ax.get_lines()
lines[0].set_label("Infected Population")
lines[1].set_label("Cured Population")
ax.legend()                                  # Display legend in plot.

# Use Axes object to set labels after plotting.
ax.legend( ("Healthy", "Recovered") )
```

No legend will appear in your plot until you call `ax.legend()` or `plt.legend()`.

These examples show that you can add a lot of useful information to a plot and make it visually appealing. You can also see that this involves a lot of typing. Once you have your plot looking the way you want—or once you get a single aspect of it looking the way you want—copy the commands you used into a script. At the end of the process, you will have a file that can generate the same wonderful graphics at any time on any computer that runs Python.

**Your Turn 3A**

Start with the code at the beginning of Section 3.3.1 and write a script to produce a figure with a smooth, thick red line. Add labels for the axes, a title, and a legend. Make the text big enough to read easily.

### 3.3.3 $T_2$ Error bars

To make a graph with error bars, use

```
plt.errorbar(x_values, y_values, yerr=y_errors, xerr=x_errors)
```

This function doesn't add error bars to an existing plot; rather, it creates a new plot (or a line within an existing plot) with error bars. Its syntax is similar to **plt.plot**, but it accepts additional optional arguments. Point `n` will be augmented by error bars that extend coordinate distance `y_errors[n]` above and below the point, and coordinate distance `x_errors[n]` to the left and right of the point. Both error arguments are optional. If you supply only one, there will be no error bars in the other direction. If you do not supply either, **plt.errorbar** behaves like **plt.plot**.

### 3.3.4 3D graphs

Sometimes a graph consists of points or a curve in a three-dimensional space. To make 3D plots, we must import an additional tool from one of the Matplotlib modules:

```
# line3d.py     [get code]
from mpl_toolkits.mplot3d import Axes3D     # Get 3D plotting tool.
```

We must also interact with PyPlot in a different way. The following script will generate a helix:

```
fig = plt.figure()                    # Create a new figure.
ax = Axes3D(fig)                      # Create 3D plotter attached to figure.
t = np.linspace(0, 5*np.pi, 501)      # Define parameter for parametric plot.
ax.plot(np.cos(t), np.sin(t), t)      # Draw 3D plot.
```

Notice that this time we are using the **plot** method attached to a specific object. We created an **Axes3D** object and can now use its methods to generate three-dimensional plots. The method **ax.plot** accepts three arrays, whose corresponding entries are interpreted as the $x$, $y$, and $z$ coordinates of the points to be drawn. The object generated by **Axes3D**`(fig)` can also create surface plots, three-dimensional contour plots, three-dimensional histograms, and much more.

Your screen is two-dimensional. If you ask for a "3D" plot, what you actually get must be a two-dimensional projection—what a camera would see looking at your plot from some outside **viewpoint**. Switching to a different viewpoint may make it easier to see what's going on. You may be able to click and drag within your plot to change the viewpoint. (This depends on the graphics back end you are using. See Section A.2.) If this is not possible, you can change the viewpoint from the IPython command prompt or in a script with the command `ax.`**view_init**`(elev=30,azim=30)`. You specify the viewpoint by giving the elevation and azimuth, in degrees—not radians.

### 3.3.5 Multiple plots

#### Multiple plots on the same axes

You probably noticed in the previous examples that every new **plt.plot** command adds a new curve to an existing figure, if one is already open.

*By default, a new plot is added to the axes in the active figure window.*

You can also create several curves on the same axes with a single plot command. Try

```
x = np.linspace(0, 1, 51)
y1 = np.exp(x)
y2 = x**2
plt.plot(x, y1, x, y2)
```

This example shows that you can give **plot** more than one set of $(x, y)$ pairs. Python will choose different colors for each curve, or you can specify them manually:

```
plt.plot(x, y1, 'r', x, y2, 'ko')
```

A third way to draw multiple curves on one set of axes is to give **plt.plot** an ordinary vector of $x$ values, but a two-dimensional array of $y$ values. Each column of `y` will be used to draw a separate curve using a common set of $x$ values. This approach can be useful when you wish to look at a function for several values of a parameter.

**Your Turn 3B**

Try the following code:

```
num_curves = 3
x = np.linspace(0, 1, 51)
y = np.zeros( (x.size, num_curves) )
for n in range(num_curves):
    y[:, n] = np.sin((n+1) * x * 2 * np.pi)
plt.plot(x, y)
```

Add a legend explaining which curve is which, then use the methods of Section 3.3.2 to embellish the plot.

### Replacing curves

Sometimes you do not need a new figure—you may simply want to replot a modified version of your data. You can remove all data and formatting from the current figure by calling **plt.cla()**, PyPlot's clear axes command, then use **plt.plot** again.

Alternatively, you can use the **hold** method of an `Axes` object:[12]

```
t = np.linspace(0, 5*np.pi, 501)        # Define parameter for parametric plot.
plt.plot(t, np.sin(t)**2)               # Generate plot.
plt.plot(t, np.sin(t)**3)               # Create new plot on same axes.
ax = plt.gca()                          # Gain control of axes.
ax.hold(False)                          # Turn hold off.
plt.plot(t, np.sin(t)**4)               # Create new plot; old data erased.
```

If you call `ax.hold()` without any arguments, it will **toggle** the hold property, that is, do the opposite of what it was doing before. When the hold property is **False**, Python will remove all data and formatting from your figure the next time you issue a plot command.

There is also a function **plt.hold** that can change the hold property of the current figure.

[12] The examples here are aimed at clarity rather than efficiency of typing. Once you are more familiar with Python, you will be able to combine multiple commands, as in **plt.gca().hold()**. Until that statement makes sense to you, it is best to stick to the less compact and more explicit commands given here.

### Multiple plot windows

If you want two or more separate plots open, use **plt.figure()** to open a new figure window and make it active, then use **plt.plot** to generate the second plot. If you don't supply an argument, **plt.figure()** chooses a figure number that is not in use yet. You can also assign a name you choose yourself. For example, **plt.figure('Joe')** will create a figure named "Joe" (or switch to it if it already exists). The next **plt.plot** command will draw its output in the active figure window, with no effect on other figure windows. Likewise, **plt.gca()** will return the `Axes` object of the active figure, so you can select "Joe" when you want to adjust its attributes.

If you are generating several plots in a session, you will probably want to create a new figure each time by using **plt.figure()**. However, these plots can use a fair amount of your computing resources, and finding the graph you want among a dozen figure windows is not always simple. Once you are finished with a figure, you can close it. You can use **plt.close()** to close the active figure, or you can close a figure by name: **plt.close('Joe')**.

*You can close* all *open figure windows with* **plt.close('all')**.

Include this command near the top of a script to ensure that all figures come from the latest run.

## 3.3.6 Subplots

You may wish to place multiple plots side by side in a single figure window for comparison. In Python jargon, you'd like multiple *axes* occupying the same *figure*. The function **plt.subplot**(M,N,p) divides the current figure window into a grid of cells with `M` rows and `N` columns, then makes cell number `p` the active one. Here, `p` must be an integer between 1 and `M*N`. (This is one case where Python does *not* start counting from 0.) Any graphing command issued now will go into cell number `p`. Try this example:

```
# subplots.py    [get code]
from numpy.random import random
t = np.linspace(0, 1, 101)
plt.figure()
plt.subplot(2, 2, 1); plt.hist(random(20))                        # Upper left
plt.subplot(2, 2, 2); plt.plot(t, t**2, t, t**3 - t)              # Upper right
plt.subplot(2, 2, 3); plt.plot(random(20), random(20), 'r*')      # Lower left
plt.subplot(2, 2, 4); plt.plot(t*np.cos(10*t), t*np.sin(10*t))    # Lower right
```

Each **plt.subplot** command selects a subregion of the figure. The subsequent **plt.plot** command sets up axes and draws a plot in the current subplot region. If your subplots do not fit nicely into the figure window or overlap one another, you can resize the figure window or try the command **plt.tight_layout()**.

Make sure all your **plt.subplot** commands for a particular figure use the same values for `N` and `M`. If you change either, all existing subplots will be erased. For more freedom in placing subplots and insets, see **help**(**plt.axes**).

## 3.3.7 Saving figures

All of your figures will disappear when you end your session in Spyder. If you used a script to create and adjust your plots, you can recreate them later, but you may also want to use plots in other

applications or print them out. Saving a plot to a file is straightforward. The figure window has a `Save` icon that will open a dialog allowing you to save the current figure in a variety of formats. The default is `.png`, but most Python distributions will also allow you to save the figure in `.pdf`, `.jpg`, `.eps`, or `.tif` format. To get the complete list, consult the figure:

```
fig = plt.gcf()                                  # Get current figure object.
fig.canvas.get_supported_filetypes()
```

All you need to do is give the figure a name with the appropriate extension; PyPlot will create a graphics file of the type you requested.

You can also save a figure from the IPython command prompt or from within a script by using the command **`plt.savefig`**. For example, to save the current figure as a `.pdf` file, type

```
plt.savefig("greatest_figure_ever.pdf")
```

The plot will be saved in your current working directory. (You can force Python to save in a different directory by giving the complete path, as discussed in Section 3.1.2, page 47.)

### 3.3.8 T2 Using figures in other applications

If you save a figure as an `.eps` or `.svg` file, you can open and modify it in a *vector-graphics* application such as Inkscape or Xfig (both freeware), or a commercial alternative.[13] Text such as the title or axis labels may be rendered as "outlines," however, and hence difficult to edit in another application. If you encounter this problem, you can instruct Python to save text as "type" in an `.svg` file. Before saving the figure, modify the parameter that controls how fonts are saved in `.svg` figures:

```
import matplotlib
matplotlib.rcParams['svg.fonttype'] = 'none'
```

Then, save your figure as an `.svg` file.

Some applications won't read `.svg` files prepared in this way. An alternative that still gives "type" is to use

```
import matplotlib
matplotlib.rcParams['pdf.fonttype'] = 42
```

Then, save your figure as a `.pdf` file, and edit it in another application. For more information about **`matplotlib.rcParams`**, see "Customizing matplotlib" at `matplotlib.org`.

If you are going to use figures in a presentation, you may instead need to save images in a *raster* format (also called **bitmap**), such as `.gif`, `.png`, `.jpg`, or `.tif`. For publication, however, vector art is best.

[13] Inkscape: `www.inkscape.org/en/`; Xfig: `www.xfig.org`.

# CHAPTER 5

# More Python Constructions

*At its highest level, numerical analysis is a mixture of science, art, and bar-room brawl.*

—T. W. Koerner, *The Pleasures of Counting*

The preceding chapters have developed a basic set of techniques for importing, creating, and modeling data sets and visualizing the results. This chapter introduces additional techniques for exploring mathematical models and their predictions:

- Random numbers and Monte Carlo simulations
- Solutions of nonlinear equations of a single variable
- Solutions of linear systems of equations
- Numerical integration of functions
- Numerical solution of ordinary differential equations

In addition, this chapter introduces several new methods for visualizing data, including histograms, surface plots, contour plots, vector field plots, and streamlines.

We start with a discussion of writing your own functions, an invaluable tool for exploring physical models.

## 5.1 WRITING YOUR OWN FUNCTIONS

Section 2.6.5 introduced a principle:

> *Don't duplicate. Define once, and reuse often.*

In the context of parameters (fixed quantities), this means you should define a parameter's value just once at the start of your code and refer to it by name throughout the rest of the program. But code itself can contain duplications if we wish to do the same (or nearly the same) task many times. Just as with parameter values, you may later realize that something needs to be changed in your code. Changing every instance of a recurring code fragment can be tedious and prone to error. It is better to define a *function* once, then invoke it whenever needed. You may even use the same fragment of code in more than one of your scripts. If each script imports the same externally defined function, then changes that you make once to the function file will apply to all of your scripts.

Functions in Python can do almost anything. They can carry out mathematical operations, they can make plots, they can read and write files, and much more. Your own functions can do all of these things as well. Functions are ideal for writing code once and reusing it often.

### 5.1.1 Defining functions in Python

A function can be defined at the command prompt or in a file. The following example is a basic template for creating a function in Python:

```
def taxicab(pointA, pointB):
    """
    Taxicab metric for computing distance between points A and B.
        pointA = (x1, y1)
        pointB = (x2, y2)
    Returns |x2-x1| + |y2-y1|. Distances are measured in city blocks.
    """
    interval = abs(pointB[0] - pointA[0]) + abs(pointB[1] - pointA[1])
    return interval
```

The "taxicab metric" determines the billable distance between points "as the cab drives" rather than "as the crow flies."

The function consists of the following elements:

**Declaration:** The keyword **def** tells Python you are about to define a function. Function names must adhere to the same rules as variable names. It is wise to give your functions descriptive names. Remember: You will only have to define it once, but you will wish to reuse it often. Now is not the time to save typing by giving it a forgettable name like `f`.

**Argument list:** The name of the function is followed by the names of all its **arguments**: the data it requires to do its calculation. In this case, `taxicab` requires two arguments. If it is called with only one argument or arguments of the wrong type, Python will raise a **TypeError** exception.

**Colon:** The colon after the list of arguments begins the block of code associated with the function. Notice the similarity with **for** and **while** loops and **if** statements. Everything from the colon to the end of the indented block of code will be executed when the function is called.

**Docstrings:** The text between the pair of triple-quotes (`""" ... """`) is a docstring. It is a special comment Python will use if someone asks for **help** on your function, and it is the standard place to describe what the function does and what arguments are required. Python will not complain if you do not provide a docstring, but someone who uses your code might.

**Body:** The body of the function is the code that does something useful with the arguments. This example is a simple function, so its body consists of just two lines. More complicated functions can include loops, **if** statements, and calls to other functions, and may extend for many lines.

**Return:** In Python, a function always returns something to the program that invokes it. If you do not specify a return value, Python will supply an object called **None**. In this example, our function returns a `float` object.

Now, let's look at how a function call works. Enter the function definition for `taxicab` at the command prompt. (You are allowed to omit the docstring, but only this one time!) Then, type

```
fare_rate = 0.40            # Fare rate in dollars per city block
start = (1, 2)
stop = (4, 5)
trip_cost = taxicab(start, stop) * fare_rate
```

When the fourth line is executed, `taxicab` behaves like **`np.sqrt`** and other predefined functions:

- First, Python assigns the variables in the function's argument list to the objects passed as arguments. It binds `pointA` to the same object as `start`, and `pointB` to the same object as `stop`.
- Then, Python transfers control to `taxicab`.
- When `taxicab` is finished, Python substitutes the return value of `taxicab(start,stop)` into the assignment statement for `trip_cost`.
- Python finishes evaluating the expression and assigns the answer to `trip_cost`.

Although we defined `start` and `stop` as tuples, we can call our new function with any objects that understand what `thing[0]` and `thing[1]` mean: lists, tuples, or NumPy arrays.

**Your Turn 5A**

Define a function to compute the distance between two points in three-dimensional space "as the crow flies." Give it a descriptive name and an informative docstring. See what happens when you do not provide the proper number or type of arguments, and ensure that using **`help`** on your function will enable a user to diagnose and resolve the issue.

In some ways, a function is similar to a script. It is a segment of code that is executed when asked for by name. Unlike a script, a function can be called by another script or function. The function communicates with the calling program by way of its arguments and its return value. After evaluating the function, Python discards all of its local variables, so remember:

*If a function performs a calculation,* **`return`** *the result.*

You can define functions at the IPython command prompt. However, if you plan to use a function more than once, you should save it in a file. You can place a single function in its own file such as `taxicab.py` and run this file as a script prior to using the function. (Running the file will define the function as if it had been entered at the command prompt.) You can also define a function within the script that uses it, as long as its definition gets executed before it is first called.

If you will be using the same function in multiple scripts and interactive sessions, you can create a **module** in which you define one or more functions in a single `.py` file. A module is a script that contains a collection of definitions and assignments. It can be imported into your session from the command prompt, or by placing an **`import`** command within a script. You can also import selectively, as described in Section 1.3.

Place `taxicab` and the function you wrote in Your Turn 5A (called, for example, `crow`) in a single file called `measurements.py` in your working directory. Then, type **`import`** `measurements`. You now have access to both functions under the names `measurements.taxicab` and `measurements.crow`. Type **`help`**`(measurements)` and **`help`**`(measurements.taxicab)` to see how Python uses the docstrings you provide.

Modules that you write and wish to use should be located in the same folder as your main script, which is probably the global working directory you specified during setup (as described in Sections 3.1.1 and A.2).[1]

[1] T2 If you know about "paths," then you can create folders that Python can access from any working directory.

### 5.1.2 Updating functions

If you have modified a function in a script or in your own module, you will need to instruct Python to use the newest versions. If the function is defined in a script, you can run the script by using the RUN button (if the script is open in the Editor) or the `%run` command.

If your function is part of a module you have imported, you will need to relaunch Spyder for the changes to take effect. Just typing `import my_module` again will *not* update the module. Alternatively, you can use a function called `reload` to update any module. This function is part of the `imp` module.[2] For example, if you modify the `taxicab` function in `measurements.py` after you import `measurements` as a module, you need to save the file and then type the following commands:

```
from imp import reload
reload(measurements)
```

Again, calling `import` will only cause Python to load a module *if it has not already been imported.*

> *You must save a module and then use* `reload` *to update functions in a module you have already imported.*

Reloading a module is usually only necessary in debugging. Once your functions are working properly, you need only import them once in a script or interactive session.

### 5.1.3 Arguments, keywords, and defaults

We have already seen that keyword arguments can modify the behavior of functions like **`plt.plot`**. You can also use keywords and give arguments default values in functions you write. The following example demonstrates both of these techniques:

```
# measurement.py     [get code]
def distance(pointA, pointB=(0, 0), metric='taxi'):
    """
    Return distance in city blocks between points A and B.
    If metric is 'taxi' (or omitted), use taxicab metric.
    Otherwise, use Euclidean distance.
        pointA = (x1, y1)
        pointB = (x2, y2)
    If pointB is omitted, use the origin.
    """
    if metric == 'taxi':
        interval = abs(pointB[0] - pointA[0]) + abs(pointB[1] - pointA[1])
    else:
        interval = np.sqrt( (pointB[0] - pointA[0])**2 \
                          + (pointB[1] - pointA[1])**2 )
    return interval
```

The `distance` function decides how to calculate the distance between points based on the value of `metric`. Both `pointB` and `metric` have default values. If the values for these arguments are passed to the function, it will use them; if not, it will use the defaults. Try

```
distance( (3, 4) )
```

[2] In Python 2.7, `reload` is a built-in function and does not need to be imported.

```
distance( (3, 4), (1, 2), 'euclid' )
distance( (3, 4), 'euclid' )
distance( pointB=(1, 2), metric='normal', pointA=(3, 4) )
```

The arguments to a function must either be in the correct order according to the function definition, as in the first two lines of this example, or be paired with a keyword, as in the last line. The third line results in an error because Python assigns the string literal `'euclid'` to the variable `pointB`.

### 5.1.4 Return values

A function can take any type or number of arguments—or none at all. The return value of a function is a single object, but that object may be a number, an array, a string, or a collection of objects in a tuple or list. A function will return an object called **`None`** if no other return value is specified.

Suppose that you wish to write a function to rotate a two-dimensional vector. What should the arguments be? What should the function return?

The arguments should include the vector to be rotated and an angle of rotation. The function should return the rotated vector. Here is a function that will accomplish the task:

```
# rotate.py     [get code]
def rotate_vector(vector, angle):
    """
    Rotate a two-dimensional vector through given angle.
        vector = (x,y)
        angle = rotation angle in radians (counterclockwise)
    Returns the image of a vector under rotation as a NumPy array.
    """
    rotation_matrix = np.array([[ np.cos(angle), -np.sin(angle) ],
                                [ np.sin(angle),  np.cos(angle) ]])
    return np.dot(rotation_matrix, vector)
```

This implementation of rotation creates a $2 \times 2$ matrix and then multiplies this matrix and the vector supplied as the first argument. The function does not modify the contents of `vector`. Instead, **`np.dot`** creates a new array, which is the return value of the function.

Python allows you to **unpack** compound return values—to assign the individual elements of an object to separate variables. Multiple assignments on a single line, such as `x,y=(1,2)`, are a simple example of unpacking. Python can unpack any iterable object: a tuple, a list, a string, or an array.

There are several ways to unpack an object. The following calls to `rotate_vector` all work, but the components of the rotated vector are split up in different ways:

```
vec = [1, 1]
theta = np.pi/2
r = rotate_vector(vec, theta)
x, y = rotate_vector(vec, theta)
_, z = rotate_vector(vec, theta)
first, *rest = rotate_vector(vec, theta)
```

After executing these commands, you should find that `r` is a NumPy array with two components. The other variables, `x`, `y`, and `z`, contain the individual components of the rotated vector. The **underscore**, `_`, is a dummy variable whose value is discarded. (Underscore is a special variable, whose value is the result of the most recent command. We are using it to temporarily store values that we do not

need.) In the final line, `first` contains the first element of the rotated vector and `rest` is a list that contains everything else.[3] In this case, `rest` only contains one element, but if the function returned an array with 100 elements, `first` would still contain only the first value and `rest` would be a list (*not* a NumPy array) containing the other 99 elements. You could convert it to an array with **`np.array`**`(rest)`.

### 5.1.5 Functional programming

Python offers programmers great flexibility in writing functions. We discuss many of these nuances in Appendix D. However, we strongly endorse the following guidelines for writing your own functions:

**1.** Pass data to a function only through its arguments.

**2.** Do not modify the arguments of a function.

**3.** Return the result of a calculation with a **`return`** statement.

Python allows you to circumvent all of these conventions, but your code will be easier to write, interpret, and debug if you adhere to them. Your functions will accept input (arguments) and produce output (a return value), without side effects.

A **side effect** is any effect on the computer's state other than returning a value. Avoiding side effects is the cornerstone of *functional programming*. Sometimes side effects are useful, but consider the alternatives when writing your own functions.

Unintended side effects on arrays can be particularly troublesome. Python does *not* automatically create a local copy of an array inside a function. As a result, a function can modify the data in an array using array methods—including assignment of individual elements. (See Section 2.2.6.) Array methods like `x.sort()` and `x.fill(3)`, as well as in-place arithmetic like `x+=1` and `x*=2`, can also modify array data. Such side effects have the potential to speed up your code and reduce memory usage for operations on large arrays. You can **overwrite** the array and modify its elements without making any copies. However, this increase in performance comes at the cost of difficulty in debugging.

If you really do intend for a function to modify the elements of an array, you can still avoid side effects. Create a local copy of the array within the function, operate on the copy, and return the new array. Alternatively, you can create an empty array within the function, fill it with new values, and return the new array. NumPy adheres to this principle: Functions like **`np.cos`**`(x)` and **`np.exp`**`(x)` return new arrays, with no effect on `x`. If you wish, you can replace the original array with the new array returned by the function in the main code, rather than inside the function.

The following function demonstrates the principles of functional programming:

```
# average.py     [get code]
def running_average(x):
    """
    Return cumulative average of an array.
    """
    y = np.zeros(len(x))                    # Empty array to store result
    current_sum = 0.0                       # Running sum of elements of x
    for i in range(len(x)):
        current_sum += x[i]                 # Increment sum.
        y[i] = current_sum / (i + 1.0)      # Update running average.
    return y
```

[3] This type of unpacking is not available in Python 2.7.

The array being processed is passed as an argument. This array is not modified; the result of the calculation is returned in a new array.

If you need the extra performance and reduced memory usage that come from overwriting an array, use the information in Appendix D to plan your code carefully and avoid unintended consequences. However, try vectorizing your code using NumPy's efficient array operations before you resort to overwriting. Vectorized code will almost always run faster than loops you write yourself in Python.

## 5.2 RANDOM NUMBERS AND SIMULATION

There are many interesting problems in which we do not have complete knowledge of a system, but we do know the probabilities of the outcomes of simple events. For example, you know the probability of any given outcome in the roll of a single die is 1/6, but do you know how likely it is that the sum of a roll of 5 dice is less than 13? Rather than work out the combinatorics, you could roll 5 dice many times and determine the probability empirically.

A random number generator makes it possible for a computer to "roll dice" millions of times per second. Thus, you can use a random number generator to simulate a system described by a stochastic model in which the probability distributions of the parameters are known. You can determine the likely behavior of the system, even if you cannot you can't work out the details analytically. Such calculations are often called "Monte Carlo simulations."

### 5.2.1 Simulating coin flips

The simplest example of a stochastic system is a coin flip. Suppose that you want to simulate flipping a coin 100 times, record the number of heads or tails, and then repeat the series of 100 flips $N$ times. This will generate a set of $N$ numbers, each falling in the range of 0 to 100, inclusive. How do we get Python to flip the coin for us? First, try typing `1>2` at the IPython console prompt, and then `2>1`. You'll see that Python returns a Boolean value of **`True`** or **`False`** when it evaluates each proposition. You can simulate a coin flip by generating a uniformly distributed random number between 0 and 1, then checking to see if it is less than 0.5. If the comparison returns **`True`**, we record heads; if **`False`**, we record tails. Python can also use such values in numeric calculations: It converts **`True`** to 1 and **`False`** to 0.

The module **`np.random`** contains random number generators for several different probability distributions. The function **`np.random.random`** generates numbers from the "continuous uniform distribution" over the interval $[0, 1)$, and will serve our purposes for this chapter. You should explore the other functions in the **`np.random`** module as well. To save typing, we will import **`np.random.random`** under the nickname `rng`:

```
from numpy.random import random as rng
```

In the future, if you wish to switch to a different random number generator, this is the only line of code you will need to change: Just import a different function under the name `rng`.

To make a series of independent flips, first create an array called `samples` that contains 100 random numbers generated by `rng`. (Consult **`help`**`(rng)` for a clue about how to do this easily.) You can convert the random samples to a simulation of coin flips with `flips=(samples<0.5)`. Python applies the comparison item by item and stores the result in `flips`. You can then count the number

of heads by using `np.sum(flips)` or the array method `flips.sum()`. Repeat this several times to get a feel for how likely it is that exactly 50 heads occur in 100 flips of a fair coin.

### 5.2.2 Generating trajectories

We can adapt coin flipping to study random walks, Brownian motion, and a host of other interesting biological and physical systems. Let's create a random walk of 500 steps. Our trajectory will consist of 500 $x$ values and 500 $y$ values. Following good practice (Section 2.6.5), begin with

```
num_steps = 500
```

The idea behind our random walk is that steps along the $x$ and $y$ directions are statistically independent. You know from the preceding section how to get an array containing 500 random binary digits. Make two such arrays called `x_step` and `y_step`. For a coin flip, **`True`** and **`False`** or 1 and 0 were sufficient. For a random walk, we need a random string of $+1$ and $-1$ values.

**Your Turn 5B**

a. Do a simple operation on `x_step` to convert its entries to $+1$ or $-1$. Do the same for `y_step`.
b. Next, convert these arrays into successive positions of the random walker. Consult **`help(np.cumsum)`** to see why that function is useful.
c. Write a script to make a picture of your random walk, and run it several times.

Try writing a function called `get_trajectory` that will take the single argument `num_steps` and return the two arrays that contain the coordinates of a random walker.

## 5.3 HISTOGRAMS AND BAR GRAPHS

A **histogram** is a graphical representation of a discrete probability distribution. Suppose that you wish to check that the random number generator `rng` really gives a uniform distribution. To make a simple histogram, try

```
# histogram.py     [get code]
from numpy.random import random as rng
data = rng(100)
plt.hist(data)
```

A histogram appears, but a number of things have happened without your supervision. The function **`plt.hist`** has

- Inspected the list of data and determined its range;
- Divided that range into a number of equally spaced **bins**;
- Counted how many elements of `data` fall into each bin;
- Graphed the result as a set of bars; and
- Returned a tuple of two arrays and a strange-looking item called `<a list of 10 Patch objects>`.

Consulting **help**(**plt.hist**) reveals the content of the return values as well as the keywords and defaults that you can use to control the output. (See Section 5.1.3 above.) For instance, *bins*=10 means that **plt.hist** will generate 10 equally spaced bars in the histogram unless you specify another value, and *align*='mid' specifies that the bars will be centered on the midpoint of each bin by default. Compare the output when you specify 10, 100, and 1000 bins. For more control, you can provide the range over which you wish to bin the data, or you can specify the edges of each bin yourself. (If you specify the bin edges or a range, then Python will ignore any data points that fall outside the range you provide.)

For control over presentation or for further analysis, you can get the counts and bins used to generate the plot by unpacking the return value of **plt.hist**. (See Section 5.1.4 for details.)

```
counts, bin_edges, _ = plt.hist(data)
```

If you do not wish to generate a plot and only want the histogram data, you can use **np.histogram** instead. It is the same function that **plt.hist** uses to generate data for plotting:

```
counts, bin_edges = np.histogram(data)
```

**Your Turn 5C**

Try it, and inspect the return values to make sure you understand how these functions generate a histogram. Note, in particular, the numbers of elements in the two arrays.

Now that we've got the binned data, we can plot it in another style or transform it prior to plotting. The **plt.hist** (or **np.histogram**) function has simply done the sorting and counting for us. For example, we can make a custom bar graph by using **plt.bar**. The arguments of this function are an array of left edges, an array of heights, and a single width or an array of widths, which **plt.bar** uses to draw rectangles. All the arrays must be the same length, so we have to discard the last element of bin_edges returned by **plt.hist** or **np.histogram**, which is the right-hand edge of the last bin. The following code uses **plt.bar** to generate a colorful plot of the data, in which the width of a bar in the histogram is proportional to its height:

```
bin_size = bin_edges[1] - bin_edges[0]
new_widths =  bin_size * counts / counts.max()
plt.bar(bin_edges[:-1], counts, width=new_widths, color=['r','g','b'])
```

Sometimes it is necessary to specify explicitly the range of values that each bin collects instead of the number of bins we want. Both **plt.hist** and **np.histogram** can easily accommodate this. Instead of passing an integer number of bins to the function, simply pass an array whose entries are the edges of the bins. For example, to divide the random numbers into bins that span an inverse power of 2, such as $0, \frac{1}{128}, \frac{1}{64}, \frac{1}{32}, \ldots, \frac{1}{2}, 1$, we can use **np.logspace** to generate the bins:

```
log2bins = np.logspace(-8, 0, num=9, base=2)
log2bins[0] = 0.0                 # Set first bin edge to zero instead of 1/256.
plt.hist(data, bins=log2bins)
```

## 5.4 CONTOUR PLOTS AND SURFACES

In earlier chapters, we saw several methods for plotting data sets with a single independent variable, such as a time series. Such data lend themselves to two-dimensional plots. However, models with two or more independent variables require higher dimensional plots.

A function of two variables, $h(x, y)$, can be interpreted as a surface whose height over each point $(x, y)$ has been specified, just as Earth's topography is specified by the altitude at a given latitude and longitude. Two useful graphical representations of the height are contour plots and surface plots. A contour plot is a two-dimensional drawing in which contour lines or color are used to represent the height, as in a topographic map. A surface plot is a 3D perspective drawing of the surface itself.

### 5.4.1 Generating a grid of points

In order to make a plot of $h(x, y)$, you must specify the heights at a finite set of points. Typically you'll want those points to form a grid in the $xy$ plane. Python gives a convenient way to construct such a grid: First, set up a 1D array of $x$ values covering the desired range and an analogous 1D array of $y$ values. Then, call the function **np.meshgrid** to create the grid. If the first array has $N$ entries and the second has $M$ entries, then **np.meshgrid** will return two arrays, each with $N \times M$ entries. The arrays contain the $x$ and $y$ coordinates of each grid point, respectively. Try

```
x_vals = np.linspace(-3, 3, 21)
y_vals = np.linspace(0, 10, 11)
X, Y = np.meshgrid(x_vals, y_vals)
```

Inspect `X` and `Y` to make sure you understand the result: **np.meshgrid** returned a list containing two arrays, which we unpacked and assigned to `X` and `Y`. Each is an array with $21 \times 11 = 231$ entries giving the $x$ and $y$ coordinates of the grid points.[4] You can now evaluate a function on these arrays, perhaps by using vectorized math, to produce a third array called `Z`, and then create contour or surface plots using `(X,Y,Z)`. Sometimes it is useful to create the intermediate array of distances, `R=np.sqrt(X**2+Y**2)`, if the height only depends on the distance from the origin.

### 5.4.2 Contour plots

Suppose that we wish to visualize the function $z(x, y) = \cos x \sin y$. Creating a contour plot is simple using the arrays `X` and `Y` generated above:

```
Z = np.cos(X) * np.sin(Y)
plt.contour(X, Y, Z)
```

The number of contour lines is 10 by default. To change this, add a fourth argument to the function call: **plt.contour**`(X,Y,Z,20)` . Instead of an integer, you can also pass an array that contains the "heights" ($z$ values) for which you want to see contours. You can control the appearance of contour lines by using keywords, and you can even label the contour lines with the following commands:

```
# contour.py     [get code]
cs = plt.contour(X, Y, Z, 10, linewidths=3, colors='k')
plt.clabel(cs,fontsize=10)
```

[4] Python sets `X[i,j]=x_vals[i]` and `Y[i,j]=y_vals[j]`.

The **plt.contour** function returns a ContourSet object, and the **plt.clabel** command knows how to add labels to the contours in this object.

The **plt.contour** function draws only contour lines. There is a related function called **plt.contourf** that will draw filled contours. You can change the set of colors used by PyPlot via the command **plt.set_cmap**. If you enter the name of a nonexistent color map, Python will return an error that lists all of the available color maps.

### 5.4.3 Surface plots

Creating a surface is similar to creating a contour plot. The arguments to the functions are the same, but PyPlot has to access functions from a different module in order to create three-dimensional graphics. Recall Section 3.3.4:

```
from mpl_toolkits.mplot3d import Axes3D      # Import 3D plotting tool.
```

An **Axes3D** object can draw a variety of three-dimensional plots. For example, the following commands will create a surface plot:

```
ax = Axes3D( plt.figure() )      # Create 3D plotter attached to new figure.
ax.plot_surface(X, Y, Z)         # Generate 3D plot.
```

It takes a lot of computational resources to create a three-dimensional plot, so Python tries to use a small number of points when generating a surface. Its default is to use just one percent of the points you provide by skipping ten rows and ten columns at a time between points. This is good if you have a huge array, but if you have just 20 points along each direction, your surface will probably look terrible. To use all of your points, supply the keyword arguments *rstride* and *cstride* ("row stride" and "column stride") with the value 1:

```
ax.plot_surface(X, Y, Z, rstride=1, cstride=1)
```

**Your Turn 5D**

Make a surface plot of the function $z = x^2 + y^2$ over a suitable grid of values, where $x$ and $y$ range from $-1$ to 1.

The default of **plot_surface** is to use a single color and shading from imaginary sources of light to render a surface. You can get a more colorful plot by supplying a color map with the *cmap* keyword. If you will be printing an assignment in grayscale, you may get better results if you use a colorless color map like `'gray'` or `'bone'`.

## 5.5 NUMERICAL SOLUTION OF NONLINEAR EQUATIONS

It is often necessary to solve nonlinear equations when studying physical and biological systems. For example, determining the fixed points in a single-gene toggle system requires finding the roots of a sixth-order polynomial. There are no general analytic solutions for polynomials of degree greater than four, and even for cubic equations the formulas are cumbersome. Biological oscillators may involve still more complicated functions that are not even polynomials. Numerical methods for finding the roots of such expressions are quite useful.

### 5.5.1 General real functions

The methods developed to find the roots of equations are closely related to methods for optimizing functions. Neither Python nor NumPy has a collection of optimization tools, but SciPy has an extensive library called `scipy.optimize`.[5] To gain access to the functions in this module, we need to import them. You can import the entire module:

```
import scipy.optimize as so
```

The command `dir(so)` will tell you the names of all of the functions available in this module, and `help` will provide the details. However, we will not need most of the functions in this library, so we will simply import the individual functions as they are required.

A useful function for finding roots is called `fsolve`. It takes two arguments: a function name and a point (or array of points) at which `fsolve` begins searching for roots.

You can define the function to be solved by using the methods described in Section 5.1. If you need to solve a relation like $g(x) = 7$, define your function as $f(x) = g(x) - 7$. The `fsolve` function determines the zeroes of whatever function you provide.

While `fsolve` is quite good at finding roots, it will do even better if you help it. You can provide an array of points *near* the zeros of a function, then have `fsolve` refine these. A good way to generate initial guesses is to plot the function and estimate where it crosses the $x$ axis.

***Example:*** Try the following and explain the results:

```
from scipy.optimize import fsolve
def f(x): return x**2 - 1
fsolve(f, 0.5)
fsolve(f, -0.5)
fsolve(f, [-0.5, 0.5])
```

*Solution:* The equation $x^2 - 1 = 0$ has two roots, but which root `fsolve` returns depends on where we tell the function to begin searching. Providing an array of starting points yields an array of roots found from those starting points.

***Example:*** Now try the following, and explain what you see:

```
def f(x): return np.sin(x)**10
fsolve(np.sin, 1)
fsolve(f, 1)
```

*Solution:* The last expression above does not return exactly 0. What you are seeing is an example of *numerical error*, caused by the finite number of digits that Python uses to represent a number. The result is a number that is close the correct solution, but not exact.

The phrase "numerical error" is standard, but unfortunate. *You* didn't make any error; neither did Python. Your computer's hardware simply has limited precision.

A more serious type of error can occur when searching for the roots of a function with a singularity. Inspect the output of `fsolve(f,2)` when $f(x) = 1/(x-1)$. This function has a discontinuity at $x = 1$ and never crosses the $x$ axis. However, `fsolve` returns a solution without raising any errors or warnings. We can learn more by using a keyword to instruct `fsolve` to provide more information:

[5] T2 Its functions are adapted from the highly optimized MINPACK library of FORTRAN functions.

```
def f(x): return 1/(x-1)
fsolve(f, 2, full_output=True)
```

This time, the value of the root is returned along with an object that contains a lot more information. You can refer to `help(fsolve)` to learn what the terms mean, but the last line is a sure sign of trouble: The function reached its maximum number of evaluations without satisfying its requirements for a root of the equation. If your numerical result doesn't make sense, set `full_output=True` and consult `help(fsolve)`.

### 5.5.2 Complex roots of polynomials

The solutions returned by `fsolve` are restricted to be real numbers. Consider the equation $1/x = 1+x^3$. In order to use `fsolve`, we must manipulate this into $f(x) = x(1 + x^3) - 1$ and solve for the roots. Assume that we have some good reason to guess that the real roots are near 1 and $-1$:

```
def f(x): return x * (1 + x**3) - 1
fsolve(f, 1)
fsolve(f, -1)
```

A quartic equation has four roots; but, in the present case, `fsolve` can only find two roots because the other two are complex.[6]

You can find all of the roots—real or complex—of any *polynomial* by using the NumPy function `np.roots`. This function takes a vector of coefficients that define a polynomial (see `help(np.roots)` for details). You can find all of the roots of the polynomial above with `np.roots([1,0,0,1,-1])`.

For arbitrary nonlinear equations (those that are not polynomials) with complex roots, Wolfram Alpha and *Mathematica* are reliable tools. For example, the function $\frac{1}{x} = 1 + x^{2.4}$ has three roots, but only one real root. It cannot be completely solved by `fsolve`.[7]

## 5.6 SOLVING SYSTEMS OF LINEAR EQUATIONS

Often we need to solve a set of simultaneous linear equations. For example, a least-squares fit of a two-parameter linear model to a data set involves solving equations of the general form

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

We are given (or can find) the $a$'s and $c$'s, and wish to know the $x$'s.

Python and NumPy do not provide a collection of linear algebra tools, but SciPy provides a library for matrix mathematics called `scipy.linalg`. Its functions are adapted from the highly optimized LAPACK library of FORTRAN functions. To gain access to the functions in this module, we need to import them. You can import the entire module. A convenient and memorable nickname for SciPy linear algebra is `sla`:

```
import scipy.linalg as sla
```

[6] One way to see this is to use a symbolic math program like Wolfram Alpha: `solve(x*(1+x^3)-1,x)` . See Section 7.3.
[7] Actually, you *can* solve this particular equation by using `np.roots`. Rewrite it as $x^{3.4} = 1 - x$, raise both sides to the fifth power to generate a 17th-order polynomial, get all 17 roots from `np.roots`, then use a `for` loop to substitute them into the original expression and discard the spurious solutions.

To learn more about the module and the functions it provides, use `dir(sla)` and `help`. Since we will not need most of the functions, we will simply import individual functions as they are required. Some of the more commonly used functions are

| | |
|---|---|
| `inv` | matrix inverse |
| `det` | determinant |
| `sqrtm` | matrix square root |
| `expm` | matrix exponentiation |
| `eig` | eigenvalues and eigenvectors of a matrix |
| `eigh` | eigenvalues and eigenvectors of a Hermitian matrix |
| `svd` | singular value decomposition |

To solve the linear system above, we need to compute the inverse of the matrix C and multiply the vector $\boldsymbol{a}$ by this matrix.

```
# matrix_inversion.py     [get code]
from scipy.linalg import inv
a = np.array( [-1, 5] )
C = np.array( [ [1, 3], [3, 4] ] )
x = np.dot(inv(C), a)
```

This code puts the solution to the problem into the array `x`, as you can confirm by computing `np.dot(C,x)-a`. The order of multiplication is important. The solution of $\boldsymbol{a} = \mathsf{C} \cdot \boldsymbol{x}$ is obtained by multiplying both sides on the left by $\mathsf{C}^{-1}$.

## 5.7 NUMERICAL INTEGRATION

Integrating a function is a common task in physical modeling. For example, you may wish to integrate a probability density function to determine the probability that its variable will fall within a certain range of values.

As with linear algebra, Python and NumPy do not support numerical integration, but SciPy does. The module is called `scipy.integrate`.

```
import scipy.integrate as si
```

The command `dir(si)` will tell you the names of all of the functions in this module, and `help` will provide more information. However, we will not need most of the functions in the library, so we will simply import the individual functions as they are required.

There are several integration routines available in the module, each with particular strengths. The all-purpose workhorse is called `quad`, adapted from the highly optimized QUADPACK library of FORTRAN functions.

### 5.7.1 Integrating a predefined function

To carry out a numerical integration, you must provide a function name for the integrand and the limits of integration. Optional arguments allow more control over the integration. To see how this works, let's evaluate

$$\int_0^{x_{\max}} \mathrm{d}x \, \cos(x)$$